

第三、四问支撑材料附录

本文件为合并到最终论文的附录素材，内容含文件清单和完整源码。独立页码仅供本包查阅；整合论文时应接续论文页码。

1 第三、四问支撑材料清单

本清单覆盖第三、四问。第三问采用 v6，第四问采用 M32 joint。正式加密日志各三份，原文件名和内容保持不变。完整源码见后续附录。压缩包包含本清单所列文件，但不包含压缩包自身；哈希清单不计算自身和外层压缩包。第一、二问清单在整合时另行加入。

1. .gitignore
2. AI工具使用详情.md
3. AI工具使用详情.pdf
4. README.md
5. SHA256SUMS.json
6. formal-results.json
7. local-simulator-snapshot/README.md
8. local-simulator-snapshot/b_local_simulator.py
9. local-simulator-snapshot/calibrated_profile.json
10. local-simulator-snapshot/defaults.json
11. local-simulator-snapshot/requirements.txt
12. local-simulator-snapshot/run_local.py
13. q3/analysis/analyze.py
14. q3/analysis/data/analysis.json
15. q3/analysis/data/official_groups.json
16. q3/analysis/data/official_literature_results.json
17. q3/analysis/data/optical_events.jsonl
18. q3/analysis/data/optical_example.json
19. q3/analysis/data/optical_provenance.json
20. q3/analysis/data/paired_results.json
21. q3/analysis/evaluation_protocol.json
22. q3/analysis/figures/q3-optical.pdf
23. q3/analysis/figures/q3-optical.png
24. q3/analysis/figures/q3-optical.svg
25. q3/analysis/figures/q3-performance.pdf

26. q3/analysis/figures/q3-performance.png
27. q3/analysis/figures/q3-performance.svg
28. q3/analysis/figures/q3-route-coverage.pdf
29. q3/analysis/figures/q3-route-coverage.png
30. q3/analysis/figures/q3-route-coverage.svg
31. q3/analysis/official/history-v4-v6.csv
32. q3/analysis/official/results.json
33. q3/analysis/official/statistics.json
34. q3/analysis/official/v6-extra-30.csv
35. q3/analysis/optical_revision_manifest.json
36. q3/analysis/paired/baseline/2026092410-actions.json
37. q3/analysis/paired/baseline/fixtures.json
38. q3/analysis/paired/baseline/manifest.json
39. q3/analysis/paired/baseline/results.json
40. q3/analysis/paired/v6/2026092410-actions.json
41. q3/analysis/paired/v6/fixtures.json
42. q3/analysis/paired/v6/manifest.json
43. q3/analysis/paired/v6/results.json
44. q3/analysis/replot.py
45. q3/analysis/replot_optical.py
46. q3/baselines/CG/api_client.py
47. q3/baselines/CG/bearing_geometry.py
48. q3/baselines/CG/config.example.json
49. q3/baselines/CG/legacy_strategy.py
50. q3/baselines/CG/policies.json
51. q3/baselines/CG/run_robot.py
52. q3/baselines/CG/strategy.py
53. q3/baselines/v4/api_client.py
54. q3/baselines/v4/bearing_geometry.py
55. q3/baselines/v4/clearance_geometry.py
56. q3/baselines/v4/combined_cost_cover.py
57. q3/baselines/v4/config.example.json

- 58. q3/baselines/v4/convex_cover.py
- 59. q3/baselines/v4/cost_aware.py
- 60. q3/baselines/v4/exact_coverage.py
- 61. q3/baselines/v4/joint_strategies.py
- 62. q3/baselines/v4/legacy_strategy.py
- 63. q3/baselines/v4/merged_cover.py
- 64. q3/baselines/v4/policies.json
- 65. q3/baselines/v4/reception_region.py
- 66. q3/baselines/v4/run_robot.py
- 67. q3/baselines/v4/strategy.py
- 68. q3/formal/logs/formal-p3-1-BXUJ-W9QZ-KQ54-NKCU.jlog
- 69. q3/formal/logs/formal-p3-2-QNXC-8U8P-SQBF-A2A6.jlog
- 70. q3/formal/logs/formal-p3-3-89UC-2392-TZ5M-ZUTB.jlog
- 71. q3/formal/records/formal-p3-1-BXUJ-W9QZ-KQ54-NKCU.actions.jsonl
- 72. q3/formal/records/formal-p3-1-BXUJ-W9QZ-KQ54-NKCU.result.json
- 73. q3/formal/records/formal-p3-2-QNXC-8U8P-SQBF-A2A6.actions.jsonl
- 74. q3/formal/records/formal-p3-2-QNXC-8U8P-SQBF-A2A6.result.json
- 75. q3/formal/records/formal-p3-3-89UC-2392-TZ5M-ZUTB.actions.jsonl
- 76. q3/formal/records/formal-p3-3-89UC-2392-TZ5M-ZUTB.result.json
- 77. q3/formal/results.json
- 78. q3/v6/action_value.py
- 79. q3/v6/api_client.py
- 80. q3/v6/bearing_geometry.py
- 81. q3/v6/belief_components.py
- 82. q3/v6/clearance_geometry.py
- 83. q3/v6/combined_cost_cover.py
- 84. q3/v6/config.example.json
- 85. q3/v6/convex_cover.py
- 86. q3/v6/cost_aware.py
- 87. q3/v6/cover_milp.py
- 88. q3/v6/exact_coverage.py
- 89. q3/v6/flexible_cover.py

90. q3/v6/joint_strategies.py
91. q3/v6/legacy_strategy.py
92. q3/v6/lookahead.py
93. q3/v6/merged_cover.py
94. q3/v6/policies.json
95. q3/v6/reception_region.py
96. q3/v6/run_robot.py
97. q3/v6/service_routing.py
98. q3/v6/strategy.py
99. q3/v6/voronoi_coverage.py
100. q4/M32/api_client.py
101. q4/M32/base.py
102. q4/M32/bearing_geometry.py
103. q4/M32/config.example.json
104. q4/M32/coverage.py
105. q4/M32/joint_planner.py
106. q4/M32/joint_region.py
107. q4/M32/m06.py
108. q4/M32/m07.py
109. q4/M32/m13.py
110. q4/M32/optimize.py
111. q4/M32/policies.json
112. q4/M32/posterior.py
113. q4/M32/previous.py
114. q4/M32/research_strategy.py
115. q4/M32/ring.json
116. q4/M32/run_robot.py
117. q4/M32/strategy.py
118. q4/analysis/evidence/coverage_audit.json
119. q4/analysis/evidence/delivery_audit.json
120. q4/analysis/evidence/formal_results.json
121. q4/analysis/evidence/frozen_strategy_hashes.json

- 122. q4/analysis/evidence/joint_diagnostics.json
- 123. q4/analysis/evidence/joint_event_metrics.json
- 124. q4/analysis/evidence/joint_example.json
- 125. q4/analysis/evidence/latex_numbers.json
- 126. q4/analysis/evidence/local-cautious-manifest.json
- 127. q4/analysis/evidence/local-m07-manifest.json
- 128. q4/analysis/evidence/local-m32-manifest.json
- 129. q4/analysis/evidence/manuscript_audit.json
- 130. q4/analysis/evidence/new-official-batch-status.json
- 131. q4/analysis/evidence/official_90.csv
- 132. q4/analysis/evidence/official_diagnostics.json
- 133. q4/analysis/evidence/official_differences.json
- 134. q4/analysis/evidence/official_rows.json
- 135. q4/analysis/evidence/official_statistics.json
- 136. q4/analysis/evidence/paired_actions.json
- 137. q4/analysis/evidence/paired_analysis.json
- 138. q4/analysis/evidence/paired_rows.json
- 139. q4/analysis/evidence/paired_scene.json
- 140. q4/analysis/evidence/ring_points.json
- 141. q4/analysis/evidence/source_hashes.json
- 142. q4/analysis/figures/fig01-direction-coverage.pdf
- 143. q4/analysis/figures/fig01-direction-coverage.png
- 144. q4/analysis/figures/fig01-direction-coverage.svg
- 145. q4/analysis/figures/fig02-joint-domain.pdf
- 146. q4/analysis/figures/fig02-joint-domain.png
- 147. q4/analysis/figures/fig02-joint-domain.svg
- 148. q4/analysis/figures/fig03-official-comparison.pdf
- 149. q4/analysis/figures/fig03-official-comparison.png
- 150. q4/analysis/figures/fig03-official-comparison.svg
- 151. q4/analysis/figures/fig04-paired-paths.pdf
- 152. q4/analysis/figures/fig04-paired-paths.png
- 153. q4/analysis/figures/fig04-paired-paths.svg

- 154. q4/analysis/redraw_figures.py
- 155. q4/baselines/CG/adapter_check.py
- 156. q4/baselines/CG/api_client.py
- 157. q4/baselines/CG/base.py
- 158. q4/baselines/CG/bearing_geometry.py
- 159. q4/baselines/CG/cautious.py
- 160. q4/baselines/CG/config.example.json
- 161. q4/baselines/CG/coverage.py
- 162. q4/baselines/CG/m06.py
- 163. q4/baselines/CG/m07.py
- 164. q4/baselines/CG/m13.py
- 165. q4/baselines/CG/optimize.py
- 166. q4/baselines/CG/policies.json
- 167. q4/baselines/CG/posterior.py
- 168. q4/baselines/CG/previous.py
- 169. q4/baselines/CG/research_strategy.py
- 170. q4/baselines/CG/ring.json
- 171. q4/baselines/CG/run_robot.py
- 172. q4/baselines/CG/strategy.py
- 173. q4/baselines/M07/adapter_check.py
- 174. q4/baselines/M07/api_client.py
- 175. q4/baselines/M07/base.py
- 176. q4/baselines/M07/bearing_geometry.py
- 177. q4/baselines/M07/config.example.json
- 178. q4/baselines/M07/coverage.py
- 179. q4/baselines/M07/m06.py
- 180. q4/baselines/M07/m07.py
- 181. q4/baselines/M07/m13.py
- 182. q4/baselines/M07/optimize.py
- 183. q4/baselines/M07/policies.json
- 184. q4/baselines/M07/posterior.py
- 185. q4/baselines/M07/previous.py

- 186. q4/baselines/M07/research_strategy.py
- 187. q4/baselines/M07/ring.json
- 188. q4/baselines/M07/run_robot.py
- 189. q4/baselines/M07/strategy.py
- 190. q4/formal/logs/formal-p4-1-9N77-TH84-ME6J-NRK4.jlog
- 191. q4/formal/logs/formal-p4-2-QGEZ-KRWA-MNGB-6CMH.jlog
- 192. q4/formal/logs/formal-p4-3-KTZ8-EZQ5-937B-QNEV.jlog
- 193. q4/formal/records/formal-p4-1-9N77-TH84-ME6J-NRK4.actions.jsonl
- 194. q4/formal/records/formal-p4-1-9N77-TH84-ME6J-NRK4.result.json
- 195. q4/formal/records/formal-p4-2-QGEZ-KRWA-MNGB-6CMH.actions.jsonl
- 196. q4/formal/records/formal-p4-2-QGEZ-KRWA-MNGB-6CMH.result.json
- 197. q4/formal/records/formal-p4-3-KTZ8-EZQ5-937B-QNEV.actions.jsonl
- 198. q4/formal/records/formal-p4-3-KTZ8-EZQ5-937B-QNEV.result.json
- 199. q4/formal/results.json
- 200. requirements.txt
- 201. source-provenance.json
- 202. validation.json
- 203. verify_materials.py
- 204. 三四问支撑材料.zip
- 205. 提交要求核对.md
- 206. 论文附录/README.md
- 207. 论文附录/appendix.tex
- 208. 论文附录/三四问附录.pdf
- 209. 论文附录/完整源码附录.tex
- 210. 论文附录/支撑材料清单.tex

2 第三、四问完整源程序

下列代码按文件实际内容列出；字节完全相同的公共模块只印一次，并列明其全部对应路径。每个策略的独立磁盘副本均已随支撑材料提交。代码换行仅为排版，不改变程序。

源文件组 1

local-simulator-snapshot/b_local_simulator.py

```

1  """Calibrated scenario profiles and an observation-only Robot for Q3/Q4.
2
3  This is an empirical practice-model approximation, not the official generator.
4  Legacy fixtures and frozen strategies are never changed. No network/UI imports.
5  """
6  import math, hashlib, struct, json
7  from pathlib import Path
8  import numpy as np
9
10 def fixture(seed,problem=4,profile=None,n=None,k=None,error_mode=None):
11     profile=profile or json.loads(Path(__file__).with_name('calibrated_profile.json').read_text(encoding='utf-8'))
12     rng=np.random.default_rng(seed)
13     n=int(rng.integers(10,17)) if n is None else int(n)
14     if problem==3:k=0
15     elif k is None:
16         law=profile['q4_directional_count_model']
17         if law=='uniform_0_N':k=int(rng.integers(0,n+1))
18         elif law=='uniform_1_N':k=int(rng.integers(1,n+1))
19         elif law=='uniform_0_Nminus1':k=int(rng.integers(0,n))
20         elif law=='uniform_1_Nminus1':k=int(rng.integers(1,n))
21         elif law=='binomial_N_half':k=int(rng.binomial(n,5))
22         elif law=='forced_mixed_binomial':k=1+int(rng.binomial(n-2,.5))
23         else:raise ValueError(law)
24     assert 10<=n<=16 and 0<=k<=n
25     channels=rng.choice(np.arange(1,21),n,replace=False);sources={}
26     for i,ch in enumerate(channels):
27         a=rng.uniform(0,2*np.pi);r=1800*np.sqrt(rng.random())
28         heading=rng.uniform(0,2*np.pi);radius=rng.uniform(1000,1500)
29         sources[int(ch)]=dict(point=[float(r*np.cos(a)),float(r*np.sin(a))],radius=float(radius),heading=float(heading) if i<k else None)
30     return dict(seed=seed,n=n,directional=k,problem=problem,sources=sources,errors=error_mode or 'sin',layout='area',profile=profile.copy())
31
32 class World:
33     def __init__(self,data):
34         self.data={**data,'sources':{int(ch):s for ch,s in data['sources'].items()}}
35         # Optional conditioning of research hypotheses. Never emitted by fixture().
36         # Exact spatial keys preserve persistent readings; no rounding of positions.
37         self.error_anchors={}
38         for anchor in data.get('error_anchors',[]):
39             key=self.anchor_key(anchor['point'],anchor['channel'])
40             value=float(anchor['error'])
41             if not math.isfinite(value) or not -1<=value<=1:raise ValueError('Invalid bounded error anchor')
42             if key in self.error_anchors and self.error_anchors[key]!=value:raise ValueError('Conflicting persistent error anchors')
43             self.error_anchors[key]=value
44         self.position=np.zeros(2);self.channel=1;self.cleared=set();self.discovered=set()
45         self.virtual=0.;self.distance=0.;self.measures=0;self.switches=0;self.failures=0;self.events=[]
46     @staticmethod
47     def anchor_key(q,ch):
48         if len(q)!=2 or not all(math.isfinite(float(v)) for v in q):raise ValueError('Invalid anchor point')
49         if int(ch)!=ch or not 1<=int(ch)<=20:raise ValueError('Invalid anchor channel')
50         return (int(ch),float(q[0]) or 0.,float(q[1]) or 0.)
51     def move(self,q):
52         q=np.asarray(q,float);d=float(np.linalg.norm(q-self.position));self.distance+=d;self.virtual+=d/5;self.position=q.copy()
53         if self.virtual>349000 or len(self.events)>12000:raise RuntimeError('local action/time cap')
54     def error(self,q,ch):
55         if self.error_anchors:
56             key=self.anchor_key(q,ch)
57             if key in self.error_anchors:return self.error_anchors[key]
58         mode=self.data['errors'];seed=self.data['seed']
59         if mode=='sin':return math.sin(q[0]*.017+q[1]*.031+ch*2.7+seed)

```

```

60     if mode=='plus':return 1.
61     if mode=='minus':return -1.
62     if mode=='checker':return 1. if (math.floor(q[0]/10)+math.floor(q[1]/10)+ch)%2 else -1.
63     if mode=='hash_uniform':
64         # Stable same-point noise; 0 and -0 are normalized. Not Python's salted hash.
65         raw=struct.pack('<qqdd',seed,int(ch),float(q[0]) or 0.,float(q[1]) or 0.)
66         u=int.from_bytes(hashlib.blake2b(raw,digest_size=8).digest(),'little')/2**64
67         return 2*u-1
68     raise ValueError(mode)
69 def measure(self,q,ch):
70     self.move(q);sw=int(ch!=self.channel);self.channel=ch;self.measures+=1;self.switches+=sw;self.virtual+=5+sw
71     s=self.data['sources'].get(ch);z={'measure_result':'no_signal'}
72     if s is not None and ch not in self.cleared:
73         delta=self.position-np.array(s['point']);d=float(np.linalg.norm(delta));h=s['heading']
74         visible=h is None or float(delta@np.array([math.cos(h),math.sin(h)]))>=-1e-10
75         if d<=s['radius'] and visible:
76             self.discovered.add(ch)
77             z={'measure_result':'near'} if d<=5 else
78             ↪ {'measure_result':'direction','svd_deg':round((math.degrees(math.atan2(-delta[1],-delta[0]))+self.error(self.position,ch))%360,2)%360}
79     self.events.append(dict(kind='measure',point=self.position.tolist(),channel=int(ch),result=z.copy(),time_s=self.virtual))
80     return z
81 def clear(self,q,ch):
82     self.move(q);s=self.data['sources'].get(ch)
83     ok=s is not None and ch not in self.cleared and math.dist(q,s['point'])<=20
84     self.virtual+=5 if ok else 3
85     if ok:self.cleared.add(ch)
86     else:self.failures+=1
87     self.events.append(dict(kind='clear',point=self.position.tolist(),channel=int(ch),success=bool(ok),time_s=self.virtual))
88     return bool(ok)
89 def condition_q3_history(data,actions):
90     """Return a shared World that exactly explains and has replayed a Q3 prefix.
91
92     This is an existence construction for bounded persistent errors, not posterior
93     sampling. All hypothesized sources (including cleared ones) must be supplied.
94     Neither the real fixture nor its hidden parameters are needed by callers.
95     Past cost is retained: compare future costs after subtracting world.virtual.
96     """
97     world=World(data)
98     sources=world.data['sources']
99     if not 10<=len(sources)<=16:raise ValueError('Q3 hypothesis source count outside 10..16')
100    for ch,s in sources.items():
101        World.anchor_key(s['point'],ch)
102        if s.get('heading') is not None:raise ValueError('Q3 requires omnidirectional sources')
103        if not 1000<=float(s['radius'])<=1500 or math.hypot(*s['point'])>1800:raise ValueError('Invalid Q3 source geometry')
104    for index,a in enumerate(actions):
105        q=a['point'];ch=int(a['channel']);World.anchor_key(q,ch)
106        if a['kind']=='measure':
107            z=a['result']
108            if z['measure_result']=='direction':
109                s=sources.get(ch)
110                if s is None or ch in world.cleared:raise ValueError(f'Positive history lacks active source at {index}')
111                delta=np.asarray(q,float)-np.asarray(s['point'],float)
112                angle=math.degrees(math.atan2(-delta[1],-delta[0]))
113                reading=float(z['svd_deg'])
114                if not math.isfinite(reading) or not 0<=reading<360:raise ValueError('Invalid bearing reading')
115                key=World.anchor_key(q,ch)
116                if key not in world.error_anchors:
117                    diff=(reading-angle+180)%360-180
118                    lo=max(-1.,diff-.005);hi=min(1.,diff+.005)
119                    if lo>hi:raise ValueError(f'History bearing outside error bound at {index}')
120                    value=(lo+hi)/2
121                    if round((angle+value)%360,2)%360!=reading:raise ValueError(f'No rounding witness at {index}')
122                    world.error_anchors[key]=value
123                # Shared measure below checks reception, near, previous clears,
124                # and exact repeated readings with this same fixed error.
125                actual=world.measure(q,ch)
126                if actual!=z:raise ValueError(f'Incompatible measurement history at {index}')
127            elif a['kind']=='clear':
128                if world.clear(q,ch)!=a['success']:raise ValueError(f'Incompatible clear history at {index}')
129            else:raise ValueError(f'Unknown history action at {index}')

```

```

130 world.data['error_anchors']=[dict(channel=ch,point=[x,y],error=e) for (ch,x,y),e in world.error_anchors.items())
131 return world
132
133 class Robot:
134     def __init__(self,world):self.__world=world;self.scanned_sites=0;self.decisions=[]
135     @property
136     def position(self):return self.__world.position.copy()
137     @property
138     def channel(self):return self.__world.channel
139     @property
140     def cleared(self):return frozenset(self.__world.cleared)
141     @property
142     def discovered(self):return frozenset(self.__world.discovered)
143     def measure(self,q,ch):return self.__world.measure(q,ch)
144     def clear(self,q,ch):return self.__world.clear(q,ch)
145     def record(self,event):self.decisions.append(event)
146     def check_budget(self):
147         if self.__world.virtual>349000:raise RuntimeError('local time cap')
148
149 class ReplayRobot:
150     """Exact logged-action replay. Rejects deviations; cannot evaluate new actions."""
151     def __init__(self,actions):
152         self.__actions=[x for x in actions if x[0] in ('/measure','/clear')];self.index=0
153         self.position=np.zeros(2);self.channel=1;self.cleared=set();self.discovered=set();self.scanned_sites=0;self.virtual=0
154     def step(self,endpoint,q,ch):
155         if self.index>=len(self.__actions):raise AssertionError('Replay exhausted; no off-policy inference allowed')
156         path,p,z=self.__actions[self.index]
157         assert path==endpoint and ch==p['channel'] and np.allclose(q,[p['position']['x'],p['position']['y']],rtol=0,atol=1e-6),f'Replay diverged at {self.index}'
158         self.index+=1;self.position=np.asarray(q,float);self.virtual=z['virtual_time_s'];return z
159     def measure(self,q,ch):
160         z=self.step('/measure',q,ch);self.channel=ch
161         if z['measure_result']!='no_signal':self.discovered.add(ch)
162         return {k:z[k] for k in ('measure_result','svd_deg') if k in z}
163     def clear(self,q,ch):
164         ok=self.step('/clear',q,ch)['clear_result']=='success'
165         if ok:self.cleared.add(ch)
166         return ok
167     def record(self,event):pass
168     def check_budget(self):pass
169     def assert_complete(self):assert self.index==len(self.__actions)

```

源文件组 2

local-simulator-snapshot/run_local.py

```

1 """Default Q3/Q4 local simulator. No official HTTP or desktop operations."""
2 from pathlib import Path
3 import argparse, contextlib, hashlib, importlib, io, json, platform, shutil, sys, time
4 import numpy as np
5 from b_local_simulator import fixture, World, Robot
6
7 HERE=Path(__file__).resolve().parent
8
9 def read(p):return json.loads(Path(p).read_text(encoding='utf-8'))
10 def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
11 def save(p,value):
12     p=Path(p);tmp=p.with_suffix(p.suffix+'.tmp')
13     tmp.write_text(json.dumps(value,ensure_ascii=False,indent=2,allow_nan=False),encoding='utf-8');tmp.replace(p)
14
15 def main():
16     ap=argparse.ArgumentParser(description=__doc__)
17     ap.add_argument('--problem',type=int,choices=[3,4],required=True)
18     ap.add_argument('--policy',help='Bundled: v4/v6 for Q3, j500 for Q4; custom policy with --strategy-dir')
19     ap.add_argument('--strategy-dir',type=Path,help='New candidate source directory; factory receives only Robot')
20     ap.add_argument('--factory',default='strategy:Planner',help='module:callable')
21     ap.add_argument('--planner-kwarg',type=Path,help='JSON constructor arguments, replacing the default policy keyword')
22     ap.add_argument('--count',type=int,default=50)
23     ap.add_argument('--start',type=int,default=2026091300,help='Fresh seed range, distinct from calibration runs')
24     ap.add_argument('--errors',choices=['sin','hash_uniform','plus','minus','checker'],default='sin')

```

```

25 ap.add_argument('--pure-types',action='store_true',help='Q4 alternating K=0/K=N, N=10..16')
26 ap.add_argument('--output',type=Path,required=True)
27 a=ap.parse_args();defaults=read(HERE/'defaults.json');profile=read(HERE/defaults['profile'])
28 if a.count<1:ap.error('--count must be positive')
29 if a.pure_types and a.problem!=4:ap.error('--pure-types applies only to Q4')
30 policy=a.policy or defaults['problem_defaults'][str(a.problem)]
31 if a.strategy_dir:
32     source=a.strategy_dir.resolve();spec=None
33     if not source.is_dir():ap.error('--strategy-dir does not exist')
34 else:
35     spec=defaults['policies'].get(policy)
36     if spec is None or spec['problem']!=a.problem:ap.error('Policy/problem mismatch; use --strategy-dir for a new candidate')
37     source=HERE/spec['directory']
38     for name,digest in spec['source_sha256'].items():
39         if sha(source/name)!=digest:raise RuntimeError(f'Frozen baseline changed: {name}; use a separate candidate directory')
40     kwargs=read(a.planner_kwargs) if a.planner_kwargs else {'policy':spec['policy_argument'] if spec else policy}
41     if not isinstance(kwargs,dict):ap.error('--planner-kwags must contain a JSON object')
42     a.output=a.output.resolve();a.output.mkdir(parents=True,exist_ok=False)
43     snap=a.output/'simulator_snapshot';snap.mkdir()
44     for name in ['b_local_simulator.py','calibrated_profile.json','defaults.json','run_local.py']:
45         shutil.copy2(HERE/name,snap/name)
46     strategy_snap=a.output/'strategy_snapshot';strategy_snap.mkdir()
47     # Freeze the actual candidate before importing; nested packages are retained.
48     for p in source.rglob('*'):
49         if p.is_file() and p.suffix in ('.py','.json') and p.name!='config.json' and not any(x in ('runs','__pycache__','.git') for x in p.relative_to(source).parts):
50             dest=strategy_snap/p.relative_to(source);dest.parent.mkdir(parents=True,exist_ok=True);shutil.copy2(p,dest)
51     sys.path.insert(0,str(strategy_snap))
52     module,name=a.factory.split(':',1)
53     factory=getattr(importlib.import_module(module),name)
54     rows=[];scenes=[]
55     manifest=dict(simulator_id=defaults['simulator_id'],simulator_entry=str(HERE/'b_local_simulator.py'),official=False,problem=a.problem,
56         policy=policy,factory=a.factory,planner_kwargs=kwargs,source=str(source),command=sys.argv,python=platform.python_version(),errors=a.errors,
57         start=a.start,count=a.count,pure_types=a.pure_types,status='running',
58         simulator_sha256={p.name:sha(p) for p in snap.iterdir()},strategy_sha256={p.relative_to(strategy_snap).as_posix():sha(p) for p in
59             ↳ strategy_snap.rglob('*') if p.is_file()})
60     save(a.output/'manifest.json',manifest)
61     print(f"Simulator: {defaults['simulator_id']} | Q{a.problem} | {policy}",flush=True)
62     for i in range(a.count):
63         n=10+(i//2)%7 if a.pure_types else None;k=(n if i%2 else 0) if a.pure_types else None
64         data=fixture(a.start+i,a.problem,profile,n=n,k=k,error_mode=a.errors);scenes.append(data)
65         w=World(data);robot=Robot(w);begin=time.perf_counter();error=None
66         try:
67             with contextlib.redirect_stdout(io.StringIO()):factory(robot,**kwargs).run()
68         except Exception as exc:error=repr(exc)
69         full=len(w.cleared)==data['n'] and error is None
70         row=dict(seed=data['seed'],problem=a.problem,policy=policy,n=data['n'],k=data['directional'],cleared=len(w.cleared),full=full,error=error,
71             time_s=w.virtual,per_source_s=w.virtual/data['n'],distance_m=w.distance,measures=w.measures,switches=w.switches,failures=w.failures,compute
72             ↳ _s=time.perf_counter()-begin)
73         rows.append(row);save(a.output/f'{data["seed"]}-actions.json',dict(actions=w.events,decisions=robot.decisions))
74         save(a.output/'results.json',rows);save(a.output/'fixtures.json',scenes)
75         print(f"[{i+1}]/{a.count} {row['cleared']}/{data['n']} | {row['per_source_s']:.2f} s/source | {error or 'OK'}",flush=True)
76     manifest.update(status='completed',runs=len(rows),full=sum(r['full'] for r in rows),mean_s=float(np.mean([r['per_source_s'] for r in rows])) if all(r['full']
77         ↳ for r in rows) else None)
78     save(a.output/'manifest.json',manifest)
79     return 0 if all(r['full'] for r in rows) else 1
80 if __name__=='__main__':sys.exit(main())

```

源文件组 3

q3/analysis/analyze.py

```

1 from pathlib import Path
2 import csv,json,hashlib,math
3 import numpy as np
4 from scipy import stats
5
6 HERE=Path(__file__).resolve().parent
7 BATCH=HERE/'official'

```

```

8 OLD=BATCH/'history-v4-v6.csv'
9 PAIRED=HERE/'paired'
10 DATA=HERE/'data';DATA.mkdir(exist_ok=True)
11 def read(p):return json.loads(p.read_text(encoding='utf-8'))
12 def save(p,d):p.write_text(json.dumps(d,ensure_ascii=False,indent=2),encoding='utf-8')
13 def norm(r,local=False):
14     if local:
15         n=r['n'];full=r['full'];time=r['time_s'];d=r['distance_m'];m=r['measures'];sw=r['switches'];f=r['failures'];cl=r['cleared'];case=str(r['seed'])
16     else:
17         n=int(r['official_jammer_count']);full=str(r['full_clear']).lower()=='true';time=float(r['virtual_time_s']);d=float(r['distance_m']);m=int(r['measure_c']
            ↪ ount');sw=int(r['switches']);f=int(r['clear_failures']);cl=int(r['cleared_count']);case=r['case_code']
18     cost=np.array([d/5,5*m,sw,5*cl,3*f])/n
19     assert abs(cost.sum()-time/n)<1e-4,(case,cost.sum(),time/n)
20     return dict(case=case,n=n,full=full,time_s=time,per_source_s=time/n,cost=cost.tolist(),distance_m=d,measures=m,switches=sw,failures=f,cleared=cl)
21 def summary(rows):
22     full=[r for r in rows if r['full']];a=np.array([r['per_source_s'] for r in full]);n=len(a)
23     return dict(runs=len(rows),full=n,sources=sum(r['n'] for r in
            ↪ rows),mean=float(a.mean()),sd=float(a.std(ddof=1)),median=float(np.median(a)),min=float(a.min()),max=float(a.max()),cost=np.mean([r['cost'] for
            ↪ r in full],axis=0).tolist(),sem_ci95=(a.mean()+np.array([-1,1])*stats.t.ppf(.975,n-1)*a.std(ddof=1)/math.sqrt(n)).tolist())
24 def welch(a,b):
25     a=np.array(a);b=np.array(b);va=a.var(ddof=1)/len(a);vb=b.var(ddof=1)/len(b);se=math.sqrt(va+vb);df=(va+vb)**2/(va**2/(len(a)-1)+vb**2/(len(b)-1)
            ↪ );diff=a.mean()-b.mean()
26     return dict(saving=float(diff),ci95=(diff+np.array([-1,1])*stats.t.ppf(.975,df)*se).tolist(),df=float(df))
27
28 official=read(BATCH/'results.json');assert len(official)==30,'Official batch not yet complete'
29 for r in official:
30     assert r['export_verified']
31     # Original official files are retained locally. This portable script
32     # recomputes statistics; it does not repeat the original jlog hash audit.
33     old=list(csv.DictReader(OLD.open(encoding='utf-8-sig')))
34     groups={'literature':[norm(r) for r in official],'v4':[norm(r) for r in old if r['group']=='v4'],'v6':[norm(r) for r in old if r['group']=='v6']}
35     report={'official':{k:summary(v) for k,v in groups.items()}}
36     for key in ['literature','v4']:
37         report[key+' _vs _v6']=welch([r['per_source_s'] for r in groups[key] if r['full']], [r['per_source_s'] for r in groups['v6'] if r['full']])
38         report[key+' _vs _v6']['saving_percent']=100*(1-report['official']['v6']['mean']/report['official'][key]['mean'])
39         common=sorted(set(r['n'] for r in groups[key])&set(r['n'] for r in groups['v6']))
40         layers=[]
41         for n in common:
42             a=[r['per_source_s'] for r in groups[key] if r['n']==n and r['full']];b=[r['per_source_s'] for r in groups['v6'] if r['n']==n and r['full']]
43             if not a or not b:continue
44             layers.append(dict(n=n,old_count=len(a),v6_count=len(b),weight=len(a)+len(b),old_mean=float(np.mean(a)),v6_mean=float(np.mean(b))))
45         weights=np.array([x['weight'] for x in layers]);weights=weights/weights.sum()
46         report[key+' _vs _v6']['standardized']={k:float(np.dot(weights,[x[k] for x in layers])) for k in ['old_mean','v6_mean']}
47         report[key+' _vs _v6']['strata']=layers
48
49 local={key:[norm(r,True) for r in read(PAIRED/key/'results.json')] for key in ['baseline','v6']}
50 assert all(len(x)==30 for x in local.values())
51 assert read(PAIRED/'baseline/fixtures.json')==read(PAIRED/'v6/fixtures.json')
52 pairs=[]
53 for a,b in zip(local['baseline'],local['v6']):
54     assert a['case']==b['case']
55     pairs.append(dict(seed=int(a['case']),n=a['n'],both_full=a['full'] and b['full'],baseline=a,v6=b,saving_s=a['per_source_s']-b['per_source_s']))
56     valid=[p for p in pairs if p['both_full']];d=np.array([p['saving_s'] for p in valid]);median=float(np.median(d));representative=min(valid,key=lambda
            ↪ p:(abs(p['saving_s']-median),p['seed']))
57     report['local']={k:summary(v) for k,v in local.items()}
58     report['local'].update(paired_successes=len(valid),wins=int(sum(d>0)),mean_saving=float(d.mean()),median_saving=median,ci95=(d.mean()+np.array([-1,1]
            ↪ ))*stats.t.ppf(.975,len(d)-1)*d.std(ddof=1)/math.sqrt(len(d))).tolist(),representative=representative)
59     save(DATA/'official_groups.json',groups);save(DATA/'paired_results.json',pairs);save(DATA/'analysis.json',report)
60     save(DATA/'official_literature_results.json',official)
61     print(json.dumps(report,ensure_ascii=False,indent=2))

```

源文件组 4

q3/analysis/replot.py

```

1 from pathlib import Path
2 import json
3 from collections import Counter
4 import numpy as np

```

```

5 import matplotlib
6 matplotlib.use('Agg')
7 import matplotlib.pyplot as plt
8 from matplotlib.lines import Line2D
9 from matplotlib.patches import Circle
10
11 HERE=Path(__file__).resolve().parent
12 OUT=HERE/'figures';OUT.mkdir(exist_ok=True)
13 PAIR=HERE/'paired'
14 plt.rcParams.update({'font.family':'Microsoft YaHei','font.size':8,'axes.titlesize':9,'axes.labelsize':8,'xtick.labelsize':7,'ytick.labelsize':7,'axes.spines.top':False,'ax
    ↳ es.spines.right':False,'pdf.fonttype':42,'svg.fonttype':None,'axes.unicode_minus':False})
15 colors=['#AD5B00','#777777','#176B9B']
16 def read(p):return json.loads(p.read_text(encoding='utf-8'))
17 def save(fig,name):
18     for ext in ['pdf','svg','png']:
19         fig.savefig(OUT/(name+'.'+ext),dpi=360,facecolor='white')
20     plt.close(fig)
21
22 a=read(PAIR/'baseline/results.json');b=read(PAIR/'v6/results.json')
23 assert len(a)==len(b)==30 and all(x['full'] for x in a+b)
24 delta=np.array([x['per_source_s']-y['per_source_s'] for x,y in zip(a,b)])
25 i=min(range(30),key=lambda j:(abs(delta[j]-np.median(delta)),a[j]['seed']))
26 seed=a[i]['seed'];assert seed==b[i]['seed']
27 fixture=read(PAIR/'baseline/fixtures.json')[i]
28 fig,axs=plt.subplots(1,2,figsize=(160/25.4,87/25.4))
29 fig.subplots_adjust(left=.10,right=.98,top=.79,bottom=.20,wspace=.28)
30 fig.suptitle(f'同场景本地对照 · 种子 {seed} · {fixture["n"]} 个源',y=.99,fontsize=8)
31 for ax,key,row,color,title in zip(axs,['baseline','v6'],[a[i],b[i]],[colors[0],colors[2]],[(a) 谨慎测向: 逐源处理,(b) 全程协同: 联合安排任务]):
32     events=read(PAIR/key/f'{seed}-actions.json')['actions']
33     points=np.vstack([e['point'] for e in events])/1000
34     points=points[np_r_[True,np.linalg.norm(np.diff(points,axis=0),axis=1)>1e-7]]
35     ax.add_patch(Circle((0,0),1.8,fill=False,color='#777777',lw=.7,linestyle='--'))
36     ax.plot(points[:,0],points[:,1],color=color,lw=.7,alpha=.8,zorder=2)
37     # Arrow every fourth nonzero leg, all actual legs remain visible.
38     for j in range(0,len(points)-1,4):
39         p,q=points[j],points[j+1];mid=(p+q)/2
40         ax.annotate(' ',xy=mid+.045*(q-p)/np.linalg.norm(q-p),xytext=mid-.045*(q-p)/np.linalg.norm(q-p),arrowprops=dict(arrowstyle='->',color=color,lw=.8,
    ↳ mutation_scale=6))
41     source=np.array([s['point'] for s in fixture['sources'].values()])/1000
42     ax.scatter(source[:,0],source[:,1],marker='x',s=15,c='#222222',lw=.65,zorder=4)
43     counts=Counter(tuple(np.round(e['point'],4)) for e in events if e['kind']=='measure')
44     sites=np.array([p for p,n in counts.items() if n>=4])/1000
45     if len(sites):ax.scatter(sites[:,0],sites[:,1],s=30,facecolors='white',edgecolors=color,lw=1,zorder=5)
46     ax.scatter([0],[0],s=28,marker='s',c='black',zorder=6)
47     ax.set(xlim=(-1.95,1.95),ylim=(-1.95,1.95),xticks=[-1.5,0,1.5],yticks=[-1.5,0,1.5],xlabel='x / km',ylabel='y / km')
48     ax.set_aspect('equal');ax.grid(alpha=.15,lw=.5)
49     ax.set_title(title,pad=23,loc='left')
50     ax.text(.5,1.025,f'{row["distance_m"]/1000:.2f} km · {row["measures"]} 次检测 · {row["time_s"]:.1f} s',ha='center',transform=ax.transAxes,fontsize=7)
51 fig.legend(handles=[Line2D([],[],c='#666666',lw=.8,label='实际移动路线'),Line2D([],[],c='black',marker='x',ls='',label='源真值 (仅事后绘图)
    ↳ ),Line2D([],[],c='#666666',marker='o',mfc='white',ls='',label='同点至少 4 次检测'),Line2D([],[],c='black',marker='s',ls='',label='起点')],loc='lower
    ↳ center',bbox_to_anchor=(.5,.04),ncol=2,frameon=False,fontsize=7,columnspacing=1.6)
52 save(fig,'q3-route-coverage')
53
54 if not (HERE/'data/analysis.json').exists():
55     print('Paired route complete; official plots wait for all 30 results.').raise SystemExit(0)
56 groups=read(HERE/'data/official_groups.json');report=read(HERE/'data/analysis.json')
57 fig,axs=plt.subplots(1,2,figsize=(160/25.4,78/25.4),gridspec_kw={'width_ratios':[1,1.15]})
58 fig.subplots_adjust(left=.105,right=.975,bottom=.28,top=.85,wspace=.52)
59 ax=axs[0];rng=np.random.default_rng(20260913)
60 names=['谨慎测向\n30 场','局部协同\n50 场','全程协同\n50 场']
61 for j,(key,col) in enumerate(zip(['literature','v4','v6'],colors)):
62     values=[r['per_source_s'] for r in groups[key] if r['full']]
63     ax.scatter(j+rng.uniform(-.16,.16,len(values)),values,c=col,s=8,alpha=.5,linewidths=0)
64     s=report['official'][key];ax.errorbar(j,s['mean'],yerr=np.array([(s['mean']-s['sem_ci95'])[0]],s['sem_ci95'])[1]-s['mean']]),fmt='D',ms=4,c='black',capsize=3,lw
    ↳ =1,zorder=5)
65 ax.set(xticks=range(3),xticklabels=names,ylabel='场均每源耗时 / s',ylim=(0,450),xlim=(-.5,2.5))
66 ax.set_title('(a) 模拟器演练测试: 保留每场观测',loc='left',pad=13)
67 ax.yaxis.grid(alpha=.2,lw=.5)
68 ax.text(.02,.98,◆ 均值及 95% 均值置信区间',transform=ax.transAxes,va='top',fontsize=6.7)
69 ax=axs[1];cost_colors=['#176B9B','#D9902B','#707070','#37816A','#B8475B'];labels=['移动','检测','换频','成功清除','失败清除'];hatches=['','','','xx','\
    ↳ \\\']

```

```

70 for j,key in enumerate(['literature','v4','v6']):
71     left=0
72     for k,value in enumerate(report['official'][key]['cost']):
73         ax.barh(j,value,left=left,color=cost_colors[k],height=.55,label=labels[k] if j==0 else None,hatch=hatches[k],edgecolor='white',lw=.35)
74         if k<2:ax.text(left+value/2,j,f'{value:.1f}',ha='center',va='center',fontsize=7,color='white' if k==0 else 'black')
75         left+=value
76     ax.text(left+4,j,f'{left:.1f}',va='center',fontsize=7)
77 ax.set(yticks=range(3),yticklabels=['谨慎测向','局部协同','全程协同'],xlabel='平均分项时间 / (s · 源 $\sim 1$)',xlim=(0,380),ylim=(2.55,-.65))
78 ax.set_title('(b) 分项时间比较',loc='left',pad=13);ax.xaxis.grid(alpha=.15,lw=.5)
79 ax.legend(loc='lower
    ↪ center',bbox_to_anchor=(.75,.005),bbox_transform=fig.transFigure,ncol=3,frameon=False,fontsize=6.5,columnspacing=1,handlelength=1.3)
80 save(fig,'q3-performance')
81 print('Saved official comparison and paired local route figures.')

```

源文件组 5

q3/analysis/replot_optical.py

```

1 from pathlib import Path
2 import json,hashlib
3 import numpy as np
4 import matplotlib
5 matplotlib.use('Agg')
6 import matplotlib.pyplot as plt
7 from matplotlib.patches import Circle,Polygon,Rectangle
8 from shapely import wkt
9
10 BASE=Path(__file__).resolve().parent/'data'
11 OUT=Path(__file__).resolve().parent/'figures'
12 OUT.mkdir(exist_ok=True)
13 case={'case_code':'T446-VHAY-76FH-9VYW'}
14 log=BASE/'optical_events.jsonl'
15 events=[json.loads(x) for x in log.read_text(encoding='utf-8').splitlines()]
16 i=next(i for i,e in enumerate(events) if e['kind']=='optical_failure_update')
17 failed=events[i];ch=failed['channel']
18 before=next(e for e in reversed(events[i:]) if e['kind']=='optical_region' and e['channel']==ch)
19 after=next(e for e in events[i+1:] if e['kind']=='optical_region' and e['channel']==ch)
20 meta=json.loads((BASE/'optical_example.json').read_text(encoding='utf-8'))
21 origin=np.array(meta['origin']);rot=np.array(meta['rotation_columns'])
22 clear=meta['clear_certificate']
23 BLUE,TEAL,ORANGE,GRAY,RED='#245B83','#26756B','#B46A24','#64717C','#A54246'
24 plt.rcParams.update({'font.family':'sans-serif','font.sans-serif':['SimSun','DejaVu Sans'],
25 'mathtext.fontset':'stix','font.size':9,'axes.labelsize':9,'axes.titlesize':9.5,
26 'xtick.labelsize':8,'ytick.labelsize':8,'axes.linewidth':.65,
27 'pdf.fonttype':42,'ps.fonttype':42,'svg.fonttype':'none','axes.unicode_minus':False,
28 'savefig.facecolor':'white','figure.facecolor':'white'})
29 def xy(a):return (np.array(a)-origin)@rot
30 def region(ax,record,color,alpha):
31     geom=wkt.loads(record['wkt'])
32     for g in getattr(geom,'geoms',[geom]):
33         ax.add_patch(Polygon(xy(g.exterior.coords),fc=color,ec=color,lw=.7,alpha=alpha))
34 def layers(ax,k):
35     if k==0:
36         region(ax,before,BLUE,.25);region(ax,failed,TEAL,.65)
37         ax.add_patch(Circle((0,0),20,fc=ORANGE,ec=ORANGE,alpha=.18,lw=.8,ls='--'))
38     else:
39         region(ax,failed,GRAY,.18);region(ax,after,BLUE,.6)
40         ax.add_patch(Circle(xy(clear['point']),19.5,fill=False,ec=TEAL,lw=.9,ls='--'))
41 fig,axes=plt.subplots(1,2,figsize=(160/25.4,65/25.4))
42 fig.subplots_adjust(left=.085,right=.985,bottom=.25,top=.86,wspace=.28)
43 for k,ax in enumerate(axes):
44     layers(ax,k)
45     ax.set(aspect='equal',xlim=(-98,98),ylim=(-48,48),xlabel=r"$x\backslash,\backslash\mathrm{m}$",ylabel=r"$y\backslash,\backslash\mathrm{m}$")
46     ax.set_xticks([-80,-40,0,40,80]);ax.set_yticks([-40,0,40]);ax.grid(alpha=.15,lw=.5)
47     ax.add_patch(Rectangle((-26,-1),10,5,fill=False,ec=RED,lw=.7))
48     ins=ax.inset_axes([.54,.10,.43,.30]);layers(ins,k)
49     ins.set(aspect='equal',xlim=(-26,-16),ylim=(-1,4))
50     ins.set_xticks([-25,-20]);ins.set_yticks([0,3]);ins.tick_params(labelsize=6,pad=1,length=2)
51     ins.set_title('红框局部放大 (等比例)',fontsize=6.1,pad=1)

```

```

52     for spine in ins.spines.values():spine.set_color(RED);spine.set_linewidth(.65)
53     ax.plot([-16,0],[1.5,-16],color=RED,lw=.55,ls='-')
54     axes[0].scatter([0],[0],s=25,marker='x',c=RED,zorder=5)
55     axes[0].annotate('试清位置',(0,0),(16,32),fontsize=8,arrowprops={'arrowstyle':'-','color':GRAY,'lw':.6})
56     axes[0].set_title('(a) 试清失败后保留两分区区域',loc='left',pad=9)
57     cq=xy(clear['point']);axes[1].scatter(*cq,s=35,marker='*',c=TEAL,zorder=6)
58     axes[1].scatter([0],[0],s=20,facecolors='white',edgecolors=BLUE,zorder=5)
59     axes[1].annotate('可靠清除位置',cq,(-75,32),fontsize=8,arrowprops={'arrowstyle':'-','color':GRAY,'lw':.6})
60     axes[1].set_title('(b) 再测向后进入可靠清除范围',loc='left',pad=9)
61     fig.text(.5,.075,'主图与插图分别保持等比例; 插图刻度单位为米',ha='center',fontsize=8)
62     for ext in ['pdf','svg','png']:fig.savefig(OUT/fq3-optical.{ext},dpi=400)
63     plt.close(fig)
64     report={'input':sha256(hashlib.sha256(log.read_bytes()).hexdigest()),'case':case['case_code'],
65            'channel':ch,'transform':'rigid translation and rotation only','main_aspect':'equal',
66            'inset_aspect':'equal','inset_xlim':[-26,-16],'inset_ylim':[-1,4],
67            'outputs':{'ext':hashlib.sha256((OUT/fq3-optical.{ext})).read_bytes()).hexdigest() for ext in ['pdf','svg','png']}}
68     (Path(__file__).parent/'optical_revision_manifest.json').write_text(json.dumps(report,indent=2),encoding='utf-8')
69     print(report)

```

源文件组 6

q3/baselines/CG/api_client.py

q3/baselines/v4/api_client.py

q3/v6/api_client.py

q4/baselines/CG/api_client.py

q4/baselines/M07/api_client.py

q4/M32/api_client.py

```

1  """Serial, idempotent simulator client. No network calls on import."""
2  import datetime as dt
3  from http.client import HTTPException
4  import json
5  import math
6  from pathlib import Path
7  import time
8  import uuid
9  from urllib.error import HTTPError, URLError
10 from urllib.request import Request, build_opener, ProxyHandler
11
12
13 class BudgetReached(Exception):
14     pass
15
16
17 class Robot:
18     def __init__(self, team, url, directory):
19         self.team, self.url = team, url.rstrip('/')
20         self.directory = Path(directory)
21         self.directory.mkdir(parents=True, exist_ok=False)
22         self.log = (self.directory / 'actions.jsonl').open('w', encoding='utf-8')
23         self.opener = build_opener(ProxyHandler({}))
24         self.prefix, self.sequence = uuid.uuid4().hex[:12], 0
25         self.virtual, self.deadline = 0., float('inf')
26         self.started = None
27         self.exit_confirmed = self.transport_uncertain = False
28         self.cleared, self.discovered = set(), set()
29         self.measure_count = self.clear_failures = self.scanned_sites = 0
30         self.position, self.channel = (0., 0.), 1
31         self.distance_m = 0.
32         self.switches = 0
33
34     def record(self, value):
35         self.log.write(json.dumps(value, ensure_ascii=False, allow_nan=False) + '\n')
36         self.log.flush()
37

```

```

38 def check_budget(self):
39     if time.monotonic() > self.deadline - 20 or self.virtual > 349000:
40         raise BudgetReached('接近运行时间上限, 保留退出余量')
41     if self.transport_uncertain:
42         raise RuntimeError('上一动作是否执行尚不确定, 停止发送新动作')
43
44 def call(self, path, **fields):
45     if self.transport_uncertain:
46         raise RuntimeError('上一动作执行状态不确定, 不能发送新请求')
47     if path != '/exit':
48         self.check_budget()
49     self.sequence += 1
50     payload = dict(arena_id='default', robot_id=self.team,
51                   request_id=f'{self.prefix}-{self.sequence}', **fields)
52     body = json.dumps(payload, allow_nan=False).encode('utf-8')
53     for attempt in range(1, 4):
54         self.record(dict(kind='request', time=dt.datetime.now().isoformat(), path=path,
55                          attempt=attempt, payload=payload))
56     try:
57         req = Request(self.url + path, data=body,
58                      headers={'Content-Type': 'application/json'}, method='POST')
59     try:
60         with self.opener.open(req, timeout=4) as response:
61             status, raw = response.status, response.read()
62     except HTTPError as error:
63         status, raw = error.code, error.read()
64     if status >= 500:
65         raise ConnectionError(f'HTTP {status}: action state uncertain')
66     data = json.loads(raw.decode('utf-8'))
67     self.record(dict(kind='response', request_id=payload['request_id'],
68                    http_status=status, response=data))
69     if not isinstance(data, dict) or 'accepted' not in data:
70         raise ValueError('接口响应缺少 accepted 字段')
71     if status != 200 or data['accepted'] is not True:
72         # Explicit rejection is definite; its zero clock is not a reset.
73         raise RuntimeError(f'{path} 未接受: HTTP {status}, {data}')
74     virtual = float(data['virtual_time_s'])
75     if not math.isfinite(virtual) or virtual < self.virtual - 1e-5:
76         raise ValueError('接口时钟无效或倒退')
77     if path == '/measure':
78         if data.get('measure_result') not in ('no_signal', 'near', 'direction'):
79             raise ValueError('无效测量结果')
80         if data['measure_result'] == 'direction' and not math.isfinite(float(data['svd_deg'])):
81             raise ValueError('无效示向度')
82     if path == '/clear' and data.get('clear_result') not in ('success', 'no_target_in_range'):
83         raise ValueError('无效清除结果')
84     if path == '/enter':
85         remaining = float(data['remaining_real_duration_s'])
86         if not math.isfinite(remaining) or remaining < 0:
87             raise ValueError('无效剩余时间')
88         self.deadline = self.started + min(remaining, 1200.)
89     self.virtual = virtual
90     if path == '/exit':
91         self.exit_confirmed = True
92     return data
93 except (URLError, TimeoutError, ConnectionError, OSError, HTTPException,
94        ValueError, KeyError, TypeError) as error:
95     self.record(dict(kind='uncertain_response', attempt=attempt, error=str(error)))
96     if attempt == 3:
97         self.transport_uncertain = True
98         raise RuntimeError('同一请求重试仍未确认; 已停止新动作, 请查看模拟器和日志') from error
99     time.sleep(.25)
100
101 def _move(self, point):
102     self.distance_m += math.dist(self.position, point)
103     self.position = tuple(point)
104
105 def measure(self, point, channel):
106     data = self.call('/measure', position={'x': float(point[0]), 'y': float(point[1])}, channel=channel)
107     self._move(point)
108     self.switches += int(channel != self.channel)

```

```

109     self.channel = channel
110     self.measure_count += 1
111     if data['measure_result'] != 'no_signal':
112         self.discovered.add(channel)
113     return data
114
115 def clear(self, point, channel):
116     data = self.call('/clear', position={'x': float(point[0]), 'y': float(point[1])}, channel=channel)
117     self._move(point)
118     success = data['clear_result'] == 'success'
119     if success:
120         self.cleared.add(channel)
121         print(f' 清除频道 {channel:2d}; 累计 {len(self.cleared)} 个; 虚拟时间 {self.virtual:.1f} 秒', flush=True)
122     else:
123         self.clear_failures += 1
124     # /clear does not change the measurement channel.
125     return success

```

源文件组 7

q3/baselines/CG/bearing_geometry.py

q3/baselines/v4/bearing_geometry.py

q3/v6/bearing_geometry.py

q4/baselines/CG/bearing_geometry.py

q4/baselines/M07/bearing_geometry.py

q4/M32/bearing_geometry.py

```

1  """Q1 M01 v01: directed bearing wedges, polygon diameter and enclosing disk.
2
3  All coordinates are metres, public angles degrees, inequalities A @ x <= b.
4  This module makes no stochastic noise assumption and no simulator requests.
5  """
6  from dataclasses import dataclass
7  from itertools import combinations
8  import numpy as np
9  from scipy.optimize import linprog
10
11
12  @dataclass
13  class Region:
14      status: str
15      vertices: np.ndarray
16      max_residual: float = 0.0
17
18
19  def unit(degrees):
20      a = np.deg2rad(degrees)
21      return np.array([np.cos(a), np.sin(a)])
22
23
24  def bearing(point, sensor):
25      d = np.asarray(point) - np.asarray(sensor)
26      return float(np.rad2deg(np.arctan2(d[1], d[0])) % 360)
27
28
29  def angle_difference(a, b):
30      return (np.asarray(a) - np.asarray(b) + 180) % 360 - 180
31
32
33  def wedge(sensor, angle_deg, epsilon_deg=1.0):
34      """Closed, forward wedge; valid for 0 < epsilon < 90 degrees."""
35      if not 0 < epsilon_deg < 90:
36          raise ValueError("epsilon must lie strictly between 0 and 90 degrees")
37      s = np.asarray(sensor, dtype=float)
38      lower, upper = unit(angle_deg - epsilon_deg), unit(angle_deg + epsilon_deg)

```

```

39 A = np.array([[lower[1], -lower[0]], [-upper[1], upper[0]]])
40 return A, A @ s
41
42
43 def observations_halfplanes(observations, epsilon_deg=1.0):
44     parts = [wedge(o['position'], o['bearing_deg'], epsilon_deg) for o in observations]
45     if not parts:
46         return np.empty((0, 2)), np.empty(0)
47     return np.vstack([p[0] for p in parts]), np.hstack([p[1] for p in parts])
48
49
50 def unique_points(points, tol=1e-7):
51     kept = []
52     for p in points:
53         if not any(np.linalg.norm(p - q) <= tol for q in kept):
54             kept.append(p)
55     return np.array(kept, dtype=float).reshape(-1, 2)
56
57
58 def halfplane_region(A, b, tol=1e-7):
59     """Classify without an artificial clipping box; enumerate bounded vertices.
60
61     Four LP support problems distinguish unbounded sets from bounded polygons.
62     Pairwise line intersections cost  $O(m^3)$ , appropriate for few observations.
63     The result is floating point, not interval-arithmetic certification.
64     """
65     A, b = np.asarray(A, float).reshape(-1, 2), np.asarray(b, float)
66     if not len(A):
67         return Region('unbounded', np.empty((0, 2)))
68     lengths = np.linalg.norm(A, axis=1)
69     zero = lengths < 1e-15
70     if np.any(b[zero] < -tol):
71         return Region('empty', np.empty((0, 2)))
72     A, b = A[~zero] / lengths[~zero, None], b[~zero] / lengths[~zero]
73     if not len(A):
74         return Region('unbounded', np.empty((0, 2)))
75     feasible = linprog([0, 0], A_ub=A, b_ub=b, bounds=[(None, None)] * 2, method='highs')
76     if feasible.status == 2:
77         return Region('empty', np.empty((0, 2)))
78     if not feasible.success:
79         raise RuntimeError(feasible.message)
80     # Translation improves conditioning for noncentral observations.
81     origin = feasible.x
82     rhs = b - A @ origin
83     extrema = []
84     for c in [[1, 0], [-1, 0], [0, 1], [0, -1]]:
85         sol = linprog(c, A_ub=A, b_ub=rhs, bounds=[(None, None)] * 2, method='highs')
86         if sol.status == 3:
87             return Region('unbounded', np.empty((0, 2)))
88         if not sol.success:
89             raise RuntimeError(sol.message)
90         extrema.append(sol.x)
91     points = []
92     for i, j in combinations(range(len(A)), 2):
93         rows = A[[i, j]]
94         if abs(np.linalg.det(rows)) < 1e-12:
95             continue
96         x = np.linalg.solve(rows, rhs[[i, j]])
97         if np.max(A @ x - rhs) <= tol:
98             points.append(x)
99     if not points:
100         points = extrema # Degenerate point/segment fallback, never arbitrary box.
101     vertices = unique_points(points, tol) + origin
102     if len(vertices) > 2:
103         center = vertices.mean(axis=0)
104         order = np.argsort(np.arctan2(vertices[:, 1] - center[1], vertices[:, 0] - center[0]))
105         vertices = vertices[order]
106     status = 'point' if len(vertices) == 1 else 'segment' if len(vertices) == 2 else 'polygon'
107     return Region(status, vertices, float(max(0.0, np.max(A @ vertices.T - b[:, None]))))
108
109

```

```

110 def diameter(vertices):
111     p = np.asarray(vertices, float).reshape(-1, 2)
112     if not len(p):
113         raise ValueError('diameter needs a nonempty bounded region')
114     d2 = np.sum((p[:, None] - p[None, :]) ** 2, axis=2)
115     i, j = np.unravel_index(np.argmax(d2), d2.shape)
116     return float(np.sqrt(d2[i, j])), (int(i), int(j))
117
118
119 def enclosing_circle(vertices, tol=1e-8):
120     """Enumerate support disks (2 or 3 points), with a diameter-disk shortcut.
121
122     Exact finite-support algorithm in real arithmetic;  $O(h^4)$  worst case.
123     Final radius is recomputed from all points, giving a conservative numerical
124     covering disk even when tolerance affects the support choice.
125     """
126     p = np.asarray(vertices, float).reshape(-1, 2)
127     D, (i, j) = diameter(p)
128     center = (p[i] + p[j]) / 2
129     if np.max(np.linalg.norm(p - center, axis=1)) <= D / 2 + tol:
130         return center, float(np.max(np.linalg.norm(p - center, axis=1))), [i, j]
131     best = (np.inf, None, None)
132     for i, j, k in combinations(range(len(p)), 3):
133         d = p[j, k] - p[i]
134         determinant = np.linalg.det(d)
135         if abs(determinant) <= 1e-13 * max(1., np.linalg.norm(d[0]) * np.linalg.norm(d[1])):
136             continue
137         offset = np.linalg.solve(2 * d, np.sum(d * d, axis=1))
138         r = np.linalg.norm(offset)
139         if r >= best[0] + tol:
140             continue
141         c = p[i] + offset
142         distances = np.linalg.norm(p - c, axis=1)
143         if distances.max() <= r + tol:
144             best = (float(distances.max()), c, [i, j, k])
145     if best[1] is None:
146         raise ArithmeticError('Could not construct a covering disk; inspect conditioning')
147     return best[1], best[0], best[2]
148
149
150 def summarize_region(region):
151     if region.status == 'empty':
152         return {'status': 'empty', 'diameter_m': None, 'cover_radius_m': None}
153     if region.status == 'unbounded':
154         return {'status': 'unbounded', 'diameter_m': 'infinity', 'cover_radius_m': 'infinity'}
155     D, pair = diameter(region.vertices)
156     c, r, support = enclosing_circle(region.vertices)
157     return dict(status=region.status, vertices=region.vertices.tolist(), diameter_m=D,
158               diameter_pair=list(pair), cover_center=c.tolist(), cover_radius_m=r,
159               support_indices=support, diameter_disk_covers=bool(r <= D / 2 + 1e-7),
160               guaranteed_clear_possible=bool(r <= 20 * 1e-6),
161               max_halfplane_residual_m=region.max_residual)
162
163
164 def clip_polygon(poly, normal, offset, tol=1e-9):
165     """Sutherland-Hodgman clipping; caller must supply a genuine prior polygon."""
166     p = np.asarray(poly, float).reshape(-1, 2)
167     if not len(p):
168         return p
169     values = p @ normal - offset
170     output = []
171     for i in range(len(p)):
172         j = (i + 1) % len(p)
173         inside_i, inside_j = values[i] <= tol, values[j] <= tol
174         if inside_i:
175             output.append(p[i])
176         if inside_i != inside_j:
177             t = values[i] / (values[i] - values[j])
178             output.append(p[i] + np.clip(t, 0, 1) * (p[j] - p[i]))
179     return np.asarray(output, float).reshape(-1, 2)
180

```

```

181
182 def clip_halfplanes(poly, A, b):
183     for a, rhs in zip(A, b):
184         poly = clip_polygon(poly, a, rhs)
185         if not len(poly):
186             break
187     return poly
188
189
190 def outer_disk(center, radius, sides=360):
191     """Circumscribed regular polygon, never an inscribed disk approximation."""
192     angles = (np.arange(sides) + .5) * 2 * np.pi / sides
193     return np.asarray(center) + radius / np.cos(np.pi / sides) * np.column_stack((np.cos(angles), np.sin(angles)))
194
195
196 def disk_halfplanes(center, radius, sides=360):
197     angles = np.arange(sides) * 2 * np.pi / sides
198     A = np.column_stack((np.cos(angles), np.sin(angles)))
199     return A, radius + A @ np.asarray(center)

```

源文件组 8

q3/baselines/CG/legacy_strategy.py

q3/baselines/v4/legacy_strategy.py

q3/v6/legacy_strategy.py

```

1  """P3 M01/A01 v1: bounded-bearing regions, local routing, certified search.
2
3  No simulator connection at import; coordinates metres, angles degrees.
4  Heuristic efficiency; safety checks use outer regions, not point estimates.
5  """
6  import math
7  import numpy as np
8  from bearing_geometry import (outer_disk, disk_halfplanes, wedge,
9                               clip_halfplanes, enclosing_circle, diameter)
10
11  VERSION = 'p3-region-routing-v2'
12  EPS = 1.0051 # bounded error plus 0.005 degree rounding and numerical margin
13  CLEAR_RADIUS = 19.5
14  SIDES = 180
15
16
17  def scan_sites():
18      return [(0., 0.)] + [(1350 * math.cos(i * math.pi / 3),
19                          1350 * math.sin(i * math.pi / 3)) for i in range(6)]
20
21
22  class Coverage:
23      """Whole-square coverage certificate of the continuous 1800 m arena.
24
25      Keep EVERY square intersecting the arena, including boundary squares.
26      no_signal implies exclusion of the closed 1000 m disk for P3 only.
27      Cover a square only if its farthest corner is within 999.99 m.
28      """
29      def __init__(self, level=7):
30          self.level = level
31          self.half = 1800 / 2 ** level
32          axis = np.linspace(-1800 + self.half, 1800 - self.half, 2 ** level)
33          x, y = np.meshgrid(axis, axis)
34          centers = np.column_stack((x.ravel(), y.ravel()))
35          nearest = np.maximum(np.abs(centers) - self.half, 0)
36          self.centers = centers[np.sum(nearest ** 2, axis=1) <= 1800 ** 2 + 1e-6]
37          self.covered = np.zeros((21, len(self.centers)), dtype=bool)
38          self.samples = {c: [] for c in range(1, 21)}
39
40      def mask(self, point):
41          far = np.abs(self.centers - np.asarray(point)) + self.half
42          return np.sum(far ** 2, axis=1) <= 999.99 ** 2

```

```

43
44 def add(self, channel, point):
45     self.covered[channel] |= self.mask(point)
46     self.samples[channel].append([float(v) for v in point])
47
48 def absent(self, channel):
49     return bool(np.all(self.covered[channel]))
50
51 def gain(self, point, channels):
52     if not channels:
53         return 0.
54     return float(np.mean((~self.covered[channels] & self.mask(point))))
55
56
57 class Track:
58     def __init__(self, channel):
59         self.channel = channel
60         self.poly = outer_disk((0, 0), 1800, SIDES)
61         self.history = []
62         self.center = None
63         self.radius = float('inf')
64
65     def update(self, point, observation):
66         kind = observation['measure_result']
67         if kind == 'no_signal':
68             return # Disk exclusion is nonconvex; do not cut it incorrectly.
69         if kind not in ('near', 'direction'):
70             raise ValueError(f'Unknown measure_result: {kind}')
71         poly = self.poly
72         if kind == 'direction':
73             A, b = wedge(point, float(observation['svd_deg']), EPS)
74             poly = clip_halfplanes(poly, A, b)
75         A, b = disk_halfplanes(point, 5.000001 if kind == 'near' else 1500.000001, SIDES)
76         poly = clip_halfplanes(poly, A, b)
77         if not len(poly):
78             raise ArithmeticError(f'频道 {self.channel} 观测区域为空; 保留日志后退出, 不能据此清除')
79         self.poly = poly
80         self.center, self.radius, _ = enclosing_circle(poly)
81         self.history.append(dict(position=[float(v) for v in point], **observation))
82
83     def safe_clear_point(self, position):
84         """Nearest safe point along center->robot segment (not global projection)."""
85         if self.radius > CLEAR_RADIUS:
86             return None
87         c, p = self.center, np.asarray(position, float)
88         if np.max(np.linalg.norm(self.poly - p, axis=1)) <= CLEAR_RADIUS:
89             return p.copy()
90         low, high = 0., 1.
91         for _ in range(45):
92             t = (low + high) / 2
93             q = c + t * (p - c)
94             if np.max(np.linalg.norm(self.poly - q, axis=1)) <= CLEAR_RADIUS:
95                 low = t
96             else:
97                 high = t
98         return c + low * (p - c)
99
100     def next_point(self, position):
101         """Move toward the region with a bounded cross-bearing offset.
102
103         If every possible source is <= 1000 m away, reception is guaranteed.
104         Also recognize Q2's first-observation safe lens, valid even if a
105         source's first distance was > 1000 m. The lens is a sufficient test.
106         """
107         c, r = self.center, self.radius
108         if r <= 120:
109             candidates = [c]
110         else:
111             _, (i, j) = diameter(self.poly)
112             d = self.poly[j] - self.poly[i]
113             normal = np.array([-d[1], d[0]]) / max(np.linalg.norm(d), 1e-12)

```

```

114         offset = min(150., r * .25)
115         candidates = [c + offset * normal, c - offset * normal]
116     first = self.history[0]
117     s = np.asarray(first['position'])
118     safe = []
119     for q in candidates:
120         all_close = np.max(np.linalg.norm(self.poly - q, axis=1)) <= 999.9
121         in_lens = False
122         if first['measure_result'] == 'direction':
123             a = math.radians(float(first['svd_deg']))
124             u, v = np.array([math.cos(a), math.sin(a)]), np.array([-math.sin(a), math.cos(a)])
125             ab = np.array([np.dot(q - s, u), np.dot(q - s, v)])
126             ep = math.radians(EPS)
127             length2 = float(np.dot(ab, ab))
128             in_lens = (length2 <= 1000 ** 2 - 1e-3 and
129                 length2 <= 2000 * (ab[0] * math.cos(ep) - abs(ab[1]) * math.sin(ep)) - 1e-3)
130         if all_close or in_lens:
131             safe.append(q)
132     # Region fits a disk of radius < 1000 after one direction report.
133     if not safe:
134         if r > 999.9:
135             raise ArithmeticError('未找到可证明接收的下一观测点')
136         safe = [c]
137     q = min(safe, key=lambda p: float(np.linalg.norm(p - position)))
138     # Same-coordinate noise is fixed: never repeat an uninformative point.
139     if any(math.dist(q, h['position']) < .1 for h in self.history):
140         _, (i, j) = diameter(self.poly)
141         d = self.poly[j] - self.poly[i]
142         normal = np.array([-d[1], d[0]]) / max(np.linalg.norm(d), 1e-12)
143         alternates = [c + normal * 40, c - normal * 40]
144         safe = [p for p in alternates if np.max(np.linalg.norm(self.poly - p, axis=1)) <= 999.9]
145     if not safe:
146         raise ArithmeticError('重复观测点且没有安全侧移点')
147     q = min(safe, key=lambda p: float(np.linalg.norm(p - position)))
148     return np.asarray(q)
149
150
151 class Planner:
152     def __init__(self, robot, *, opportunistic=False, nearest_order=True):
153         self.robot = robot
154         self.coverage = Coverage()
155         self.tracks = {}
156         self.anchors = scan_sites()
157         self.opportunistic = opportunistic
158         self.nearest_order = nearest_order
159         self.stop_reason = None
160
161     def unknown(self):
162         return [c for c in range(1, 21) if c not in self.robot.discovered
163             and not self.coverage.absent(c)]
164
165     def observe(self, point, channel):
166         p = tuple(float(v) for v in point)
167         observation = self.robot.measure(p, channel)
168         if observation['measure_result'] == 'no_signal':
169             self.coverage.add(channel, p)
170         else:
171             track = self.tracks.setdefault(channel, Track(channel))
172             track.update(p, observation)
173             self.robot.record(dict(kind='region', channel=channel, position=list(p),
174                 radius_m=track.radius, center=track.center.tolist(),
175                 vertices=track.poly.tolist(), observations=len(track.history)))
176         return observation
177
178     def scan(self, point, full=False):
179         unknown = self.unknown()
180         scan_unknown = full or self.coverage.gain(point, unknown) >= .05
181         channels = set(unknown if scan_unknown else [])
182         for c, track in self.tracks.items():
183             if c in self.robot.cleared:
184                 continue

```

```

185         if full or (track.radius > CLEAR_RADIUS and
186                     math.dist(point, track.history[-1]['position']) >= 200 and
187                     np.max(np.linalg.norm(track.poly - point, axis=1)) <= 999.9):
188             channels.add(c)
189     if channels:
190         self.robot.scanned_sites += 1
191     # Start with current measurement channel if possible. Clear does not switch it.
192     for c in sorted(channels, key=lambda c: (c != self.robot.channel, c)):
193         self.observe(point, c)
194
195     def solve_track(self, track):
196         for _ in range(16):
197             self.robot.check_budget()
198             q = track.safe_clear_point(self.robot.position)
199             if q is not None:
200                 bound = float(np.max(np.linalg.norm(track.poly - q, axis=1)))
201                 self.robot.record(dict(kind='clear_certificate', channel=track.channel,
202                                     point=q.tolist(), worst_distance_m=bound))
203                 if not self.robot.clear(tuple(float(v) for v in q), track.channel):
204                     raise ArithmeticError('有界误差清除证书与接口失败冲突; 请保留记录检查')
205                 return
206             q = track.next_point(np.asarray(self.robot.position))
207             observation = self.observe(q, track.channel)
208             if observation['measure_result'] == 'no_signal':
209                 raise ArithmeticError('保证接收点返回无信号; 检查问题模式、参数和角度约定')
210             raise RuntimeError('频道 {track.channel} 达到 16 次定位预算, 仍未达到可靠清除精度')
211
212     def run(self):
213         self.scan(self.anchors.pop(0), full=True)
214         while True:
215             self.robot.check_budget()
216             pending = [t for c, t in self.tracks.items() if c not in self.robot.cleared]
217             if pending:
218                 def score(t):
219                     q = t.safe_clear_point(self.robot.position)
220                     q = t.center if q is None else q
221                     return math.dist(q, self.robot.position) + .15 * t.radius
222                 track = min(pending, key=score if self.nearest_order else lambda t: t.channel)
223                 self.solve_track(track)
224                 if len(self.robot.cleared) < 16 and self.opportunistic:
225                     self.scan(self.robot.position)
226                 continue
227             if len(self.robot.cleared) == 16:
228                 self.stop_reason = 'cleared_upper_bound_16'
229                 break
230             unknown = self.unknown()
231             if not unknown:
232                 self.stop_reason = 'continuous_coverage_certificate'
233                 break
234             if not self.anchors:
235                 raise RuntimeError('七点扫描后仍有频道未被覆盖证书排除; 不能声称全部清除')
236             site = min(self.anchors, key=lambda p: (math.dist(p, self.robot.position) + 100) /
237                  max(self.coverage.gain(p, unknown), 1e-6))
238             self.anchors.remove(site)
239             self.scan(site, full=True)
240             self.robot.record(dict(kind='completion_certificate', **self.certificate()))
241             return self.stop_reason
242
243     def certificate(self):
244         absent = [c for c in range(1, 21) if c not in self.robot.discovered and self.coverage.absent(c)]
245         return dict(stop_reason=self.stop_reason, grid_level=self.coverage.level,
246                   square_side_m=self.coverage.half * 2, square_count=len(self.coverage.centers),
247                   absent_channels=absent, cleared_channels=sorted(self.robot.cleared),
248                   no_signal_positions={str(c): self.coverage.samples[c] for c in absent})

```

源文件组 9

q3/baselines/CG/run_robot.py

```

1  """ 问题三多策略研究版；尚未达到平均 200 秒/点目标。"""
2  import argparse
3  import datetime as dt
4  import hashlib
5  import json
6  from pathlib import Path
7  import shutil
8  import sys
9  import time
10 import uuid
11
12 ROOT = Path(__file__).resolve().parent
13
14
15 def main():
16     if hasattr(sys.stdout, 'reconfigure'):
17         sys.stdout.reconfigure(encoding='utf-8')
18     parser = argparse.ArgumentParser(description=__doc__)
19     parser.add_argument('--policy', choices=['literature_cautious'], default='literature_cautious', help='选择冻结策略，默认 v6 光学动作价值候选')
20     parser.add_argument('--team', help='默认复用旧策略的 config.json 队号')
21     parser.add_argument('--port', type=int, help='默认复用旧配置，缺省端口 2026')
22     parser.add_argument('--config', type=Path, help='指定含 team_no、port 的配置文件')
23     parser.add_argument('--check', action='store_true', help='仅检查依赖和配置，不连接模拟器')
24     args = parser.parse_args()
25     try:
26         import numpy
27         import scipy
28         import shapely
29         from api_client import Robot, BudgetReached
30         from strategy import Planner, VERSION, POLICIES
31     except ImportError as error:
32         print(f'缺少依赖：{error}\n 请安装 requirements.txt 中的依赖后重试。')
33         return 2
34     paths = [args.config] if args.config else [ROOT / 'config.json', ROOT.parent / 'simple-strategy' / 'config.json']
35     config_path = next((p for p in paths if p and p.is_file()), None)
36     config = json.loads(config_path.read_text(encoding='utf-8-sig')) if config_path else {}
37     team, port = args.team or config.get('team_no'), args.port or config.get('port', 2026)
38     if not team:
39         print('找不到队号配置，请使用 --config 指定旧 config.json，或使用 --team 参数。')
40         return 2
41     if args.check:
42         print(f'策略 {args.policy}；研究版，200 秒/点目标尚未达成。')
43         print(f'自检通过：Python {sys.version.split()[0]} / numpy {numpy.__version__} / scipy {scipy.__version__}')
44         print(f'配置可读；端口 {port}；未连接模拟器。')
45         return 0
46     print(f'问题 3 研究策略 {VERSION} / {args.policy} | 端口 {port}')
47     print('这是可运行的研究候选，尚未达到平均 200 秒/点目标。')
48     print('请在模拟器选择 【问题 3 演练测试】，开始后等待 “接口已就绪/等待机器狗进入”。')
49     print('接口无法查询演练/正式模式，程序按问题 3 全向干扰器规则工作。')
50     input('界面就绪后按 Enter 开始：')
51     folder = ROOT / 'runs' / (dt.datetime.now().strftime('%Y%m%d-%H%M%S') + '-' + uuid.uuid4().hex[:6])
52     robot = Robot(team, f'http://127.0.0.1:{port}', folder)
53     planner = Planner(robot, policy=args.policy)
54     snapshot = folder / 'source_snapshot'
55     snapshot.mkdir()
56     hashes = {}
57     for p in list(ROOT.glob('*.py')) + [ROOT / 'policies.json']:
58         shutil.copy2(p, snapshot / p.name)
59         hashes[p.name] = hashlib.sha256(p.read_bytes()).hexdigest()
60     (folder / 'manifest.json').write_text(json.dumps(dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy],
61     python=sys.version, numpy=numpy.__version__, scipy=scipy.__version__, shapely=shapely.__version__, source_sha256=hashes,
62     config_path=str(config_path), mode='user_selected_P3_practice_not_API_verified'),
63     ensure_ascii=False, indent=2), encoding='utf-8')
64     outcome, error_text, entered = 'completed', None, False
65     robot.started = time.monotonic()
66     try:
67         robot.call('/enter')
68         entered = True
69         planner.run()
70     except BudgetReached as error:
71         outcome, error_text = 'budget_stop', str(error)

```

```

72 except (Exception, KeyboardInterrupt) as error:
73     outcome, error_text = 'error', str(error) or '用户中断'
74     # Interruption during HTTP could leave execution state unknown.
75     if isinstance(error, KeyboardInterrupt):
76         robot.transport_uncertain = True
77 finally:
78     if entered and not robot.transport_uncertain:
79         try:
80             robot.call('/exit')
81         except Exception as error:
82             outcome = 'exit_unconfirmed'
83             error_text = (error_text or '') + '\n退出未确认: ' + str(error)
84     count = len(robot.cleared)
85     summary = dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy], outcome=outcome, error=error_text,
86                   exit_confirmed=robot.exit_confirmed, transport_uncertain=robot.transport_uncertain,
87                   cleared_count=count, cleared_channels=sorted(robot.cleared),
88                   observed_but_uncleared=sorted(robot.discovered - robot.cleared),
89                   measure_count=robot.measure_count, clear_failures=robot.clear_failures,
90                   scanned_sites=robot.scanned_sites, distance_m=robot.distance_m, switches=robot.switches,
91                   virtual_time_s=robot.virtual, average_clear_time_s=robot.virtual / count if count else None,
92                   local_elapsed_s=time.monotonic() - robot.started,
93                   actual_jammer_count=None, completion=planner.certificate(),
94                   note='实际目标总数和正式成绩以模拟器为准; 完整性证书以问题 3 全向规则为前提。')
95     (folder / 'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
96     robot.log.close()
97     print(f'\n结束: 清除 {count} 个; 虚拟时间 {robot.virtual:.1f} 秒; 移动 {robot.distance_m:.1f} 米。')
98     print(f'状态 {outcome}; 退出确认 {robot.exit_confirmed}; 结束依据 {planner.stop_reason}')
99     if error_text:
100         print(error_text)
101     print(f'运行记录: {folder}')
102     print('请保存模拟器结束界面、案例编码、实际目标数, 并导出官方 .jlog。')
103     return 0 if outcome == 'completed' and robot.exit_confirmed else 1
104
105
106 if __name__ == '__main__':
107     sys.exit(main())

```

源文件组 10

q3/baselines/CG/strategy.py

```

1  """Literature-inspired sequential cautious baseline; NOT an exact reproduction.
2
3  Vander Hook et al. (2014), Algorithm 1: principal-axis normal, shrinking
4  stand-off, nearer of two locations. Uniform polygon moments replace EKF;
5  bounded-region certificates replace probabilistic convergence claims.
6  No simulator truth is accessed. Frozen before official practice evaluation.
7  """
8  import json
9  import math
10 from pathlib import Path
11 from statistics import NormalDist
12 import numpy as np
13 from legacy_strategy import Planner as BasePlanner, CLEAR_RADIUS
14
15 VERSION = 'q3-literature-cautious-adaptation-20260913'
16 POLICIES = json.loads(Path(__file__).with_name('policies.json').read_text())
17
18 def polygon_moments(poly):
19     # Exact area moments of a uniform convex polygon; numerical translation.
20     origin = np.mean(poly, axis=0)
21     p = poly-origin; q = np.roll(p, 1, axis=0)
22     cross = p[:,0]*q[:,1]-q[:,0]*p[:,1]
23     area = cross.sum()/2
24     if abs(area)<1e-10:
25         return origin, (p.T@p)/max(1,len(p))
26     mean = ((p+q)*cross[:,None]).sum(axis=0)/(6*area)
27     xx = ((p[:,0]**2+p[:,0]*q[:,0]+q[:,0]**2)*cross).sum()/(12*area)
28     yy = ((p[:,1]**2+p[:,1]*q[:,1]+q[:,1]**2)*cross).sum()/(12*area)

```

```

29 xy = ((2*p[:,0]*p[:,1]+p[:,0]*q[:,1]+q[:,0]*p[:,1]+2*q[:,0]*q[:,1])*cross).sum()/(24*area)
30 return origin+mean, np.array([[xx,xy],[xy,yy]])-np.outer(mean,mean)
31
32 class CautiousPlanner(BasePlanner):
33     def __init__(self, robot, beta=.1, scan_radius_m=1200, **kwargs):
34         super().__init__(robot)
35         self.anchors = [(0,0.0)]+[(scan_radius_m*math.cos(i*math.pi/3),scan_radius_m*math.sin(i*math.pi/3)) for i in range(6)]
36         self.beta=beta
37         self.max_observations=kwargs.get('max_observations_per_target',40)
38
39     def cautious_point(self, track):
40         center,cov=polygon_moments(track.poly)
41         values,vectors=np.linalg.eigh(cov)
42         axis=vectors[:,1]; normal=np.array([-axis[1],axis[0]])
43         sigma_beta=math.pi/(2*NormalDist().inv_cdf(1-self.beta/2))
44         radius=math.sqrt(max(0,values[-1]))/math.sqrt(sigma_beta**2-math.radians(1)**2/3)
45         candidates=[]
46         for sign in [-1,1]:
47             # Range-limited adaptation: largest stand-off up to the proposed
48             # value for which every possible source remains within 999.9 m.
49             lo,hi=0.,radius
50             if np.max(np.linalg.norm(track.poly-center,axis=1))>999.9: continue
51             for _ in range(40):
52                 mid=(lo+hi)/2; point=center+sign*mid*normal
53                 if np.max(np.linalg.norm(track.poly-point,axis=1))<=999.9: lo=mid
54                 else: hi=mid
55             point=center+sign*lo*normal
56             if all(math.dist(point,h['position'])>=.1 for h in track.history):
57                 candidates.append(point)
58         if not candidates: return track.next_point(np.asarray(self.robot.position))
59         point=min(candidates,key=lambda q:math.dist(q,self.robot.position))
60         self.robot.record(dict(kind='cautious_choice',channel=track.channel,
61                               point=point.tolist(),moment_center=center.tolist(),proposed_standoff_m=radius,
62                               actual_standoff_m=float(np.linalg.norm(point-center)),region_radius_m=track.radius))
63         return point
64
65     def solve_track(self, track):
66         for _ in range(self.max_observations):
67             self.robot.check_budget()
68             point=track.safe_clear_point(self.robot.position)
69             if point is not None:
70                 self.robot.record(dict(kind='clear_certificate',channel=track.channel,
71                                       point=point.tolist(),worst_distance_m=float(np.max(np.linalg.norm(track.poly-point,axis=1)))))
72             if not self.robot.clear(tuple(point),track.channel):
73                 raise ArithmeticError('Bounded-region clear certificate failed')
74             return
75             result=self.observe(self.cautious_point(track),track.channel)
76             if result['measure_result']=='no_signal':
77                 raise ArithmeticError('Reception certificate failed')
78             raise RuntimeError('Cautious localization observation budget exhausted')
79
80 def Planner(robot, *, policy='literature_cautious'):
81     return CautiousPlanner(robot,**POLICIES[policy])

```

源文件组 11

q3/baselines/v4/clearance_geometry.py

q3/v6/clearance_geometry.py

```

1 """Euclidean projection onto the intersection of equal-radius clear disks."""
2 import math
3 import numpy as np
4
5
6 def nearest_clear_point(vertices,position,radius=19.5,feasible_point=None):
7     """Return the global nearest safe point, or None if no candidate is feasible.
8
9     An external projection lies on one circular arc, or a two-circle vertex.
10    Smooth-arc candidates are radial projections; vertices are enumerated.

```

```

11 A supplied feasible point is a numerical fallback, not an optimality claim.
12 """
13 v=np.asarray(vertices,float);p=np.asarray(position,float)
14 if not len(v):return None
15 def feasible(q):return np.max(np.linalg.norm(v-q,axis=1))<=radius+1e-8
16 if feasible(p):return p.copy()
17 candidates=[]
18 for center in v:
19     d=p-center;length=float(np.linalg.norm(d))
20     if length>1e-12:
21         q=center+radius*d/length
22         if feasible(q):candidates.append(q)
23 for i,a in enumerate(v):
24     for b in v[i:]:
25         d=b-a;length=float(np.linalg.norm(d))
26         if length<1e-10 or length>2*radius+1e-9:continue
27         offset=math.sqrt(max(0.,radius**2-length**2/4))*np.array([-d[1],d[0]])/length
28         for q in [(a+b)/2+offset,(a+b)/2-offset]:
29             if feasible(q):candidates.append(q)
30 if feasible_point is not None and feasible(np.asarray(feasible_point)):
31     candidates.append(np.asarray(feasible_point).copy())
32 if not candidates:return None
33 return min(candidates,key=lambda q:float(np.linalg.norm(q-p))).copy()

```

源文件组 12

q3/baselines/v4/combined_cost_cover.py

q3/v6/combined_cost_cover.py

```

1 """Combine the independently developed localization and scan-cover candidates."""
2 from cost_aware import CostAwarePlanner
3 from merged_cover import MergedCoverPlanner
4
5
6 class CombinedCostCoverPlanner(CostAwarePlanner):
7     candidates = MergedCoverPlanner.candidates
8
9     def __init__(self, robot, *, merge_stations=True, merge_target_scans=True, **kwargs):
10         super().__init__(robot, **kwargs)
11         self.merge_stations = merge_stations
12         self.merge_target_scans = merge_target_scans

```

源文件组 13

q3/baselines/v4/convex_cover.py

q3/v6/convex_cover.py

```

1 """Continuous waypoint relocation with an explicit full-coverage constraint."""
2 import math
3 import numpy as np
4 from scipy.optimize import minimize
5 from scipy.spatial import ConvexHull
6 from joint_strategies import JointPlanner,open_route
7
8
9 class ConvexCoverPlanner(JointPlanner):
10     def __init__(self,robot,*,cover_sweeps=2,**kwargs):
11         super().__init__(robot,family='adaptive',**kwargs)
12         self.cover_sweeps=cover_sweeps
13
14     def candidates(self):
15         targets=[]
16         for c,t in self.tracks.items():
17             if c in self.robot.cleared:continue
18             q=t.safe_clear_point(self.robot.position)
19             q=t.center if q is None else q

```

```

20     targets.append(('target',c,np.asarray(q)))
21     unknown=self.unknown()
22     if not unknown: return targets
23     actual=self.coverage.covered[unknown[0]]
24     # Targets are mandatory route vertices but optional sensing sites.
25     points=[n[2].copy() for n in targets]+[np.array(p) for p in self.anchors]
26     n_target=len(targets)
27     active=np.ones(len(points),dtype=bool)
28     present=np.ones(len(points),dtype=bool)
29     masks=[self.coverage.mask(p) for p in points]
30     start=np.asarray(self.robot.position)
31     def route_order():
32         indices=[i for i in range(len(points)) if present[i]]
33         return [indices[j] for j in open_route(start,[points[i] for i in indices])]
34     def total_without(i):
35         total=actual.copy()
36         for j in range(len(points)):
37             if j!=i and active[j]:total+=masks[j]
38         return total
39     def prune():
40         while True:
41             order=route_order();gains=[]
42             for k,i in enumerate(order):
43                 if not active[i]:continue
44                 gain=30*len(unknown)
45                 if i>=n_target:
46                     a=start if k==0 else points[order[k-1]]
47                     gain+=np.linalg.norm(points[i]-a)
48                     if k+1<len(order):
49                         b=points[order[k+1]]
50                         gain+=np.linalg.norm(points[i]-b)-np.linalg.norm(a-b)
51                 gains.append((gain,i))
52             removed=False
53             for gain,i in sorted(gains,reverse=True):
54                 if np.all(total_without(i)):
55                     active[i]=False
56                     if i>=n_target:present[i]=False
57                     removed=True;break
58             if not removed:break
59     prune()
60     for _ in range(self.cover_sweeps):
61         order=route_order()
62         improvement=0.
63         for k,i in enumerate(order):
64             if i<n_target or not active[i]:continue
65             needed=self.coverage.centers[~total_without(i)]
66             if not len(needed):continue
67             h=self.coverage.half
68             corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in [(-1,-1),(-1,1),(1,-1),(1,1)]]])
69             witnesses=corners[ConvexHull(corners).vertices]/1000
70             before=points[i]/1000
71             previous=(start if k==0 else points[order[k-1]])/1000
72             following=points[order[k+1]]/1000 if k+1<len(order) else None
73             def objective(q):
74                 value=np.sqrt(np.dot(q-previous,q-previous)+1e-16)
75                 if following is not None:value+=np.sqrt(np.dot(q-following,q-following)+1e-16)
76                 return float(value)
77             def gradient(q):
78                 value=(q-previous)/max(np.linalg.norm(q-previous),1e-8)
79                 if following is not None:value+=(q-following)/max(np.linalg.norm(q-following),1e-8)
80                 return value
81             cons={'type':'ineq','fun':lambda q: .9998999**2-np.sum((q-witnesses)**2,axis=1),
82                  'jac':lambda q:-2*(q-witnesses)}
83             sol=minimize(objective,before,jac=gradient,constraints=cons,method='SLSQP',
84                          options={'ftol':1e-10,'maxiter':60})
85             if np.max(np.linalg.norm(witnesses-sol.x,axis=1))<=.99999 and objective(sol.x)<=objective(before)+1e-8:
86                 improvement+=objective(before)-objective(sol.x)
87                 points[i]=sol.x*1000;masks[i]=self.coverage.mask(points[i])
88         prune()
89         if improvement<1e-5:break
90     union=actual.copy()

```

```

91     for i in range(len(points)):
92         if active[i]:union|=masks[i]
93     if not np.all(union):raise ArithmeticError('Relocated cover lost cells')
94     scans=[('scan',-100-i,points[i]) for i in range(len(points)) if active[i]]
95     nodes=targets+scans
96     self.cover_targets=set()
97     order=open_route(start,[n[2] for n in nodes])
98     if nodes[order[0]][0]!='target':
99         first=nodes[order[0]]
100         matches=[n for n in scans if math.dist(n[2],first[2])<1e-5]
101         if matches:return [matches[0]]
102     return nodes

```

源文件组 14

q3/baselines/v4/cost_aware.py

q3/v6/cost_aware.py

```

1  """Cost-aware localization and paid sharing of intermediate sensing sites.
2
3  Inspired by the cost objectives of Vander Hook (2014), multi-target planning
4  of Engin & Isler (2020), and approach-to-source routing in Gu et al. (2018).
5  No Gaussian guarantees, true target inputs, or free simultaneous sensing.
6  """
7  import copy
8  import math
9  import numpy as np
10 from convex_cover import ConvexCoverPlanner
11 from legacy_strategy import Track,CLEAR_RADIUS,EPS
12 from bearing_geometry import diameter
13 from clearance_geometry import nearest_clear_point
14 from reception_region import reception_violation
15
16
17 class RefinedTrack(Track):
18     def safe_clear_point(self,position):
19         if self.radius>CLEAR_RADIUS:return None
20         return nearest_clear_point(self.poly,position,CLEAR_RADIUS,self.center)
21
22
23 def reception_safe(track,q):
24     if np.max(np.linalg.norm(track.poly-q,axis=1))<=999.9:return True
25     first=track.history[0]
26     if first['measure_result']!='direction':return False
27     angle=math.radians(first['svd_deg']);u=np.array([math.cos(angle),math.sin(angle)])
28     v=np.array([-u[1],u[0]]);d=q-np.asarray(first['position'])
29     a,b=np.dot(d,u),np.dot(d,v);l2=np.dot(d,d);ep=math.radians(EPS)
30     return l2<=1000**2-1e-3 and l2<=2000*(a*math.cos(ep)-abs(b)*math.sin(ep))-1e-3
31
32
33 def forecast(track,start,first_point,truth,error,exit_point=None):
34     t=copy.copy(track);t.poly=track.poly.copy();t.history=list(track.history)
35     p=np.asarray(start);q=first_point;cost=0.
36     for k in range(5):
37         cost+=float(np.linalg.norm(q-p))/5+5;p=q
38         if np.linalg.norm(truth-q)<=5:obs={'measure_result':'near'}
39         else:
40             angle=math.degrees(math.atan2(truth[1]-q[1],truth[0]-q[0]))
41             obs={'measure_result':'direction','svd_deg':round((angle+error)%360,2)%360}
42         t.update(q,obs)
43         clear=t.safe_clear_point(q)
44         if clear is not None:
45             cost+=float(np.linalg.norm(clear-q))/5+5
46             if exit_point is not None:cost+=float(np.linalg.norm(clear-exit_point))/5
47         return cost
48         q=t.next_point(p)
49     return cost+1000.
50
51

```

```

52 class CostAwarePlanner(ConvexCoverPlanner):
53     def __init__(self, robot, *, projection=True, cost_action=False, cost_risk=.25,
54                 intermediate_known=False, intermediate_gain=2, exit_cost=False, exact_reception=False, **kwargs):
55         super().__init__(robot, **kwargs)
56         self.projection=projection; self.cost_action=cost_action; self.cost_risk=cost_risk
57         self.intermediate_known=intermediate_known; self.intermediate_gain=intermediate_gain
58         self.exit_cost=exit_cost
59         self.exact_reception=exact_reception
60
61     def observe(self, point, channel):
62         result=super().observe(point, channel)
63         if self.projection and channel in self.tracks and not isinstance(self.tracks[channel], RefinedTrack):
64             track=RefinedTrack(channel); track.__dict__=self.tracks[channel].__dict__
65             self.tracks[channel]=track
66         return result
67
68     def pick_point(self, track):
69         start=np.asarray(self.robot.position); center=track.center
70         default=track.next_point(start)
71         _, (i, j)=diameter(track.poly); a, b=track.poly[i], track.poly[j]
72         direction=b-a; normal=np.array([-direction[1], direction[0]])/max(np.linalg.norm(direction), 1e-9)
73         candidates=[default, start, center]
74         for f in [.35, .7, 1.]:
75             mid=start+f*(center-start)
76             for offset in [-60., 0., 60.]: candidates.append(mid+offset*normal)
77         unique=[]
78         for q in candidates:
79             if any(math.dist(q, h['position'])<.1 for h in track.history): continue
80             if any(np.linalg.norm(q-p)<.1 for p in unique): continue
81             safe=reception_safe(track, q)
82             if not safe and self.exact_reception:
83                 safe=reception_violation(track.poly, track.history[0]['position'], q)[0]<=-.01
84             if safe: unique.append(np.asarray(q))
85         samples=[a*.9+center*.1, b*.9+center*.1, center]
86         others=[t.center for c, t in self.tracks.items() if c!=track.channel and c not in self.robot.cleared]
87         exit_point=min(others, key=lambda p: np.linalg.norm(p-center)) if self.exit_cost and others else None
88         scores=[]
89         for q in unique:
90             values=[forecast(track, start, q, x, e, exit_point) for x in samples for e in [-1., 0., 1.]]
91             score=(1-self.cost_risk)*np.mean(values)+self.cost_risk*np.max(values)
92             scores.append(float(score))
93         if not scores: return default
94         best=int(np.argmin(scores))
95         self.robot.record(dict(kind='cost_action', channel=track.channel,
96                               candidates=[dict(point=q.tolist(), score_s=s) for q, s in zip(unique, scores)], selected=best,
97                               scoring='representative geometric scenarios; not a continuous minimax certificate'))
98         return unique[best]
99
100     def solve_track(self, track):
101         # The first observation may replace Track by RefinedTrack; always refresh.
102         channel=track.channel
103         if not (self.cost_action or self.intermediate_known or self.intermediate_gain<1.):
104             return super().solve_track(track)
105         for _ in range(16):
106             self.robot.check_budget(); track=self.tracks[channel]
107             if track.radius<=CLEAR_RADIUS: return super().solve_track(track)
108             q=self.pick_point(track) if self.cost_action else track.next_point(np.asarray(self.robot.position))
109             if self.observe(q, channel)['measure_result']=='no_signal':
110                 raise ArithmeticError('Certified reception point lost signal')
111             if self.intermediate_known or self.intermediate_gain<1.:
112                 old_gain, old_info=self.scan_gain, self.info_maxobs
113                 self.scan_gain=self.intermediate_gain
114                 if not self.intermediate_known: self.info_maxobs=0
115                 try: self.at_site_scan(self.robot.position)
116                 finally: self.scan_gain, self.info_maxobs=old_gain, old_info
117             raise RuntimeError('Cost-aware localization iteration limit')

```

源文件组 15

q3/baselines/v4/exact_coverage.py

q3/v6/exact_coverage.py

```

1  """Sufficient circle-arrangement certificate; coarse samples only guide routing."""
2  import math
3  import numpy as np
4  from legacy_strategy import Coverage
5
6
7  def arrangement_certificate(points, radius=999.99, arena_radius=1800.):
8      pts=np.unique(np.asarray(points,float).reshape(-1,2),axis=0)
9      if not len(pts):return False,np.array([arena_radius,0.])
10     angles=[0.]
11     for p in pts:
12         d=float(np.linalg.norm(p))
13         if d+arena_radius<=radius:return True,None
14         if d<1e-10:continue
15         cosine=(d*d+arena_radius**2-radius**2)/(2*d*arena_radius)
16         if -1<=cosine<=1:
17             a=math.atan2(p[1],p[0]);delta=math.acos(cosine)
18             angles += [(a-delta)%(2*math.pi),(a+delta)%(2*math.pi)]
19     angles=sorted(set(angles))
20     for i,a in enumerate(angles):
21         b=angles[i+1] if i+1<len(angles) else angles[0]+2*math.pi
22         q=arena_radius*np.array([math.cos((a+b)/2),math.sin((a+b)/2)])
23         if np.min(np.linalg.norm(pts-q,axis=1))>radius+1e-7:return False,q
24     # Any bounded uncovered interior component must have a vertex made by
25     # circle boundaries. Requiring a third disk's strict interior at all such
26     # crossings excludes it. Degenerate coincidences may be rejected safely.
27     for i in range(len(pts)):
28         for j in range(i):
29             v=pts[i]-pts[j];d=float(np.linalg.norm(v))
30             if d<1e-10 or d>2*radius:continue
31             mid=(pts[i]+pts[j])/2
32             normal=np.array([-v[1],v[0]])/d
33             half=math.sqrt(max(0,radius**2-d*d/4))
34             for q in [mid+half*normal,mid-half*normal]:
35                 if np.linalg.norm(q)>=arena_radius-1e-7:continue
36                 distances=np.linalg.norm(pts-q,axis=1)
37                 distances[[i,j]]=float('inf')
38                 if np.min(distances)>=radius-1e-7:return False,q
39     return True,None
40
41
42 class ExactCoverage(Coverage):
43     def __init__(self):
44         super().__init__(level=6)
45         self.centers=self.centers[np.linalg.norm(self.centers,axis=1)<=1800]
46         self.covered=np.zeros((21,len(self.centers)),dtype=bool)
47         self.cache={}
48
49     def mask(self,point):
50         return np.sum((self.centers-np.asarray(point))**2,axis=1)<=999.99**2
51
52     def status(self,channel):
53         key=tuple(tuple(p) for p in self.samples[channel])
54         if key not in self.cache:self.cache[key]=arrangement_certificate(key)
55         return self.cache[key]
56
57     def absent(self,channel):return self.status(channel)[0]
```

源文件组 16

q3/baselines/v4/joint_strategies.py

q3/v6/joint_strategies.py

```

1  """Independent candidate families for P3; all maintain the same clear certificate."""
2  import math
3  import numpy as np
4  from scipy.spatial import ConvexHull
5  from legacy_strategy import Planner as BasePlanner, CLEAR_RADIUS, Track
6  from bearing_geometry import enclosing_circle, wedge, clip_halfplanes
7  from exact_coverage import ExactCoverage
8
9
10 def exclude_disk_hull(poly, center, radius=999.9):
11     """Convex outer hull of P outside a disk (uses P-boundary candidates).
12
13     A maximum of a linear functional over P minus an open disk is attained
14     at an outside vertex or an edge/circle crossing; the inner arc is concave.
15     Shrink the excluded radius by .1 m to protect floating-point intersections.
16     """
17     center=np.asarray(center);points=[]
18     for i,p in enumerate(poly):
19         q=poly[(i+1)%len(poly)];v=q-p;offset=p-center
20         if np.dot(offset,offset)>=radius**2:points.append(p)
21         a=float(np.dot(v,v));b=2*float(np.dot(v,offset));c=float(np.dot(offset,offset)-radius**2)
22         disc=b*b-4*a*c
23         if a>1e-18 and disc>=0:
24             for t in [(-b-math.sqrt(disc))/(2*a),(-b+math.sqrt(disc))/(2*a)]:
25                 if 0<=t<=1:points.append(p+t*v)
26     if not points:raise ArithmeticError('Negative measurement contradicts region')
27     points=np.unique(np.round(np.asarray(points),9),axis=0)
28     if len(points)<3:return points
29     try:return points[ConvexHull(points).vertices]
30     except Exception:
31         low,high=points.min(axis=0)-1e-7,points.max(axis=0)+1e-7
32         return np.array([low,[high[0],low[1]],high,[low[0],high[1]]])
33
34
35 def exclude_union_hull(poly, centers, radius=999.8):
36     """Joint negative evidence, without convexifying between exclusions."""
37     if not centers:return poly
38     cs=np.unique(np.asarray(centers,float),axis=0)
39     edges=np.roll(poly,-1,axis=0)-poly
40     candidates=[p for p in poly]
41     for p,v in zip(poly,edges):
42         a=float(np.dot(v,v))
43         if a<1e-18:continue
44         for center in cs:
45             offset=p-center;b=2*float(np.dot(v,offset));c=float(np.dot(offset,offset)-radius**2)
46             disc=b*b-4*a*c
47             if disc<0:continue
48             for t in [(-b-math.sqrt(disc))/(2*a),(-b+math.sqrt(disc))/(2*a)]:
49                 if 0<=t<=1:candidates.append(p+t*v)
50     for i,c in enumerate(cs):
51         for d in cs[i:]:
52             delta=c-d:length=float(np.linalg.norm(delta))
53             if length<1e-9 or length>2*radius:continue
54             normal=np.array([-delta[1],delta[0]])/length
55             offset=math.sqrt(max(0.,radius**2-length**2/4))*normal
56             candidates += [(c+d)/2+offset,(c+d)/2-offset]
57     points=np.asarray(candidates)
58     outside=np.all(np.sum((points[:,None,:]-cs[None,:,:])**2,axis=2)>=radius**2-1e-4,axis=1)
59     delta=points[:,None,:]-poly[None,:,:]
60     inside=np.all(edges[None,:,0]*delta[:,1]-edges[None,:,1]*delta[:,0]>=-1e-6*np.maximum(1.,np.linalg.norm(edges,axis=1)),axis=1)
61     points=np.unique(np.round(points[outside&inside],9),axis=0)
62     if not len(points):raise ArithmeticError('Joint negative evidence yields empty region')
63     if len(points)<3:return points
64     return points[ConvexHull(points).vertices]
65
66
67 def open_route(start, points, end=None, restarts=1):
68     """Nearest-neighbour initialization + open-path 2-opt, fixed start, free end."""
69     if not len(points):
70         return []
71     pts = np.vstack([start, points])

```

```

72 D = np.linalg.norm(pts[:, None] - pts[None, :], axis=2)
73 end_cost=np.zeros(len(pts)) if end is None else np.linalg.norm(pts-end,axis=1)
74 def length(route):return float(sum(D[a,b] for a,b in zip(route,route[1:]))+end_cost[route[-1]])
75 rng=np.random.default_rng(42)
76 best_route,best_length=None,float('inf')
77 for restart in range(restarts):
78     left = set(range(1, len(pts)))
79     route = [0]
80     if restart:
81         first=sorted(left,key=lambda j:D[0,j])[(restart-1)%len(left)]
82         route.append(first);left.remove(first)
83     while left:
84         q = min(left, key=lambda j: D[route[-1], j])
85         route.append(q);left.remove(q)
86     for _ in range(50):
87         gain, best = 1e-8, None
88         for i in range(1, len(route) - 1):
89             for j in range(i + 1, len(route)):
90                 old = D[route[i-1], route[i]]
91                 new = D[route[i-1], route[j]]
92                 if j + 1 < len(route):
93                     old += D[route[j], route[j+1]]
94                     new += D[route[i], route[j+1]]
95                 else:
96                     old += end_cost[route[j]]
97                     new += end_cost[route[i]]
98                 if old - new > gain:
99                     gain, best = old-new, (i, j)
100     if best is not None:
101         i,j=best;route[i:j+1]=reversed(route[i:j+1]);continue
102     if restarts>1:
103         old_length=length(route);replacement=None
104         for i in range(1,len(route)):
105             reduced=route[:i]+route[i+1:]
106             for j in range(1,len(route)):
107                 trial=reduced[:j]+[route[i]]+reduced[j:]
108                 value=length(trial)
109                 if value<old_length-1e-7:
110                     old_length,replacement=value,trial
111             if replacement is not None:route=replacement;continue
112         break
113     value=length(route)
114     if value<best_length:best_length,best_route=value,route
115 return [i-1 for i in best_route[1:]]
116
117
118 class JointPlanner(BasePlanner):
119     def __init__(self, robot, *, family='joint', scan_gain=.05, info_distance=150.,
120                 initial_baseline=0., route_on=True, info_radius=60., info_maxobs=2,
121                 anchor_radius=1350., initial_force=True, initial_rotation=0., negative_regions=False,
122                 cover_move_weight=1., grid_step=450., one_step=False, probe_length=0., route_bias=0.,route_restarts=1,
123                 trial_radius=0.,trial_estimator='mec',recon_radius=0.,recon_count=3,
124                 info_range=float('inf'),rare_cover=False,exact_cover=False,smart_info=False,rotate_cover=False,
125                 joint_negatives=False,shared_waypoint=False,explore_bonus=0.,expected_count=13.,coverage_value=False,coverage_value_threshold_s=10.,
126                 ↵ motion='side',first_fraction=1.,radius_coupling=False):
127         super().__init__(robot)
128         self.family = family
129         self.scan_gain = scan_gain
130         self.info_distance = info_distance
131         self.initial_baseline = initial_baseline
132         self.route_on = route_on
133         self.info_radius = info_radius
134         self.info_maxobs = info_maxobs
135         self.attempts = {}
136         self.anchors = [(0,0.)]+[(anchor_radius*math.cos(i*math.pi/3),anchor_radius*math.sin(i*math.pi/3)) for i in range(6)]
137         self.initial_force,self.initial_rotation=initial_force,initial_rotation
138         self.negative_regions=negative_regions
139         self.cover_move_weight=cover_move_weight
140         self.grid_step=grid_step
141         self.static_cover=None
142         self.cover_targets=set()

```

```

142     self.one_step=one_step
143     self.probe_length,self.route_bias=probe_length,route_bias
144     self.route_restarts=route_restarts
145     self.trial_radius,self.trial_estimator=trial_radius,trial_estimator
146     self.failed_trials={}
147     self.recon_radius,self.recon_count=recon_radius,recon_count
148     self.info_range,self.rare_cover=info_range,rare_cover
149     self.exact_cover=exact_cover
150     self.smart_info=smart_info
151     self.rotate_cover=rotate_cover
152     self.rotation_cache=None
153     self.joint_negatives=joint_negatives
154     if joint_negatives:self.negative_regions=True
155     self.shared_waypoint=shared_waypoint
156     self.explore_bonus,self.expected_count=explore_bonus,expected_count
157     self.coverage_value,self.coverage_value_threshold_s=coverage_value,coverage_value_threshold_s
158     self.motion,self.first_fraction=motion,first_fraction
159     self.radius_coupling=radius_coupling
160     if radius_coupling:self.negative_regions=True
161     if exact_cover:self.coverage=ExactCoverage()
162     self.iterations = 0
163
164     def unknown(self):
165         if len(self.robot.discovered)==16:return []
166         return super().unknown()
167
168     def solve_track(self,track):
169         if self.motion=='midpoint':
170             for _ in range(16):
171                 self.robot.check_budget()
172                 if track.radius<=CLEAR_RADIUS:return super().solve_track(track)
173                 q=track.center.copy()
174                 if len(track.history)==1:
175                     s=np.asarray(track.history[0]['position'])
176                     q=s+self.first_fraction*(q-s)
177                 if any(math.dist(q,h['position'])<.1 for h in track.history):
178                     q=track.next_point(np.asarray(self.robot.position))
179                 if self.observe(q,track.channel)['measure_result']=='no_signal':
180                     raise ArithmeticError('Midpoint reception certificate failed')
181                 if self.one_step:return
182                 raise RuntimeError('Midpoint localization exhausted budget')
183             if CLEAR_RADIUS<track.radius<=self.trial_radius:
184                 q=track.center.copy()
185                 if self.trial_estimator=='wls':
186                     A=[];b=[]
187                     for h in track.history:
188                         if h['measure_result']!='direction':continue
189                         a=math.radians(h['svd_deg']);s=np.asarray(h['position']);w=max(10.,np.linalg.norm(q-s))
190                         normal=np.array([-math.sin(a),math.cos(a)])/w
191                         A.append(normal);b.append(np.dot(normal,s))
192                     if len(A)>=2:q=np.linalg.lstsq(np.asarray(A),np.asarray(b),rcond=None)[0]
193                 previous=self.failed_trials.get(track.channel,[])
194                 if not any(math.dist(q,p)<2 for p in previous):
195                     self.robot.record(dict(kind='speculative_clear',channel=track.channel,point=q.tolist(),
196                                             region_radius_m=track.radius,estimator=self.trial_estimator))
197                     if self.robot.clear(tuple(q),track.channel):return
198                     self.failed_trials.setdefault(track.channel,[]).append(tuple(q))
199                     if self.one_step:return
200             if self.probe_length and len(track.history)==1 and track.history[0]['measure_result']=='direction':
201                 first=track.history[0];s=np.asarray(first['position']);a=math.radians(first['svd_deg'])
202                 u=np.array([math.cos(a),math.sin(a)]);v=np.array([-math.sin(a),math.cos(a)])
203                 length=self.probe_length
204                 forward=max(.35*length,length*length/1900+length*.018)
205                 lateral=math.sqrt(length*length-forward*forward)
206                 choices=[s+forward*u+lateral*v,s+forward*u-lateral*v]
207                 q=min(choices,key=lambda q:math.dist(q,self.robot.position))
208                 if self.observe(q,track.channel)['measure_result']=='no_signal':
209                     raise ArithmeticError('Q2 short-baseline reception certificate failed')
210                 if self.one_step:return
211             if not self.one_step or track.radius <= CLEAR_RADIUS:
212                 return super().solve_track(track)

```

```

213     q=track.next_point(np.asarray(self.robot.position))
214     if self.observe(q,track.channel)['measure_result']=='no_signal':
215         raise ArithmeticError('Guaranteed next observation lost reception')
216
217 def observe(self, point, channel):
218     if not self.negative_regions:
219         result = super().observe(point, channel)
220     else:
221         point=tuple(float(v) for v in point)
222         result=self.robot.measure(point,channel)
223         if result['measure_result']=='no_signal':self.coverage.add(channel,point)
224     else:
225         track=self.tracks.setdefault(channel,Track(channel))
226         track.update(point,result)
227     if channel in self.tracks:
228         track=self.tracks[channel]
229         if self.radius_coupling:
230             for positive in track.history:
231                 s=np.asarray(positive['position'])
232                 for negative in self.coverage.samples[channel]:
233                     n=np.asarray(negative)
234                     normal=2*(n-s)
235                     offset=float(np.dot(n,n)-np.dot(s,s))+1e-6*np.linalg.norm(normal)
236                     track.poly=clip_halfplanes(track.poly,[normal],[offset])
237                 if not len(track.poly):raise ArithmeticError('Fixed-radius positive/negative conflict')
238         if self.joint_negatives:
239             track.poly=exclude_union_hull(track.poly,self.coverage.samples[channel])
240         else:
241             for p in self.coverage.samples[channel]:
242                 track.poly=exclude_disk_hull(track.poly,p)
243             track.center,track.radius,_=enclosing_circle(track.poly)
244             self.robot.record(dict(kind='region',channel=channel,position=list(point),
245                                   radius_m=track.radius,center=track.center.tolist(),vertices=track.poly.tolist(),
246                                   observations=len(track.history),negative_positions=self.coverage.samples[channel]))
247         self.attempts[channel] = tuple(point)
248     return result
249
250 def at_site_scan(self, point, force=False):
251     """Unknown scans buy coverage; known scans buy intersecting bearings."""
252     unknown = self.unknown()
253     scan_unknown=force or self.coverage.gain(point,unknown)>=self.scan_gain
254     if not force and self.coverage_value and unknown and math.dist(point,self.robot.position)<1e-6:
255         scan_unknown=self.coverage_time_value(point,unknown)>self.coverage_value_threshold_s
256     channels = set(unknown if scan_unknown else [])
257     for c, track in self.tracks.items():
258         if c in self.robot.cleared or track.radius <= self.info_radius or len(track.history) >= self.info_maxobs:
259             continue
260         if math.dist(point,track.center)>self.info_range:continue
261         if c in self.attempts and math.dist(point,self.attempts[c]) < self.info_distance:
262             continue
263         if self.smart_info and len(track.history)>=2 and math.dist(point,track.center)>100:
264             a=math.degrees(math.atan2(track.center[1]-point[1],track.center[0]-point[0]))
265             radii=[]
266             for reading in [a-1.,a,a+1.]:
267                 A,b=wedge(point,reading,1.0051)
268                 posterior=clip_halfplanes(track.poly,A,b)
269                 if len(posterior):radii.append(enclosing_circle(posterior)[1])
270                 if not radii or not (max(radii)<=19.5 or np.mean(radii)<=.4*track.radius):continue
271         if min(math.dist(point, h['position']) for h in track.history) >= self.info_distance:
272             # No-signal is allowed for batch measurements; never corrupt P.
273             channels.add(c)
274     if channels:
275         self.robot.scanned_sites += 1
276     for c in sorted(channels, key=lambda c: (c != self.robot.channel, c)):
277         self.observe(point, c)
278
279 def coverage_time_value(self,point,unknown):
280     if self.coverage.gain(point,unknown)<.005:return -1e9
281 def estimated_cost():
282     nodes=self.candidates()
283     order=open_route(self.robot.position,[n[2] for n in nodes])

```

```

284     p=np.asarray(self.robot.position);distance=0.
285     for i in order:distance+=np.linalg.norm(nodes[i][2]-p);p=nodes[i][2]
286     return distance/5+6*len(unknown)*sum(n[0]=='scan' for n in nodes)
287 before=estimated_cost()
288 saved=self.coverage.covered[unknown].copy()
289 try:
290     self.coverage.covered[unknown]=self.coverage.mask(point)
291     after=estimated_cost()
292 finally:self.coverage.covered[unknown]=saved
293 return float(before-after-6*len(unknown))
294
295 def candidates(self):
296     pending = [t for c,t in self.tracks.items() if c not in self.robot.cleared]
297     nodes = []
298     for t in pending:
299         q = t.safe_clear_point(self.robot.position)
300         q = t.center if q is None else q
301         if self.route_bias and len(t.history)==1:
302             q=q+self.route_bias*(q-np.asarray(t.history[0]['position']))
303         nodes.append(('target', t.channel, np.asarray(q)))
304     unknown = self.unknown()
305     if not unknown:
306         return nodes
307     if not nodes and self.exact_cover and np.all(self.coverage.covered[unknown[0]]):
308         return [('scan',-2,self.coverage.status(unknown[0])[1])]
309     if self.family == 'adaptive':
310         return self.adaptive_cover(nodes,unknown)
311     if self.rotate_cover:
312         return self.rotated_cover(nodes,unknown)
313     # Keep a conservative fallback cover from original anchors. We may
314     # anticipate target visits for route scoring, but stopping uses real scans.
315     covered = self.coverage.covered[unknown[0]].copy()
316     if self.family != 'anchors':
317         for _, _, q in nodes:
318             covered |= self.coverage.mask(q)
319     anchors = [(i,p,self.coverage.mask(p)) for i,p in enumerate(self.anchors)]
320     needed = []
321     while not np.all(covered):
322         if not anchors:
323             raise RuntimeError('Search fallback cover exhausted')
324         # Greedy set cover only selects waypoints; it does not certify absence.
325         a = max(anchors, key=lambda a: int(np.count_nonzero(a[2] & ~covered)))
326         gain = np.count_nonzero(a[2] & ~covered)
327         if not gain:
328             raise RuntimeError('No progress in waypoint cover')
329         needed.append(('scan', a[0], np.asarray(a[1])))
330         covered |= a[2]
331         anchors.remove(a)
332     return nodes + needed
333
334 def rotated_cover(self,nodes,unknown):
335     if self.rotation_cache is None:
336         self.rotation_cache=[]
337     for radius in [1200.,1350.,1550.]:
338         for rotation in range(0,60,10):
339             pts=[np.zeros(2)]+[radius*np.array([math.cos(math.radians(rotation+i*60)),math.sin(math.radians(rotation+i*60))]) for i in range(6)]
340             self.rotation_cache.append([(i,p,self.coverage.mask(p)) for i,p in enumerate(pts)])
341     actual=self.coverage.covered[unknown[0]].copy()
342     for _,_,q in nodes:actual|=self.coverage.mask(q)
343     best=(float('inf'),None)
344     for choice in self.rotation_cache:
345         covered=actual.copy();remaining=choice.copy();needed=[]
346         while not np.all(covered):
347             a=max(remaining,key=lambda a:np.count_nonzero(a[2]&~covered))
348             if not np.any(a[2]&~covered):break
349             needed.append(('scan',a[0],a[1]));covered|=a[2];remaining.remove(a)
350     if not np.all(covered):continue
351     candidate=nodes+needed
352     order=open_route(self.robot.position,[n[2] for n in candidate])
353     p=np.asarray(self.robot.position);cost=0.
354     for i in order:cost+=np.linalg.norm(candidate[i][2]-p);p=candidate[i][2]

```

```

355         cost+=30*len(unknown)*len(needed)
356         if cost<best[0]:best=(cost,candidate)
357     if best[1] is None:raise RuntimeError('No rotated cover found')
358     return best[1]
359
360 def certificate(self):
361     result=super().certificate()
362     result['coverage_method']='circle_arrangement_sufficient' if self.exact_cover else 'whole_square_cover'
363     if self.exact_cover:result['grid_role']='routing_heuristic_only_not_completion_proof'
364     return result
365
366 def adaptive_cover(self,nodes,unknown):
367     def bits(mask):
368         return int.from_bytes(np.packbits(mask,bitorder='little').tobytes(),'little')
369     if self.static_cover is None:
370         pts=[np.array(p) for p in self.anchors]
371         pts += [np.array([x,y]) for x in np.arange(-1350,1351,self.grid_step)
372                 for y in np.arange(-1350,1351,self.grid_step) if math.hypot(x,y)<=1750]
373         self.static_cover=[(q,bits(self.coverage.mask(q))) for q in pts]
374     covered=bits(self.coverage.covered[unknown[0]])
375     full=(1<<len(self.coverage.centers))-1
376     p=np.asarray(self.robot.position)
377     route=open_route(p,[n[2] for n in nodes])
378     route_points=[p]+[nodes[i][2] for i in route]
379     # Marginal route insertion distance, not straight distance from dog.
380     def detour(q):
381         best=float(np.linalg.norm(route_points[-1]-q))
382         for a,b in zip(route_points,route_points[1:]):
383             best=min(best,float(np.linalg.norm(a-q)+np.linalg.norm(q-b)-np.linalg.norm(a-b)))
384         return max(0,best)
385     candidates=[('scan',-1,p,bits(self.coverage.mask(p)),0.)]
386     candidates += [(kind,c,q,bits(self.coverage.mask(q)),0.) for kind,c,q in nodes]
387     candidates += [('scan',i,q,mask,detour(q)) for i,(q,mask) in enumerate(self.static_cover)]
388     needed=[],self.cover_targets=set();selected=[];actual=covered
389     groups=[]
390     if self.rare_cover:
391         arrays=np.array([np.unpackbits(np.frombuffer(a[3].to_bytes((len(self.coverage.centers)+7)//8,'little'),dtype=np.uint8,bitorder='little')[:len(self.coverage.centers)]) for a in
392                             ↪   range.centers]) for a in
393                             ↪   candidates])
394         frequency=np.maximum(arrays.sum(axis=0),1)
395         weights=np.ceil(100/frequency).astype(int)
396         groups=[(w,bits(weights==w)) for w in np.unique(weights)]
397     while covered != full:
398         missing=full^covered
399         def score(a):
400             gain=sum(w*(a[3]&missing&mask).bit_count() for w,mask in groups) if groups else (a[3]&missing).bit_count()
401             return gain/(6*len(unknown)+self.cover_move_weight*a[4]/5)
402         best=max(candidates,key=score)
403         if not best[3]&missing:raise RuntimeError('Adaptive cover cannot close remaining cells')
404         covered |= best[3]
405         selected.append(best)
406         candidates.remove(best)
407     if self.rare_cover:
408         for a in list(reversed(selected)):
409             union=actual
410             for other in selected:
411                 if other is not a:union|=other[3]
412                 if union==full:selected.remove(a)
413     for kind,c,q,mask,d in selected:
414         if kind=='target':self.cover_targets.add(c)
415         else:needed.append((kind,c,q))
416     return nodes+needed
417
418 def run(self):
419     self.at_site_scan((0,0.), force=True)
420     if self.initial_baseline:
421         pending = list(self.tracks.values())
422         angle = (math.atan2(pending[0].center[1], pending[0].center[0]) if pending else 0.)+math.radians(self.initial_rotation)
423         q = np.array([math.cos(angle),math.sin(angle)]) * self.initial_baseline
424         self.at_site_scan(q, force=self.initial_force)
425     if self.recon_radius:

```

```

424         for i in range(self.recon_count):
425             a=2*math.pi*i/self.recon_count
426             q=self.recon_radius*np.array([math.cos(a),math.sin(a)])
427             self.at_site_scan(q,force=True)
428     # Keep all anchors available; real coverage masks prevent redundant work.
429     while True:
430         self.iterations += 1
431         self.robot.check_budget()
432         if self.iterations > 180:
433             raise RuntimeError('Joint planner iteration cap')
434         pending = [t for c,t in self.tracks.items() if c not in self.robot.cleared]
435         if not pending and (len(self.robot.cleared) == 16 or not self.unknown()):
436             self.stop_reason = 'cleared_upper_bound_16' if len(self.robot.cleared)==16 else 'continuous_coverage_certificate'
437             self.robot.record(dict(kind='completion_certificate', **self.certificate()))
438             return self.stop_reason
439         if self.family=='adaptive':
440             gain=self.scan_gain;self.scan_gain=2.
441             self.at_site_scan(self.robot.position)
442             self.scan_gain=gain
443         else:self.at_site_scan(self.robot.position)
444         nodes = self.candidates()
445         if not nodes:
446             continue
447         order = open_route(self.robot.position, [n[2] for n in nodes],restarts=self.route_restarts) if self.route_on else sorted(range(len(nodes)),key=lambda i:
↳ math.dist(nodes[i][2],self.robot.position))
448         if self.explore_bonus and nodes[order[0]][0]=='target':
449             def tour_length(start,indices):
450                 total=0;p=np.asarray(start)
451                 for i in indices:total+=np.linalg.norm(nodes[i][2]-p);p=nodes[i][2]
452                 return float(total)
453             normal=tour_length(self.robot.position,order)
454             best=(float('inf'),None)
455             for i,n in enumerate(nodes):
456                 if n[0]!='scan':continue
457                 others=[j for j in range(len(nodes)) if j!=i]
458                 rest=open_route(n[2],[nodes[j][2] for j in others])
459                 cost=math.dist(self.robot.position,n[2])+tour_length(n[2],[others[j] for j in rest])
460                 if cost<best[0]:best=(cost,i)
461             allowance=self.explore_bonus*max(1.,self.expected_count-len(self.robot.discovered))
462             if best[1] is not None and best[0]-normal<=allowance:
463                 order=[best[1]]+[i for i in order if i!=best[1]]
464             kind, c, point = nodes[order[0]]
465             if kind == 'scan':
466                 self.at_site_scan(point, force=True)
467             else:
468                 track = self.tracks[c]
469                 if self.shared_waypoint and self.family=='adaptive' and c in self.cover_targets and track.radius>CLEAR_RADIUS:
470                     self.at_site_scan(point,force=True)
471                     if math.dist(self.attempts.get(c,(1e9,1e9)),point)>1e-5:
472                         self.observe(point,c)
473                     continue
474                 self.solve_track(track)
475                 if self.family=='adaptive' and c in self.cover_targets and len(self.robot.cleared)<16:
476                     self.at_site_scan(self.robot.position,force=True)
477
478
479 class SweepPlanner(JointPlanner):
480     """Finish targets near the next coverage leg; defer distant returns."""
481     def __init__(self, robot, **kwargs):
482         super().__init__(robot, family='anchors', **kwargs)
483
484     def candidates(self):
485         nodes = super().candidates()
486         scans = [n for n in nodes if n[0]=='scan']
487         if not scans:
488             return nodes
489         route = open_route(self.robot.position,[n[2] for n in scans])
490         next_scan = scans[route[0]]
491         p,q = np.asarray(self.robot.position),next_scan[2]
492         targets=[]
493         for n in nodes:

```

```

494         if n[0]!='target':continue
495         detour = np.linalg.norm(n[2]-p)+np.linalg.norm(n[2]-q)-np.linalg.norm(q-p)
496         if detour <= 350:
497             targets.append(n)
498     return targets + [next_scan]
499
500
501 class SectorPlanner(JointPlanner):
502     """One angular sweep, local open routes with the next station as endpoint."""
503     def __init__(self,robot,* ,sector_rotation=0.,sector_width=60.,preclear=False,**kwargs):
504         super().__init__(robot,**kwargs)
505         self.sector_rotation=math.radians(sector_rotation)
506         self.sector_width=sector_width
507         self.preclear=preclear
508
509     def run(self):
510         self.at_site_scan((0.,0.),force=True)
511         radius=math.hypot(*self.anchors[1])
512         sites=[radius*np.array([math.cos(self.sector_rotation+i*math.pi/3),
513                                 math.sin(self.sector_rotation+i*math.pi/3)]) for i in range(6)]
514         for sector,site in enumerate(sites):
515             if not self.unknown():break
516             if self.preclear:
517                 for _ in range(16):
518                     p=np.asarray(self.robot.position)
519                     good=[t for c,t in self.tracks.items() if c not in self.robot.cleared and
520                          np.linalg.norm(t.center-p)+np.linalg.norm(t.center-site)-np.linalg.norm(site-p)<=100]
521                     if not good:break
522                     self.solve_track(min(good,key=lambda t:np.linalg.norm(t.center-p)))
523             self.at_site_scan(site,force=True)
524             while True:
525                 self.robot.check_budget()
526                 pending=[]
527                 for c,t in self.tracks.items():
528                     if c in self.robot.cleared:continue
529                     angle=(math.degrees(math.atan2(t.center[1],t.center[0])-self.sector_rotation)+30)%360
530                     if angle <= (sector+1)*60 or sector==5:
531                         pending.append(t)
532                     if not pending:break
533                 end=sites[sector+1] if sector<5 and self.unknown() else None
534                 route=open_route(self.robot.position,[t.center for t in pending],end=end)
535                 self.solve_track(pending[route[0]])
536                 # Only measurements of uncertain known targets, no blind scans.
537                 gain=self.scan_gain;self.scan_gain=2.
538                 self.at_site_scan(self.robot.position)
539                 self.scan_gain=gain
540             while True:
541                 pending=[t for c,t in self.tracks.items() if c not in self.robot.cleared]
542                 if not pending:break
543                 route=open_route(self.robot.position,[t.center for t in pending])
544                 self.solve_track(pending[route[0]])
545             if self.unknown():
546                 raise RuntimeError('Sector sweep did not certify full coverage')
547             self.stop_reason='cleared_upper_bound_16' if len(self.robot.cleared)==16 else 'continuous_coverage_certificate'
548             self.robot.record(dict(kind='completion_certificate',**self.certificate()))
549             return self.stop_reason
550
551
552 class OrderedPlanner(JointPlanner):
553     """Insert target actions into a monotone station tour; no per-sector lock."""
554     def __init__(self,robot,* ,clockwise=False,**kwargs):
555         super().__init__(robot,**kwargs)
556         self.remaining=list(self.anchors[1:])
557         if clockwise:self.remaining=list(reversed(self.remaining))
558
559     def candidates(self):
560         targets=[]
561         for c,t in self.tracks.items():
562             if c in self.robot.cleared:continue
563             q=t.safe_clear_point(self.robot.position)
564             q=t.center if q is None else q

```

```

565         targets.append(('target',c,np.asarray(q)))
566     unknown=self.unknown()
567     remaining=self.remaining.copy() if unknown else []
568     if unknown:
569         actual=self.coverage.covered[unknown[0]]
570         for p in list(reversed(remaining)):
571             total=actual.copy()
572             for q in remaining:
573                 if q is not p:total|=self.coverage.mask(q)
574             if np.all(total).remaining.remove(p)
575     route=[('scan',i,np.asarray(p)) for i,p in enumerate(remaining)]
576     start=('start',-1,np.asarray(self.robot.position))
577     def distance(r):
578         p=start[2];value=0.
579         for n in r:value+=np.linalg.norm(n[2]-p);p=n[2]
580         return float(value)
581     while targets:
582         best=(float('inf'),None,None)
583         base=distance(route)
584         for i,t in enumerate(targets):
585             for j in range(len(route)+1):
586                 cost=distance(route[:j]+[t]+route[j:])-base
587                 if cost<best[0]:best=(cost,i,j)
588         _,i,j=best;route.insert(j,targets.pop(i))
589     for _ in range(8):
590         best=distance(route);replacement=None
591         for i,n in enumerate(route):
592             if n[0]!='target':continue
593             reduced=route[:i]+route[i+1:]
594             for j in range(len(route)):
595                 trial=reduced[:j]+[n]+reduced[j:];cost=distance(trial)
596                 if cost<best-1e-6:best,replacement=cost,trial
597         if replacement is None:break
598         route=replacement
599     # Returning just the first action prevents the base free-order solver
600     # from undoing the station precedence constraint.
601     return route[:1]
602
603     def at_site_scan(self,point,force=False):
604         super().at_site_scan(point,force=force)
605         if force:
606             self.remaining=[p for p in self.remaining if math.dist(p,point)>1e-6]
607
608
609     class StickyPlanner(JointPlanner):
610         """Commit to a shared coverage plan instead of repeatedly buying overlap."""
611         def __init__(self,robot,**kwargs):
612             super().__init__(robot,family='adaptive',**kwargs)
613             self.committed_points=None
614             self.committed_targets=set()
615
616         def candidates(self):
617             nodes=[]
618             for c,t in self.tracks.items():
619                 if c in self.robot.cleared:continue
620                 q=t.safe_clear_point(self.robot.position)
621                 q=t.center if q is None else q
622                 nodes.append(('target',c,np.asarray(q)))
623             unknown=self.unknown()
624             if not unknown:return nodes
625             if self.committed_points is None or (not self.committed_points and not self.committed_targets):
626                 plan=self.adaptive_cover(nodes,unknown)
627                 self.committed_points=[n for n in plan if n[0]=='scan']
628                 self.committed_targets=self.cover_targets.copy()
629             self.cover_targets=self.committed_targets.copy()
630             return nodes+self.committed_points
631
632     def at_site_scan(self,point,force=False):
633         super().at_site_scan(point,force=force)
634         if force and self.committed_points is not None:
635             self.committed_points=[n for n in self.committed_points if math.dist(n[2],point)>1e-5]

```

```

636         self.committed_targets==self.robot.cleared
637
638
639 class TargetFirstPlanner(JointPlanner):
640     """Clear known targets as a routed group, ending toward the next scan."""
641     def candidates(self):
642         nodes=super().candidates()
643         targets=[n for n in nodes if n[0]=='target']
644         scans=[n for n in nodes if n[0]=='scan']
645         if not targets:return nodes
646         if not scans:
647             order=open_route(self.robot.position,[n[2] for n in targets],restarts=self.route_restarts)
648             return [targets[order[0]]]
649         best=(float('inf'),None)
650         for end in scans:
651             order=open_route(self.robot.position,[n[2] for n in targets],end=end[2],restarts=self.route_restarts)
652             p=np.asarray(self.robot.position);cost=0.
653             for i in order:cost+=np.linalg.norm(targets[i][2]-p);p=targets[i][2]
654             cost+=np.linalg.norm(end[2]-p)
655             remainder=[n for n in scans if n is not end]
656             tour=open_route(end[2],[n[2] for n in remainder]);p=end[2]
657             for j in tour:cost+=np.linalg.norm(remainder[j][2]-p);p=remainder[j][2]
658             if cost<best[0]:best=(cost,targets[order[0]])
659         return [best[1]]
660
661
662 class PathCoverPlanner(JointPlanner):
663     """Choose sensing stops on route segments; update insertion costs jointly."""
664     def __init__(self,robot,* ,path_spacing=200.,**kwargs):
665         super().__init__(robot,family='adaptive',**kwargs)
666         self.path_spacing=path_spacing
667         self.path_static=None
668
669     def adaptive_cover(self,nodes,unknown):
670         def bits(mask):return int.from_bytes(np.packbits(mask,bitorder='little').tobytes(),'little')
671         if self.path_static is None:
672             pts=[np.asarray(p) for p in self.anchors]
673             pts += [np.array([x,y]) for x in np.arange(-1500,1501,self.grid_step) for y in np.arange(-1500,1501,self.grid_step) if math.hypot(x,y)<1750]
674             self.path_static=[(q,bits(self.coverage.mask(q))) for q in pts]
675         covered=bits(self.coverage.covered[unknown[0]])
676         full=(1<<len(self.coverage.centers))-1
677         selected=[];self.cover_targets=set()
678         p=np.asarray(self.robot.position)
679         while covered!=full:
680             future=nodes+[(('scan',-100-i,q) for i,(q,mask) in enumerate(selected))]
681             order=open_route(p,[n[2] for n in future])
682             route=[p]+[future[i][2] for i in order]
683             candidates=self.path_static.copy()
684             for a,b in zip(route,route[1:]):
685                 count=max(1,int(math.ceil(np.linalg.norm(b-a)/self.path_spacing)))
686                 for t in np.linspace(0,1.,count+1):
687                     q=a+t*(b-a);candidates.append((q,bits(self.coverage.mask(q))))
688             if len(route)==1:candidates.append((p,bits(self.coverage.mask(p))))
689             def detour(q):
690                 best=float(np.linalg.norm(q-route[-1]))
691                 for a,b in zip(route,route[1:]):
692                     best=min(best,float(np.linalg.norm(a-q)+np.linalg.norm(b-q)-np.linalg.norm(a-b)))
693             return max(0.,best)
694             missing=full^covered
695             best=max(candidates,key=lambda a:(a[1]&missing).bit_count()/(6*len(unknown)+self.cover_move_weight*detour(a[0])/5))
696             if not best[1]&missing:raise RuntimeError('Path cover cannot progress')
697             covered|=best[1];selected.append(best)
698         # Reverse-delete coverage redundancy before routing the chosen stops.
699         for a in list(reversed(selected)):
700             total=bits(self.coverage.covered[unknown[0]])
701             for other in selected:
702                 if other is not a:total|=other[1]
703             if total==full:selected=[x for x in selected if x is not a]
704         return nodes+[(('scan',-100-i,q) for i,(q,mask) in enumerate(selected))]
705
706     def candidates(self):

```

```

707     nodes=super().candidates()
708     if not nodes: return nodes
709     order=open_route(self.robot.position,[n[2] for n in nodes])
710     first=nodes[order[0]]
711     if first[0]=='target':
712         matching=[n for n in nodes if n[0]=='scan' and math.dist(n[2],first[2])<1e-5]
713         if matching: return [matching[0]]
714     return nodes

```

源文件组 17

q3/baselines/v4/merged_cover.py

q3/v6/merged_cover.py

```

1  """Continuous waypoint relocation with an explicit full-coverage constraint."""
2  import math
3  import numpy as np
4  from scipy.optimize import minimize
5  from scipy.spatial import ConvexHull
6  from joint_strategies import JointPlanner, open_route
7  from bearing_geometry import enclosing_circle, diameter
8
9
10 class MergedCoverPlanner(JointPlanner):
11     def __init__(self, robot, *, cover_sweeps=2, merge_stations=True, merge_target_scans=False, **kwargs):
12         super().__init__(robot, family='adaptive', **kwargs)
13         self.cover_sweeps=cover_sweeps
14         self.merge_stations=merge_stations
15         self.merge_target_scans=merge_target_scans
16
17     def candidates(self):
18         targets=[]
19         for c,t in self.tracks.items():
20             if c in self.robot.cleared: continue
21             q=t.safe_clear_point(self.robot.position)
22             q=t.center if q is None else q
23             targets.append(('target',c,np.asarray(q)))
24         unknown=self.unknown()
25         if not unknown: return targets
26         actual=self.coverage.covered[unknown[0]]
27         # Targets are mandatory route vertices but optional sensing sites.
28         points=[n[2].copy() for n in targets]+[np.array(p) for p in self.anchors]
29         n_target=len(targets)
30         active=np.ones(len(points),dtype=bool)
31         present=np.ones(len(points),dtype=bool)
32         masks=[self.coverage.mask(p) for p in points]
33         start=np.asarray(self.robot.position)
34         def route_order():
35             indices=[i for i in range(len(points)) if present[i]]
36             return [indices[j] for j in open_route(start,[points[i] for i in indices])]
37         def total_without(i):
38             total=actual.copy()
39             for j in range(len(points)):
40                 if j!=i and active[j]: total+=masks[j]
41             return total
42         def prune():
43             while True:
44                 order=route_order(); gains=[]
45                 for k,i in enumerate(order):
46                     if not active[i]: continue
47                     gain=30*len(unknown)
48                     if i>=n_target:
49                         a=start if k==0 else points[order[k-1]]
50                         gain+=np.linalg.norm(points[i]-a)
51                     if k+1<len(order):
52                         b=points[order[k+1]]
53                         gain+=np.linalg.norm(points[i]-b)-np.linalg.norm(a-b)
54                     gains.append((gain,i))
55                 removed=False

```

```

56         for gain,i in sorted(gains,reverse=True):
57             if np.all(total_ without(i)):
58                 active[i]=False
59                 if i>=n_target:present[i]=False
60                 removed=True;break
61         if not removed:break
62     prune()
63     if self.merge_stations:
64         def full_cost(candidate_points,candidate_present,candidate_active):
65             indices=np.flatnonzero(candidate_present)
66             order=open_route(start,[candidate_points[i] for i in indices])
67             p=start,value=30*len(unknown)*int(np.count_nonzero(candidate_active))
68             for j in order:
69                 q=candidate_points[indices[j]];value+=np.linalg.norm(q-p);p=q
70             return float(value)
71     for merge_iteration in range(4):
72         old_cost=full_cost(points,present,active);best=None
73         selected=[i for i in range(0 if self.merge_target_scans else n_target,len(points)) if active[i]]
74         for at,i in enumerate(selected):
75             for j in selected[at]:
76                 remaining=actual.copy()
77                 for k in range(len(points)):
78                     if active[k] and k not in [i,j]:remaining|=masks[k]
79                 needed=self.coverage.centers[~remaining]
80                 if not len(needed):continue
81                 h=self.coverage.half
82                 corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in [(-1,-1),(-1,1),(1,-1),(1,1)])])
83                 hull=corners[ConvexHull(corners).vertices]
84                 if diameter(hull)[0]>1999.9799:continue
85                 center,radius,_=enclosing_circle(hull)
86                 if radius>999.98999:continue
87                 new_points=points+[center]
88                 new_present=np.append(present,True);new_active=np.append(active,True)
89                 for removed in [i,j]:
90                     if removed>=n_target:new_present[removed]=False
91                     new_active[removed]=False
92                 value=full_cost(new_points,new_present,new_active)
93                 if value<old_cost-1e-6 and (best is None or value<best[0]):
94                     best=(value,i,j,center,new_points,new_present,new_active)
95             if best is None:break
96         value,i,j,center,points,present,active=best
97         masks.append(self.coverage.mask(center))
98         self.robot.record(dict(kind='merge_scans',point=center.tolist(),removed=[int(i),int(j)],
99             predicted_saving_s=(old_cost-value)/5,proof='MEC of all responsibility-cell corners <= 999.99 m'))
100     prune()
101     for _ in range(self.cover_sweeps):
102         order=route_order()
103         improvement=0.
104         for k,i in enumerate(order):
105             if i<n_target or not active[i]:continue
106             needed=self.coverage.centers[~total_ without(i)]
107             if not len(needed):continue
108             h=self.coverage.half
109             corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in [(-1,-1),(-1,1),(1,-1),(1,1)])])
110             witnesses=corners[ConvexHull(corners).vertices]/1000
111             before=points[i]/1000
112             previous=(start if k==0 else points[order[k-1]])/1000
113             following=points[order[k+1]]/1000 if k+1<len(order) else None
114             def objective(q):
115                 value=np.sqrt(np.dot(q-previous,q-previous)+1e-16)
116                 if following is not None:value+=np.sqrt(np.dot(q-following,q-following)+1e-16)
117                 return float(value)
118             def gradient(q):
119                 value=(q-previous)/max(np.linalg.norm(q-previous),1e-8)
120                 if following is not None:value+=(q-following)/max(np.linalg.norm(q-following),1e-8)
121                 return value
122             cons={'type':'ineq','fun':lambda q: .99998999**2-np.sum((q-witnesses)**2,axis=1),
123                 'jac':lambda q:-2*(q-witnesses)}
124             sol=minimize(objective,before,jac=gradient,constraints=cons,method='SLSQP',
125                 options={'ftol':1e-10,'maxiter':60})
126             if np.max(np.linalg.norm(witnesses-sol.x,axis=1))<=.99999 and objective(sol.x)<=objective(before)+1e-8:

```

```

127         improvement+=objective(before)-objective(sol.x)
128         points[i]=sol.x*1000;masks[i]=self.coverage.mask(points[i])
129     prune()
130     if improvement<1e-5:break
131     union=actual.copy()
132     for i in range(len(points)):
133         if active[i]:union|=masks[i]
134     if not np.all(union):raise ArithmeticError('Relocated cover lost cells')
135     scans=[('scan',-100-i,points[i]) for i in range(len(points)) if active[i]]
136     nodes=targets+scans
137     self.cover_targets=set()
138     order=open_route(start,[n[2] for n in nodes])
139     if nodes[order[0]][0]=='target':
140         first=nodes[order[0]]
141         matches=[n for n in scans if math.dist(n[2],first[2])<1e-5]
142         if matches:return [matches[0]]
143     return nodes

```

源文件组 18

q3/baselines/v4/reception_region.py

q3/v6/reception_region.py

```

1  """Exact first-positive reception separator on a convex polygon outer region.
2
3  For each x, the least admissible reception radius is max(R0, |x-s|).
4  Evaluate the maximum violation over polygon vertices, edge/circle crossings,
5  and the antipodal stationary point on the R0-circle when inside the polygon.
6  """
7  import math
8  import numpy as np
9
10
11 def reception_violation(poly,first_sensor,query,r0=1000.):
12     poly=np.asarray(poly,float);s=np.asarray(first_sensor,float);q=np.asarray(query,float)
13     if not len(poly):raise ValueError('Empty source region')
14     points=list(poly)
15     edges=np.roll(poly,-1,axis=0)-poly
16     for p,d in zip(poly,edges):
17         a=float(np.dot(d,d))
18         if a<1e-18:continue
19         offset=p-s;b=2*float(np.dot(offset,d));c=float(np.dot(offset,offset)-r0*r0)
20         discr=b*b-4*a*c
21         if discr>=0:
22             root=math.sqrt(discr)
23             for t in [(-b-root)/(2*a),(-b+root)/(2*a)]:
24                 if 0<=t<=1:points.append(p+t*d)
25     d=q-s;length=float(np.linalg.norm(d))
26     if length>1e-12:
27         far=s-r0*d/length
28         delta=far-poly
29         cross=edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]
30         if np.all(cross>=-1e-8*np.maximum(1,np.linalg.norm(edges,axis=1))):points.append(far)
31     points=np.asarray(points)
32     values=np.sum((points-q)**2,axis=1)-np.maximum(r0*r0,np.sum((points-s)**2,axis=1))
33     k=int(np.argmax(values))
34     return float(values[k]),points[k].copy()

```

源文件组 19

q3/baselines/v4/run_robot.py

```

1  """ 问题三多策略研究版；尚未达到平均 200 秒/点目标。 """
2  import argparse
3  import datetime as dt
4  import hashlib

```

```

5 import json
6 from pathlib import Path
7 import shutil
8 import sys
9 import time
10 import uuid
11
12 ROOT = Path(__file__).resolve().parent
13
14
15 def main():
16     if hasattr(sys.stdout, 'reconfigure'):
17         sys.stdout.reconfigure(encoding='utf-8')
18     parser = argparse.ArgumentParser(description=__doc__)
19     parser.add_argument('--policy', choices=['baseline_v3', 'cost_max', 'merged_cover', 'combined_v4'], default='combined_v4', help='选择冻结策略，
        ↳ 默认v4成本与覆盖组合策略')
20     parser.add_argument('--team', help='默认复用旧策略的 config.json 队号')
21     parser.add_argument('--port', type=int, help='默认复用旧配置，缺省端口 2026')
22     parser.add_argument('--config', type=Path, help='指定含 team_no、port 的配置文件')
23     parser.add_argument('--check', action='store_true', help='仅检查依赖和配置，不连接模拟器')
24     args = parser.parse_args()
25     try:
26         import numpy
27         import scipy
28         from api_client import Robot, BudgetReached
29         from strategy import Planner, VERSION, POLICIES
30     except ImportError as error:
31         print(f'缺少依赖: {error}\n 请安装 requirements.txt 中的依赖后重试。')
32         return 2
33     paths = [args.config] if args.config else [ROOT / 'config.json', ROOT.parent / 'simple-strategy' / 'config.json']
34     config_path = next((p for p in paths if p and p.is_file()), None)
35     config = json.loads(config_path.read_text(encoding='utf-8-sig')) if config_path else {}
36     team, port = args.team or config.get('team_no'), args.port or config.get('port', 2026)
37     if not team:
38         print('找不到队号配置，请使用 --config 指定旧 config.json，或使用 --team 参数。')
39         return 2
40     if args.check:
41         print(f'策略 {args.policy}; 研究版，200 秒/点目标尚未达成。')
42         print(f'自检通过: Python {sys.version.split()[0]} / numpy {numpy.__version__} / scipy {scipy.__version__}')
43         print(f'配置可读; 端口 {port}; 未连接模拟器。')
44         return 0
45     print(f'问题 3 研究策略 {VERSION} / {args.policy} | 端口 {port}')
46     print('这是可运行的研究候选，尚未达到平均 200 秒/点目标。')
47     print('请在模拟器选择 【问题 3 演练测试】，开始后等待 “接口已就绪/等待机器狗进入”。')
48     print('接口无法查询演练/正式模式，程序按问题 3 全向干扰器规则工作。')
49     input('界面就绪后按 Enter 开始: ')
50     folder = ROOT / 'runs' / (dt.datetime.now().strftime('%Y%m%d-%H%M%S') + '-' + uuid.uuid4().hex[:6])
51     robot = Robot(team, f'http://127.0.0.1:{port}', folder)
52     planner = Planner(robot, policy=args.policy)
53     snapshot = folder / 'source_snapshot'
54     snapshot.mkdir()
55     hashes = {}
56     for p in list(ROOT.glob('*.py')) + [ROOT / 'policies.json']:
57         shutil.copy2(p, snapshot / p.name)
58         hashes[p.name] = hashlib.sha256(p.read_bytes()).hexdigest()
59     (folder / 'manifest.json').write_text(json.dumps(dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy],
60         python=sys.version, numpy=numpy.__version__, scipy=scipy.__version__, source_sha256=hashes,
61         config_path=str(config_path), mode='user_selected_P3_practice_not_API_verified'),
62         ensure_ascii=False, indent=2), encoding='utf-8')
63     outcome, error_text, entered = 'completed', None, False
64     robot.started = time.monotonic()
65     try:
66         robot.call('/enter')
67         entered = True
68         planner.run()
69     except BudgetReached as error:
70         outcome, error_text = 'budget_stop', str(error)
71     except (Exception, KeyboardInterrupt) as error:
72         outcome, error_text = 'error', str(error) or '用户中断'
73         # Interruption during HTTP could leave execution state unknown.
74         if isinstance(error, KeyboardInterrupt):

```

```

75     robot.transport_uncertain = True
76 finally:
77     if entered and not robot.transport_uncertain:
78         try:
79             robot.call('/exit')
80         except Exception as error:
81             outcome = 'exit_unconfirmed'
82             error_text = (error_text or '') + '\n 退出未确认: ' + str(error)
83     count = len(robot.cleared)
84     summary = dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy], outcome=outcome, error=error_text,
85                   exit_confirmed=robot.exit_confirmed, transport_uncertain=robot.transport_uncertain,
86                   cleared_count=count, cleared_channels=sorted(robot.cleared),
87                   observed_but_uncleared=sorted(robot.discovered - robot.cleared),
88                   measure_count=robot.measure_count, clear_failures=robot.clear_failures,
89                   scanned_sites=robot.scanned_sites, distance_m=robot.distance_m, switches=robot.switches,
90                   virtual_time_s=robot.virtual, average_clear_time_s=robot.virtual / count if count else None,
91                   local_elapsed_s=time.monotonic() - robot.started,
92                   actual_jammer_count=None, completion=planner.certificate(),
93                   note='实际目标总数和正式成绩以模拟器为准; 完整性证书以问题 3 全向规则为前提。')
94     (folder / 'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
95     robot.log.close()
96     print(f'\n 结束: 清除 {count} 个; 虚拟时间 {robot.virtual:.1f} 秒; 移动 {robot.distance_m:.1f} 米。')
97     print(f'状态 {outcome}; 退出确认 {robot.exit_confirmed}; 结束依据 {planner.stop_reason}')
98     if error_text:
99         print(error_text)
100    print(f'运行记录: {folder}')
101    print('请保存模拟器结束界面、案例编码、实际目标数, 并导出官方 .jlog。')
102    return 0 if outcome == 'completed' and robot.exit_confirmed else 1
103
104
105 if __name__ == '__main__':
106     sys.exit(main())

```

源文件组 20

q3/baselines/v4/strategy.py

```

1  """Frozen v4 factory. Research candidate; not an official 200 s/source result."""
2  import json
3  from pathlib import Path
4  from convex_cover import ConvexCoverPlanner
5  from cost_aware import CostAwarePlanner
6  from merged_cover import MergedCoverPlanner
7  from combined_cost_cover import CombinedCostCoverPlanner
8
9  VERSION = 'p3-cost-aware-v4-research'
10 POLICIES = json.loads(Path(__file__).with_name('policies.json').read_text(encoding='utf-8'))
11
12
13 def Planner(robot, *, policy='combined_v4'):
14     if policy not in POLICIES:
15         raise ValueError(f'Unknown policy: {policy}')
16     config = POLICIES[policy].copy()
17     kind = config.pop('_class')
18     factory = {'convex_cover': ConvexCoverPlanner, 'cost': CostAwarePlanner,
19              'merge': MergedCoverPlanner, 'combined': CombinedCostCoverPlanner}[kind]
20     return factory(robot, **config)

```

源文件组 21

q3/v6/action_value.py

```

1  """Observation-only action values, with persistent nonconvex optical exclusions.
2
3  Scenario averages are planning heuristics, never completion certificates.
4  Actual safety continues to use an outer region and paid API actions.
5  """

```

```

6 import copy
7 import math
8 import numpy as np
9 from shapely.geometry import Polygon, Point
10 from shapely.geometry.polygon import orient
11 from bearing_geometry import enclosing_circle, diameter
12 from legacy_strategy import Track, CLEAR_RADIUS
13 from service_routing import ServicePlanner
14 from cost_aware import reception_safe
15
16
17 class OpticalTrack(Track):
18     def sync_region(self):
19         polygon=Polygon(self.poly)
20         self.feasible=polygon if not hasattr(self,'feasible') else self.feasible.intersection(polygon)
21         self.refresh()
22
23     def refresh(self):
24         if self.feasible.is_empty or self.feasible.area<1e-14:
25             raise ArithmeticError('Optical feasible region empty or numerically degenerate')
26         self.poly=np.asarray(orient(self.feasible.convex_hull,sign=1.).exterior.coords[:-1])
27         self.center,self.radius,_=enclosing_circle(self.poly)
28
29     def update(self,point,observation):
30         super().update(point,observation)
31         self.sync_region()
32
33     def exclude_clear(self,point):
34         # Inscribed polygon is strictly inside the physical failed-clear disk.
35         # Its removal keeps a conservative OUTER set, including boundary margin.
36         self.feasible=self.feasible.difference(Point(point).buffer(19.999,quad_segs=16))
37         self.refresh()
38
39
40     def clone_track(track):
41         t=copy.copy(track);t.poly=track.poly.copy();t.history=list(track.history)
42         return t # Shapely geometries are immutable.
43
44
45     def scenarios(track,count=3):
46         region=getattr(track,'feasible',Polygon(track.poly))
47         _,(i,j)=diameter(track.poly);a,b=track.poly[i],track.poly[j]
48         points=[]
49         for f in np.linspace(.15,.85,count):
50             q=(1-f)*a+f*b
51             q=.95*q+.05*np.asarray(region.representative_point().coords[0])
52             if region.covers(Point(q)):points.append(q)
53         if len(points)<count:
54             pieces=list(region.geoms) if hasattr(region,'geoms') else [region]
55             for piece in sorted(pieces,key=lambda g:g.area,reverse=True):
56                 points.append(np.asarray(piece.representative_point().coords[0]))
57                 if len(points)>=count:break
58         return points
59
60
61     def synthetic_observation(q,truth,error):
62         if np.linalg.norm(truth-q)<=5:return dict(measure_result='near')
63         angle=math.degrees(math.atan2(truth[1]-q[1],truth[0]-q[0]))
64         return dict(measure_result='direction',svd_deg=round((angle+error)%360,2)%360)
65
66
67     def finish_cost(track,start,truth,error,first=None,exit_point=None):
68         """Bounded rollout of the unchanged geometric completion module in seconds."""
69         t=clone_track(track);p=np.asarray(start);cost=0.
70         for k in range(5):
71             q=t.safe_clear_point(p)
72             if q is not None:
73                 cost+=np.linalg.norm(q-p)/5+5
74                 if exit_point is not None:cost+=np.linalg.norm(q-exit_point)/5
75             return float(cost)
76         q=first if k==0 and first is not None else t.next_point(p)

```

```

77     cost+=np.linalg.norm(q-p)/5+6 # Conservative switch allowance in planning.
78     t.update(q,synthetic_observation(q,truth,error));p=q
79     return float(cost+1000)
80
81
82 class ActionValuePlanner(ServicePlanner):
83     def __init__(self,robot,*,optical=True,optical_radius=65.,optical_margin=1.,
84                 optical_update=True,optical_risk=.2,optical_max_trials=2,
85                 value_probes=False,value_margin=3.,**kwargs):
86         super().__init__(robot,**kwargs)
87         self.optical=optical;self.optical_radius=optical_radius
88         self.optical_margin=optical_margin;self.optical_update=optical_update
89         self.optical_risk=optical_risk;self.optical_max_trials=optical_max_trials
90         self.value_probes=value_probes;self.value_margin=value_margin
91         self.optical_attempts={}
92
93     def observe(self,point,channel):
94         result=super().observe(point,channel)
95         if channel in self.tracks:
96             t=self.tracks[channel]
97             if not isinstance(t,OpticalTrack):t.__class__=OpticalTrack
98             t.sync_region()
99             self.robot.record(dict(kind='optical_region',channel=channel,
100                                wkt=t.feasible.wkt,vertices=t.poly.tolist(),radius_m=t.radius))
101         return result
102
103     def aggregate(self,values):
104         return float((1-self.optical_risk)*np.mean(values)+self.optical_risk*np.max(values))
105
106     def optical_action(self,t):
107         if not self.optical or not CLEAR_RADIUS<t.radius<=self.optical_radius:return None
108         if len(self.optical_attempts.get(t.channel,[]))>=self.optical_max_trials:return None
109         start=np.asarray(self.robot.position);samples=scenarios(t)
110         states=[(x,e) for x in samples for e in [-1.,0.,1.]]
111         baseline=self.aggregate([finish_cost(t,start,x,e) for x,e in states])
112         candidates=[start,t.center]
113         d=start-t.center;length=np.linalg.norm(d)
114         if length>1e-8:candidates.append(t.center+d/length*min(15.,t.radius*.35))
115         _,(i,j)=diameter(t.poly)
116         if t.radius>35:
117             candidates.extend([.65*t.center+.35*t.poly[i],.65*t.center+.35*t.poly[j]])
118         best=None
119         for q in candidates:
120             if any(math.dist(q,p)<2. for p in self.optical_attempts.get(t.channel,[])):continue
121             covered=t.feasible.intersection(Point(q).buffer(19.999,quad_segs=16)).area
122             if covered<.12*t.feasible.area:continue
123             failed=clone_track(t)
124             try:
125                 if self.optical_update:failed.exclude_clear(q)
126             except ArithmeticError:continue
127             values=[]
128             for x,e in states:
129                 value=np.linalg.norm(q-start)/5
130                 if np.linalg.norm(x-q)<=20:value+=5
131                 else:value+=3+finish_cost(failed,q,x,e)
132                 values.append(value)
133             score=self.aggregate(values)
134             if score<baseline-self.optical_margin and (best is None or score<best[0]):
135                 best=(score,q,baseline,covered/t.feasible.area)
136         return best
137
138     def solve_track(self,track):
139         for _ in range(20):
140             self.robot.check_budget();track=self.tracks[track.channel]
141             if track.radius<=CLEAR_RADIUS:return super().solve_track(track)
142             proposal=self.optical_action(track)
143             if proposal is not None:
144                 score,q,baseline,fraction=proposal
145                 self.robot.record(dict(kind='optical_decision',channel=track.channel,point=q.tolist(),
146                                    predicted_s=score,radio_predicted_s=baseline,area_fraction=fraction))
147                 self.optical_attempts.setdefault(track.channel,[]).append(q.tolist())

```

```

148         if self.robot.clear(tuple(q),track.channel):return
149         if self.optical_update:track.exclude_clear(q)
150         self.robot.record(dict(kind='optical_failure_update',channel=track.channel,
151             point=q.tolist(),wkt=track.feasible.wkt,vertices=track.poly.tolist(),radius_m=track.radius))
152         continue
153         q=track.next_point(np.asarray(self.robot.position))
154         if self.observe(q,track.channel)['measure_result']=='no_signal':
155             raise ArithmeticError('Certified radio point returned no signal')
156         raise RuntimeError('Action-value localization budget exhausted')
157
158     def probe_on_leg(self,node):
159         if not self.value_probes:return super().probe_on_leg(node)
160         start=np.asarray(self.robot.position);end=np.asarray(node[2])
161         if node[0]=='target':
162             tr=self.tracks[node[1]];safe=tr.safe_clear_point(start)
163             end=safe if safe is not None else tr.next_point(start)
164         if np.linalg.norm(end-start)<250:return None
165         tracks=[t for c,t in self.tracks.items() if c not in self.robot.cleared and t.radius>40]
166         tracks=sorted(tracks,key=lambda t:np.linalg.norm(t.center-end))[:3]
167         baselines={}
168         for t in tracks:
169             states=[(x,e) for x in scenarios(t) for e in [-1.,0.,1.]]
170             baselines[t.channel]=(states,self.aggregate([finish_cost(t,end,x,e) for x,e in states]))
171         best=None
172         for fraction in [.25,.5,.75]:
173             q=start+fraction*(end-start);channels=[];net=0.
174             for t in tracks:
175                 if not reception_safe(t,q):continue
176                 if min(math.dist(q,h['position']) for h in t.history)<150:continue
177                 if math.dist(q,self.attempts.get(t.channel,(1e9,1e9)))<150:continue
178                 states,before=baselines[t.channel];values=[]
179                 for x,e in states:
180                     after=clone_track(t);after.update(q,synthetic_observation(q,x,e))
181                     values.append(finish_cost(after,end,x,e))
182                 saving=before-self.aggregate(values)-6
183                 if saving>self.value_margin:channels.append(t.channel);net+=saving
184             if channels and (best is None or net>best[0]):best=(net,q,channels)
185         if best:
186             self.robot.record(dict(kind='value_bundle',point=best[1].tolist(),channels=best[2],predicted_saving_s=best[0]))
187         return None if best is None else best[1:]

```

源文件组 22

q3/v6/belief_components.py

```

1  """Belief-sampled one-step policy improvement. No access to real world truth.
2
3  Uniform position/radius priors are efficiency assumptions, never certificates.
4  All physical actions retain bounded-region clearing and whole-cell completion.
5  """
6  import copy
7  import hashlib
8  import math
9  import numpy as np
10 from joint_strategies import JointPlanner,open_route
11
12
13 class BeliefWorld:
14     def __init__(self,targets,robot,salt):
15         self.targets=targets
16         self.position=tuple(robot.position);self.channel=robot.channel
17         self.cleared=set(robot.cleared);self.discovered=set(robot.discovered)
18         self.scanned_sites=0;self.virtual=0.;self.salt=salt
19     def check_budget(self):
20         if self.virtual>50000:raise RuntimeError('rollout cap')
21     def record(self,event):pass
22     def move(self,p):
23         self.virtual+=math.dist(p,self.position)/5;self.position=tuple(p)
24     def measure(self,p,c):

```

```

25     self.move(p);self.virtual+=5+int(c!=self.channel);self.channel=c
26     target=self.targets.get(c) if c not in self.cleared else None
27     d=math.dist(p,target[:2]) if target is not None else math.inf
28     if target is None or d>target[2]:return {'measure_result':'no_signal'}
29     self.discovered.add(c)
30     if d<=5:return {'measure_result':'near'}
31     error=math.sin(.039*p[0]+.021*p[1]+c*2.173+self.salt)
32     a=math.degrees(math.atan2(target[1]-p[1],target[0]-p[0]))
33     return {'measure_result':'direction','svd_deg':round((a+error)%360,2)%360}
34 def clear(self,p,c):
35     self.move(p);t=self.targets.get(c)
36     success=t is not None and c not in self.cleared and math.dist(p,t[:2])<=20
37     self.virtual+=5 if success else 3
38     if success:self.cleared.add(c)
39     return success
40
41
42 def execute(planner,node):
43     kind,c,p=node
44     if kind=='scan':planner.at_site_scan(p,force=True)
45     else:planner.solve_track(planner.tracks[c])
46
47
48 def finish(planner):
49     for _ in range(100):
50         planner.robot.check_budget()
51         pending=[c for c in planner.tracks if c not in planner.robot.cleared]
52         if not pending and not planner.unknown():return planner.robot.virtual
53         planner.at_site_scan(planner.robot.position)
54         nodes=planner.candidates()
55         if not nodes:continue
56         order=open_route(planner.robot.position,[n[2] for n in nodes])
57         execute(planner,nodes[order[0]])
58     raise RuntimeError('rollout exhausted')
59
60
61 class RolloutPlanner(JointPlanner):
62     def __init__(self,robot,*rollout_samples=3,rollout_actions=5,rollout_margin=0,**kwargs):
63         super().__init__(robot,**kwargs)
64         self.rollout_samples=rollout_samples;self.rollout_actions=rollout_actions
65         self.rollout_margin=rollout_margin;self.rollout_calls=0
66
67     def sample_worlds(self):
68         # Seed depends only on the observed history, never the benchmark seed.
69         digest=hashlib.sha256(repr((self.robot.position,[(c,len(t.history)) for c,t in self.tracks.items()]),self.rollout_calls)).encode()).digest()
70         rng=np.random.default_rng(int.from_bytes(digest[:8],'little'))
71         unknown=[c for c in range(1,21) if c not in self.robot.discovered]
72         radius=1800*np.sqrt(rng.random(12000));angle=rng.uniform(0,2*math.pi,12000)
73         particles=np.column_stack([radius*np.cos(angle),radius*np.sin(angle)])
74         if unknown and self.coverage.samples[unknown[0]]:
75             positions=np.asarray(self.coverage.samples[unknown[0]])
76             upper=np.minimum(1500,np.min(np.linalg.norm(particles[:,None,:]-positions[None,:,:],axis=2),axis=1))
77         else:upper=np.full(len(particles),1500.)
78         weights=np.maximum(upper-1000.,0.)
79         probability=float(np.mean(weights)/500)
80         m=len(self.robot.discovered)
81         counts=np.arange(max(10,m),17)
82         probs=np.array([math.comb(int(n),m)*probability**int(n-m) for n in counts],float)
83         if probs.sum()==0:counts=np.array([m]);probs=np.ones(1)
84         probs/=probs.sum()
85         worlds=[]
86         for sample in range(self.rollout_samples):
87             targets={}
88             for c,t in self.tracks.items():
89                 if c in self.robot.cleared:continue
90                 poly=t.poly;tri=poly[1:-1];tri2=poly[2:]
91                 d1=tri-poly[0];d2=tri2-poly[0]
92                 area=np.abs(d1[:,0]*d2[:,1]-d1[:,1]*d2[:,0])
93                 positive=np.asarray([h['position'] for h in t.history])
94                 negative=np.asarray(self.coverage.samples[c])
95                 for _ in range(100):

```

```

96         ids=rng.choice(len(area),size=128,p=area/area.sum())
97         uv=rng.random((128,2));uv[uv.sum(axis=1)>1]=1-uv[uv.sum(axis=1)>1]
98         pts=poly[0]+uv[:,0,None]*d1[ids]+uv[:,1,None]*d2[ids]
99         lower=np.maximum(1000.,np.max(np.linalg.norm(pts[:,None,:]-positive[None,:,:],axis=2),axis=1))
100         upper_r=np.full(128,1500.) if not len(negative) else np.minimum(1500.,np.min(np.linalg.norm(pts[:,None,:]-negative[None,:,:],axis=2),axis=1))
101         ok=np.flatnonzero((upper_r>lower)&(np.linalg.norm(pts,axis=1)<=1800.))
102         if len(ok):
103             mass=upper_r[ok]-lower[ok];j=int(rng.choice(ok,p=mass/mass.sum()));targets[c]=[*pts[j],rng.uniform(lower[j],upper_r[j])];break
104         else:raise RuntimeError('belief rejection exhausted')
105     count=int(rng.choice(counts,p=probs))-m
106     if count:
107         channels=rng.choice(unknown,size=count,replace=False)
108         ids=rng.choice(len(particles),size=count,p=weights/weights.sum())
109         for c,j in zip(channels,ids):targets[int(c)]=[*particles[j],rng.uniform(1000.,upper[j])]
110         worlds.append((targets,float(rng.uniform(0,1000))))
111     return worlds
112
113 def candidates(self):
114     nodes=super().candidates()
115     if len(nodes)<=1:return nodes
116     order=open_route(self.robot.position,[n[2] for n in nodes])
117     # Include baseline choice, nearest alternatives and a search action.
118     choices=[order[0]]
119     nearest=sorted(range(len(nodes)),key=lambda i:math.dist(self.robot.position,nodes[i][2]))
120     scans=[i for i in order if nodes[i][0]=='scan']
121     if scans and scans[0] not in choices:choices.append(scans[0])
122     for i in nearest:
123         if i not in choices:choices.append(i)
124         if len(choices)>=self.rollout_actions:break
125     self.rollout_calls+=1
126     try:worlds=self.sample_worlds()
127     except (RuntimeError,ValueError):return [nodes[order[0]]]
128     state=copy.deepcopy({k:v for k,v in self.__dict__.items() if k!='robot'})
129     scores=[]
130     for i in choices:
131         costs=[]
132         for targets,salt in worlds:
133             robot=BeliefWorld(targets,self.robot,salt)
134             planner=object.__new__(JointPlanner)
135             planner.__dict__=copy.deepcopy(state);planner.robot=robot
136             try:
137                 execute(planner,nodes[i]);costs.append(finish(planner))
138             except (RuntimeError,ArithmeticError,ValueError):costs.append(float('inf'))
139         scores.append(float(np.mean(costs)))
140     best=int(np.argmin(scores))
141     if scores[0]-scores[best]<self.rollout_margin:best=0
142     self.robot.record(dict(kind='rollout_decision',alternatives=[dict(kind=nodes[i][0],channel=int(nodes[i][1]),predicted_remaining_s=s if math.isfinite(s)
143     ↪ else None) for i,s in zip(choices,scores)],selected=best,samples=len(worlds)))
144     return [nodes[choices[best]]]

```

源文件组 23

q3/v6/cover_milp.py

```

1  """Joint optional sensing-stop selection and open routing via bounded MILP."""
2  import numpy as np
3  from scipy.optimize import milp,Bounds,LinearConstraint
4  from scipy.sparse import lil_matrix
5  from service_routing import ServicePlanner
6
7
8  def solve_cover_route(points,mandatory,masks,query_cost,time_limit=1.):
9      n=len(points);edges=[(i,j) for i in range(n) for j in range(n) if i!=j]
10     ne=len(edges);idx={edge:k for k,edge in enumerate(edges)}
11     # x directed edges; f single-commodity connectivity flow; y visits; z scans.
12     yi=2*ne;zi=yi+n;size=zi+n
13     c=np.zeros(size);integrality=np.zeros(size,int)
14     lower=np.zeros(size);upper=np.ones(size);upper[ne:2*ne]=n-1
15     integrality[:ne]=1;integrality[yi:]=1

```

```

16 D=np.linalg.norm(np.asarray(points)[:,None,:]-np.asarray(points)[None,:,:],axis=2)/5
17 for k,(i,j) in enumerate(edges):c[k]=0. if j==0 else D[i,j]
18 c[zi]=query_cost
19 lower[yi]=1
20 for j in mandatory:lower[yi+j]=1
21 constraints=[];lo=[];hi=[]
22 def add(items,l,u):constraints.append(items);lo.append(l);hi.append(u)
23 for j in range(n):
24     add([(idx[i,j],1.) for i in range(n) if i!=j]+[(yi+j,-1.)],0.,0.)
25     add([(idx[j,k],1.) for k in range(n) if k!=j]+[(yi+j,-1.)],0.,0.)
26     add([(zi+j,1.), (yi+j,-1.)],-np.inf,0.)
27 for j in range(1,n):
28     add([(ne+idx[i,j],1.) for i in range(n) if i!=j]+[(ne+idx[j,k],-1.) for k in range(n) if k!=j]+[(yi+j,-1.)],0.,0.)
29 for k,(i,j) in enumerate(edges):
30     add([(ne+k,1.), (k,-(n-1.))],-np.inf,0.)
31     if j==0:upper[ne+k]=0.
32 for mask in np.unique(np.asarray(masks,dtype=bool).T,axis=0):
33     if not np.any(mask):return None
34     add([(zi+j,1.) for j in np.flatnonzero(mask)],1.,np.inf)
35 A=lil_matrix((len(constraints),size))
36 for i,items in enumerate(constraints):
37     for j,value in items:A[i,j]+=value
38 result=milp(c,integrality=integrality,bounds=Bounds(lower,upper),
39 constraints=LinearConstraint(A.tocsc(),np.asarray(lo),np.asarray(hi)),
40 options={'time_limit':time_limit,'mip_rel_gap':.01})
41 if result.x is None:return None
42 selected={j for j in range(n) if result.x[yi+j]>.5};active={j for j in range(n) if result.x[zi+j]>.5}
43 successors={i:j for k,(i,j) in enumerate(edges) if result.x[k]>.5}
44 route=[0]
45 for _ in range(n):
46     j=successors.get(route[-1])
47     if j is None:return None
48     if j==0:break
49     if j in route:return None
50     route.append(j)
51 if set(route)!=selected or not set(mandatory).issubset(selected):return None
52 if len(np.asarray(masks).T) and not np.all(np.any(np.asarray(masks)[active],axis=0)):return None
53 cost=sum(D[a,b] for a,b in zip(route,route[1:]))+query_cost*len(active)
54 return dict(route=route[1:],active=active,cost=float(cost),gap=float(result.mip_gap),status=int(result.status))
55
56
57 class MilpPlanner(ServicePlanner):
58     def __init__(self,robot,*,milp_seconds=1.,milp_budget=16,**kwargs):
59         super().__init__(robot,**kwargs);self.milp_seconds=milp_seconds
60         self.milp_budget=milp_budget;self.milp_calls=0
61
62     def candidates(self):
63         baseline=super().candidates();unknown=self.unknown()
64         if not baseline or not unknown or self.pending_probe is not None or self.milp_calls>=self.milp_budget:return baseline
65         _,nodes,_=self.last_plan
66         targets=[n for n in nodes if n[0]=='target']
67         if not targets:return baseline
68         points=[np.asarray(self.robot.position)]+[n[2] for n in targets]
69         for q in [n[2] for n in nodes if n[0]=='scan']+[np.asarray(p) for p in self.anchors[1:]]:
70             if not any(np.linalg.norm(q-p)<.1 for p in points):points.append(q)
71         missing=~self.coverage.covered[unknown[0]]
72         masks=np.array([self.coverage.mask(p)[missing] for p in points])
73         self.milp_calls+=1
74         solution=solve_cover_route(points,range(1,len(targets)+1),masks,6*len(unknown),self.milp_seconds)
75         if solution is None or solution['cost']>=self.last_plan[0]/5-1e-6:return baseline
76         self.robot.record(dict(kind='joint_cover_milp',predicted_cost_s=solution['cost'],incumbent_cost_s=self.last_plan[0]/5,
77             relative_solver_gap=solution['gap'],solver_status=solution['status'],nodes=len(points),mandatory_targets=len(targets)))
78         j=0 if 0 in solution['active'] else solution['route'][0]
79         if j in solution['active']:return [('scan',-20000-j,points[j])]
80         if 1<=j<=len(targets):return [targets[j-1]]
81         return baseline

```

源文件组 24

q3/v6/flexible_cover.py

```

1  """Continuous waypoint relocation with an explicit full-coverage constraint."""
2  import math
3  import numpy as np
4  from scipy.optimize import minimize
5  from scipy.spatial import ConvexHull
6  from joint_strategies import JointPlanner, open_route
7  from bearing_geometry import enclosing_circle, diameter
8
9
10 class FlexibleCoverPlanner(JointPlanner):
11     def __init__(self, robot, *, cover_sweeps=2, merge_stations=True, merge_target_scans=False, **kwargs):
12         super().__init__(robot, family='adaptive', **kwargs)
13         self.cover_sweeps=cover_sweeps
14         self.merge_stations=merge_stations
15         self.merge_target_scans=merge_target_scans
16
17     def candidates(self):
18         targets=[]
19         for c,t in self.tracks.items():
20             if c in self.robot.cleared:continue
21             q=t.safe_clear_point(self.robot.position)
22             if q is None:
23                 q=t.next_point(np.asarray(self.robot.position)) if getattr(self,'cover_target_entry',False) else self.target_position(t)
24             targets.append(('target',c,np.asarray(q)))
25         unknown=self.unknown()
26         if not unknown:return targets
27         actual=self.coverage.covered[unknown[0]]
28         # Targets are mandatory route vertices but optional sensing sites.
29         points=[n[2].copy() for n in targets]+[np.array(p) for p in self.anchors]
30         n_target=len(targets)
31         active=np.ones(len(points),dtype=bool)
32         present=np.ones(len(points),dtype=bool)
33         masks=[self.coverage.mask(p) for p in points]
34         start=np.asarray(self.robot.position)
35         def route_order():
36             indices=[i for i in range(len(points)) if present[i]]
37             return [indices[j] for j in open_route(start,[points[i] for i in indices])]
38         def total_without(i):
39             total=actual.copy()
40             for j in range(len(points)):
41                 if j!=i and active[j]:total+=masks[j]
42             return total
43         def exact_union_without(i):
44             sites=list(self.coverage.samples[unknown[0]]+[points[j] for j in range(len(points)) if j!=i and active[j]])
45             return self.coverage.exact_status(sites)[0]<=999.98
46         def prune():
47             while True:
48                 order=route_order();gains=[]
49                 for k,i in enumerate(order):
50                     if not active[i]:continue
51                     gain=30*len(unknown)
52                     if i>=n_target:
53                         a=start if k==0 else points[order[k-1]]
54                         gain+=np.linalg.norm(points[i]-a)
55                         if k+1<len(order):
56                             b=points[order[k+1]]
57                             gain+=np.linalg.norm(points[i]-b)-np.linalg.norm(a-b)
58                     gains.append((gain,i))
59                 removed=False
60                 for gain,i in sorted(gains,reverse=True):
61                     if np.all(total_without(i)) or (getattr(self,'hybrid_cover',False) and exact_union_without(i)):
62                         active[i]=False
63                         if i>=n_target:present[i]=False
64                         removed=True;break
65                 if not removed:break
66         prune()
67         if self.merge_stations:

```

```

68     def full_cost(candidate_points,candidate_present,candidate_active):
69         indices=np.flatnonzero(candidate_present)
70         order=open_route(start,[candidate_points[i] for i in indices])
71         p=start;value=30*len(unknown)*int(np.count_nonzero(candidate_active))
72         for j in order:
73             q=candidate_points[indices[j]];value+=np.linalg.norm(q-p);p=q
74         return float(value)
75     for merge_iteration in range(4):
76         old_cost=full_cost(points,present,active);best=None
77         selected=[i for i in range(0 if self.merge_target_scans else n_target,len(points)) if active[i]]
78         for at,i in enumerate(selected):
79             for j in selected[at]:
80                 remaining=actual.copy()
81                 for k in range(len(points)):
82                     if active[k] and k not in [i,j]:remaining|=masks[k]
83                 needed=self.coverage.centers[~remaining]
84                 if not len(needed):continue
85                 h=self.coverage.half
86                 corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in [(-1,-1),(-1,1),(1,-1),(1,1)])])
87                 hull=corners[ConvexHull(corners).vertices]
88                 if diameter(hull)[0]>1999.9799:continue
89                 center,radius,_=enclosing_circle(hull)
90                 if radius>999.98999:continue
91                 new_points=points+[center]
92                 new_present=np.append(present,True);new_active=np.append(active,True)
93                 for removed in [i,j]:
94                     if removed>=n_target:new_present[removed]=False
95                     new_active[removed]=False
96                 value=full_cost(new_points,new_present,new_active)
97                 if value<old_cost-1e-6 and (best is None or value<best[0]):
98                     best=(value,i,j,center,new_points,new_present,new_active)
99             if best is None:break
100         value,i,j,center,points,present,active=best
101         masks.append(self.coverage.mask(center))
102         self.robot.record(dict(kind='merge_scans',point=center.tolist(),removed=[int(i),int(j)],
103             predicted_saving_s=(old_cost-value)/5,proof='MEC of all responsibility-cell corners <= 999.99 m'))
104         prune()
105     for _ in range(self.cover_sweeps):
106         order=route_order()
107         improvement=0.
108         for k,i in enumerate(order):
109             if i<n_target or not active[i]:continue
110             needed=self.coverage.centers[~total_without(i)]
111             if not len(needed):continue
112             h=self.coverage.half
113             corners=np.vstack([needed+np.array([x*h,y*h]) for x,y in [(-1,-1),(-1,1),(1,-1),(1,1)])])
114             witnesses=corners[ConvexHull(corners).vertices]/1000
115             before=points[i]/1000
116             previous=(start if k==0 else points[order[k-1]])/1000
117             following=points[order[k+1]]/1000 if k+1<len(order) else None
118             def objective(q):
119                 value=np.sqrt(np.dot(q-previous,q-previous)+1e-16)
120                 if following is not None:value+=np.sqrt(np.dot(q-following,q-following)+1e-16)
121                 return float(value)
122             def gradient(q):
123                 value=(q-previous)/max(np.linalg.norm(q-previous),1e-8)
124                 if following is not None:value+=(q-following)/max(np.linalg.norm(q-following),1e-8)
125                 return value
126             cons={'type':'ineq','fun':lambda q: .99998999**2-np.sum((q-witnesses)**2,axis=1),
127                 'jac':lambda q:-2*(q-witnesses)}
128             sol=minimize(objective,before,jac=gradient,constraints=cons,method='SLSQP',
129                 options={'ftol':1e-10,'maxiter':60})
130             if np.max(np.linalg.norm(witnesses-sol.x,axis=1))<=.99999 and objective(sol.x)<=objective(before)+1e-8:
131                 improvement+=objective(before)-objective(sol.x)
132                 points[i]=sol.x*1000;masks[i]=self.coverage.mask(points[i])
133             prune()
134             if improvement<1e-5:break
135     union=actual.copy()
136     for i in range(len(points)):
137         if active[i]:union|=masks[i]
138     if not np.all(union) and not (getattr(self,'hybrid_cover',False) and exact_union_without(-1)):

```

```

139         raise ArithmeticError('Relocated cover lost coverage')
140     scans=[('scan',-100-i,points[i]) for i in range(len(points)) if active[i]]
141     nodes=targets+scans
142     self.cover_targets=set()
143     return nodes

```

源文件组 25

q3/v6/lookahead.py

```

1  """Sparse posterior-consistent policy rollouts, using observation history only."""
2  import copy,math
3  import numpy as np
4  from service_routing import ServicePlanner
5  from belief_components import BeliefWorld,RolloutPlanner,execute
6
7
8  def complete(planner):
9      for _ in range(120):
10         planner.robot.check_budget()
11         pending=[c for c in planner.tracks if c not in planner.robot.cleared]
12         if not pending and not planner.unknown():return planner.robot.virtual
13         gain=planner.scan_gain;planner.scan_gain=2.
14         try:planner.at_site_scan(planner.robot.position)
15         finally:planner.scan_gain=gain
16         nodes=planner.candidates()
17         if nodes:execute(planner,nodes[0])
18         raise RuntimeError('Lookahead completion cap')
19
20
21 class LookaheadPlanner(ServicePlanner):
22     sample_worlds=RolloutPlanner.sample_worlds
23
24     def __init__(self,robot,*,rollout_samples=2,rollout_actions=3,rollout_margin=20.,**kwargs):
25         super().__init__(robot,**kwargs)
26         self.rollout_samples=rollout_samples;self.rollout_actions=rollout_actions
27         self.rollout_margin=rollout_margin;self.rollout_calls=0;self.rollout_stages=set()
28
29     def candidates(self):
30         result=super().candidates();stage=len(self.robot.cleared)//4
31         if not result or self.pending_probe is not None or stage>2 or stage in self.rollout_stages:return result
32         if self.last_plan is None or len(self.last_plan[1])<2:return result
33         self.rollout_stages.add(stage);self.rollout_calls+=1
34         _,nodes,order=self.last_plan
35         choices=list(result)
36         ordered=[nodes[i] for i in order]
37         scans=[n for n in ordered if n[0]=='scan']
38         for node in (scans[1:]+ordered):
39             if any(node[0]==old[0] and node[1]==old[1] and np.linalg.norm(node[2]-old[2])<.1 for old in choices):continue
40             choices.append(node)
41             if len(choices)>=self.rollout_actions:break
42         try:worlds=self.sample_worlds()
43         except (RuntimeError,ValueError):return result
44         state=copy.deepcopy({k:v for k,v in self.__dict__.items() if k!='robot'})
45         scores=[]
46         for node in choices:
47             costs=[]
48             for targets,salt in worlds:
49                 robot=BeliefWorld(targets,self.robot,salt)
50                 planner=object.__new__(ServicePlanner);planner.__dict__=copy.deepcopy(state);planner.robot=robot
51                 planner.cover_trials=1;planner.cover_radii=None;planner.transit_probes=False
52                 planner.service_route=False;planner.pending_probe=None;planner.merge_trials=False
53                 try:execute(planner,node);costs.append(complete(planner))
54                 except (RuntimeError,ArithmeticError,ValueError):costs.append(float('inf'))
55             scores.append(float(np.mean(costs)))
56         best=int(np.argmin(scores))
57         if not math.isfinite(scores[best]) or scores[0]-scores[best]<self.rollout_margin:best=0
58         self.robot.record(dict(kind='posterior_lookahead',stage=stage,samples=len(worlds),
59             alternatives=[dict(kind=n[0],channel=int(n[1]),predicted_remaining_s=s if math.isfinite(s) else None) for n,s in zip(choices,scores)],

```

```

60         selected=best,prior='uniform area/radius and source count 10..16; planning only'))
61     return [choices[best]]

```

源文件组 26

q3/v6/run_robot.py

```

1  """ 问题三多策略研究版；尚未达到平均 200 秒/点目标。 """
2  import argparse
3  import datetime as dt
4  import hashlib
5  import json
6  from pathlib import Path
7  import shutil
8  import sys
9  import time
10 import uuid
11
12 ROOT = Path(__file__).resolve().parent
13
14
15 def main():
16     if hasattr(sys.stdout, 'reconfigure'):
17         sys.stdout.reconfigure(encoding='utf-8')
18     parser = argparse.ArgumentParser(description=__doc__)
19     parser.add_argument('--policy', choices=['v5','v6','conservative','no_update'], default='v6', help='选择冻结策略，默认 v6 光学动作价值候选')
20     parser.add_argument('--team', help='默认复用旧策略的 config.json 队号')
21     parser.add_argument('--port', type=int, help='默认复用旧配置，缺省端口 2026')
22     parser.add_argument('--config', type=Path, help='指定含 team_no、port 的配置文件')
23     parser.add_argument('--check', action='store_true', help='仅检查依赖和配置，不连接模拟器')
24     args = parser.parse_args()
25     try:
26         import numpy
27         import scipy
28         import shapely
29         from api_client import Robot, BudgetReached
30         from strategy import Planner, VERSION, POLICIES
31     except ImportError as error:
32         print(f'缺少依赖：{error}\n 请安装 requirements.txt 中的依赖后重试。')
33         return 2
34     paths = [args.config] if args.config else [ROOT / 'config.json', ROOT.parent / 'simple-strategy' / 'config.json']
35     config_path = next((p for p in paths if p and p.is_file()), None)
36     config = json.loads(config_path.read_text(encoding='utf-8-sig')) if config_path else {}
37     team, port = args.team or config.get('team_no'), args.port or config.get('port', 2026)
38     if not team:
39         print('找不到队号配置，请使用 --config 指定旧 config.json，或使用 --team 参数。')
40         return 2
41     if args.check:
42         print(f'策略 {args.policy}；研究版，200 秒/点目标尚未达成。')
43         print(f'自检通过：Python {sys.version.split()[0]} / numpy {numpy.__version__} / scipy {scipy.__version__}')
44         print(f'配置可读；端口 {port}；未连接模拟器。')
45         return 0
46     print(f'问题 3 研究策略 {VERSION} / {args.policy} | 端口 {port}')
47     print('这是可运行的研究候选，尚未达到平均 200 秒/点目标。')
48     print('请在模拟器选择【问题 3 演练测试】，开始后等待“接口已就绪/等待机器狗进入”。')
49     print('接口无法查询演练/正式模式，程序按问题 3 全向干扰器规则工作。')
50     input('界面就绪后按 Enter 开始：')
51     folder = ROOT / 'runs' / (dt.datetime.now().strftime('%Y%m%d-%H%M%S') + '-' + uuid.uuid4().hex[6])
52     robot = Robot(team, f'http://127.0.0.1:{port}', folder)
53     planner = Planner(robot, policy=args.policy)
54     snapshot = folder / 'source_snapshot'
55     snapshot.mkdir()
56     hashes = {}
57     for p in list(ROOT.glob('*.*py')) + [ROOT / 'policies.json']:
58         shutil.copy2(p, snapshot / p.name)
59         hashes[p.name] = hashlib.sha256(p.read_bytes()).hexdigest()
60     (folder / 'manifest.json').write_text(json.dumps(dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy],
61     python=sys.version, numpy=numpy.__version__, scipy=scipy.__version__, shapely=shapely.__version__, source_sha256=hashes,
62     config_path=str(config_path), mode='user_selected_P3_practice_not_API_verified'),

```

```

63     ensure_ascii=False, indent=2), encoding='utf-8')
64 outcome, error_text, entered = 'completed', None, False
65 robot.started = time.monotonic()
66 try:
67     robot.call('/enter')
68     entered = True
69     planner.run()
70 except BudgetReached as error:
71     outcome, error_text = 'budget_stop', str(error)
72 except (Exception, KeyboardInterrupt) as error:
73     outcome, error_text = 'error', str(error) or '用户中断'
74     # Interruption during HTTP could leave execution state unknown.
75     if isinstance(error, KeyboardInterrupt):
76         robot.transport_uncertain = True
77 finally:
78     if entered and not robot.transport_uncertain:
79         try:
80             robot.call('/exit')
81         except Exception as error:
82             outcome = 'exit_unconfirmed'
83             error_text = (error_text or '') + '\n退出未确认: ' + str(error)
84 count = len(robot.cleared)
85 summary = dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy], outcome=outcome, error=error_text,
86               exit_confirmed=robot.exit_confirmed, transport_uncertain=robot.transport_uncertain,
87               cleared_count=count, cleared_channels=sorted(robot.cleared),
88               observed_but_uncleared=sorted(robot.discovered - robot.cleared),
89               measure_count=robot.measure_count, clear_failures=robot.clear_failures,
90               scanned_sites=robot.scanned_sites, distance_m=robot.distance_m, switches=robot.switches,
91               virtual_time_s=robot.virtual, average_clear_time_s=robot.virtual / count if count else None,
92               local_elapsed_s=time.monotonic() - robot.started,
93               actual_jammer_count=None, completion=planner.certificate(),
94               note='实际目标总数和正式成绩以模拟器为准; 完整性证书以问题 3 全向规则为前提。')
95 (folder / 'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
96 robot.log.close()
97 print(f'\n结束: 清除 {count} 个; 虚拟时间 {robot.virtual:.1f} 秒; 移动 {robot.distance_m:.1f} 米。')
98 print(f'状态 {outcome}; 退出确认 {robot.exit_confirmed}; 结束依据 {planner.stop_reason}')
99 if error_text:
100     print(error_text)
101 print(f'运行记录: {folder}')
102 print('请保存模拟器结束界面、案例编码、实际目标数, 并导出官方 .jlog。')
103 return 0 if outcome == 'completed' and robot.exit_confirmed else 1
104
105
106 if __name__ == '__main__':
107     sys.exit(main())

```

源文件组 27

q3/v6/service_routing.py

```

1  """Whole-service routing, rotated cover plans and paid measurements in transit."""
2  import copy, math
3  import numpy as np
4  from combined_cost_cover import CombinedCostCoverPlanner
5  from flexible_cover import FlexibleCoverPlanner
6  from joint_strategies import open_route
7  from voronoi_coverage import HybridCoverage, covering_radius
8
9
10 def directed_route(start, nodes, tracks, restarts=3):
11     exits=[np.asarray(start)]+[np.asarray(tracks[n[1]].center if n[0]=='target' else n[2]) for n in nodes]
12     size=len(nodes);D=np.zeros((size+1,size+1))
13     for i,p in enumerate(exits):
14         for j,(kind,c,q) in enumerate(nodes,1):
15             if i==j:continue
16             entry=q
17             if kind=='target':
18                 tr=tracks[c];safe=tr.safe_clear_point(p)
19                 entry=safe if safe is not None else tr.next_point(p)

```

```

20     D[i,j]=np.linalg.norm(entry-p)+np.linalg.norm(exits[j]-entry)
21 def value(r):return float(sum(D[a,b] for a,b in zip(r,r[1:])))
22 best=None
23 for restart in range(min(restarts,size)):
24     left=set(range(1,size+1));r=[0]
25     first=sorted(left,key=lambda j:D[0,j])[restart];r.append(first);left.remove(first)
26     while left:
27         j=min(left,key=lambda j:D[r[-1],j]);r.append(j);left.remove(j)
28     for _ in range(25):
29         old=value(r);chosen=None
30         for i in range(1,len(r)-1):
31             for j in range(i+1,len(r)):
32                 trial=r[:i]+list(reversed(r[i:j+1]))+r[j+1:];v=value(trial)
33                 if v<old-1e-7:old,chosen=v,trial
34             if chosen is None:break
35         r=chosen
36     val=value(r)
37     if best is None or val<best[0]:best=(val,r)
38 return [j-1 for j in best[1][1:]],best[0]
39
40
41 class ServicePlanner(CombinedCostCoverPlanner):
42     def __init__(self,robot,*service_route=False,cover_trials=1,transit_probes=False,
43         probe_gain=100,probe_min_radius=100,cover_rad=0,initial_design=False,cover_target_entry=False,target_estimator='mec',merge_trials=1,
44         warm_cover=False,hybrid_cover=False,continuous_stop=False,**kwargs):
45         super().__init__(robot,**kwargs)
46         self.service_route=service_route;self.cover_trials=cover_trials
47         self.transit_probes=transit_probes;self.probe_gain=probe_gain
48         self.probe_min_radius=probe_min_radius;self.pending_probe=None
49         self.cover_rad=cover_rad;self.initial_design=initial_design
50         self.initial_designed=False
51         self.cover_target_entry=cover_target_entry
52         self.target_estimator=target_estimator;self.merge_trials=merge_trials
53         self.warm_cover=warm_cover;self.warm_scans=[]
54         self.hybrid_cover=hybrid_cover
55         if hybrid_cover or continuous_stop:self.coverage=HybridCoverage()
56
57     def target_position(self,tr):
58         if self.target_estimator=='mec':return tr.center
59         v=tr.poly-tr.center;u=np.roll(v,-1,axis=0)
60         cross=v[:0]*u[:1]-u[:0]*v[:1]
61         center=tr.center+np.sum((v+u)*cross[:,None],axis=0)/(3*cross.sum()) if abs(cross.sum())>1e-10 else tr.center
62         if self.target_estimator=='wls':
63             A=[];b=[]
64             for h in tr.history:
65                 if h['measure_result']!='direction':continue
66                 angle=math.radians(h['svd_deg']);p=np.asarray(h['position'])
67                 normal=np.array([-math.sin(angle),math.cos(angle)])/max(10,np.linalg.norm(center-p))
68                 A.append(normal);b.append(np.dot(normal,p))
69             if len(A)>=2:
70                 estimate=np.linalg.lstsq(np.asarray(A),np.asarray(b),rcond=None)[0]
71                 edges=np.roll(tr.poly,-1,axis=0)-tr.poly;delta=estimate-tr.poly
72                 if np.all(edges[:,0]*delta[:,1]-edges[:,1]*delta[:,0]>=-1e-7):center=estimate
73         return center
74
75     def design_initial_point(self):
76         original_radius=self.initial_baseline;best=None
77         for radius in [250.,450.,650.]:
78             for angle in np.linspace(0,2*math.pi,12,endpoint=False):
79                 q=radius*np.array([math.cos(angle),math.sin(angle)]);centers=[];uncertainty=0.
80                 for tr in self.tracks.values():
81                     if np.linalg.norm(q-tr.center)>1000:
82                         centers.append(tr.center);uncertainty+=tr.radius;continue
83                     bearing=math.degrees(math.atan2(tr.center[1]-q[1],tr.center[0]-q[0]))
84                     radii=[];cs=[]
85                     for error in [-1.,0.,1.]:
86                         temporary=copy.copy(tr);temporary.history=list(tr.history);temporary.poly=tr.poly.copy()
87                         temporary.update(q,dict(measure_result='direction',svd_deg=round((bearing+error)%360,2)))
88                         radii.append(temporary.radius);cs.append(temporary.center)
89                     centers.append(np.mean(cs,axis=0));uncertainty+=np.mean(radii)
90         points=centers+[np.asarray(p) for p in self.anchors[1:]]

```

```

90         order=open_route(q,points);p=q;score=radius+uncertainty
91         for i in order:score+=np.linalg.norm(points[i]-p);p=points[i]
92         if best is None or score<best[0]:best=(score,q)
93     return best[1]
94
95 def route(self,nodes):
96     if self.service_route:return directed_route(self.robot.position,nodes,self.tracks)
97     order=open_route(self.robot.position,[n[2] for n in nodes],restarts=self.route_restarts)
98     p=np.asarray(self.robot.position);cost=0.
99     for i in order:cost+=np.linalg.norm(nodes[i][2]-p);p=nodes[i][2]
100    return order,float(cost)
101
102 def probe_on_leg(self,node):
103    start=np.asarray(self.robot.position);end=node[2]
104    if node[0]=='target':
105        tr=self.tracks[node[1]];safe=tr.safe_clear_point(start)
106        end=safe if safe is not None else tr.next_point(start)
107    if np.linalg.norm(end-start)<250:return None
108    best=None
109    for f in [.25,.5,.75]:
110        q=start+f*(end-start);channels=[];benefit=0.
111        for c,tr in self.tracks.items():
112            if c in self.robot.cleared or tr.radius<self.probe_min_radius:continue
113            if min(math.dist(q,h['position']) for h in tr.history)<150:continue
114            if math.dist(q,self.attempts.get(c,(1e9,1e9)))<150:continue
115            if np.linalg.norm(q-tr.center)>1000:continue
116            radii=[]
117            angle=math.degrees(math.atan2(tr.center[1]-q[1],tr.center[0]-q[0]))
118            for error in [-1.,0.,1.]:
119                temporary=copy.copy(tr);temporary.history=list(tr.history);temporary.poly=tr.poly.copy()
120                temporary.update(q,dict(measure_result='direction',svd_deg=round((angle+error)%360,2)))
121                radii.append(temporary.radius)
122            gain=tr.radius-float(np.mean(radii))
123            if gain>=self.probe_gain:channels.append(c);benefit+=gain-30
124            if channels and (best is None or benefit>best[0]):best=(benefit,q,channels)
125    return None if best is None else best[1:]
126
127 def plan(self):
128    original=self.anchors;radius=math.hypot(*original[1]);best=None;original_merge=self.merge_stations
129    try:
130        for trial in range(self.cover_trials*len(self.cover_radii or [radius])):
131            radius=(self.cover_radii or [radius])[trial//self.cover_trials]
132            rotation=trial%self.cover_trials
133            angle=math.pi/3*rotation/self.cover_trials
134            self.anchors=[(0.,0.)]+[(radius*math.cos(angle+i*math.pi/3),radius*math.sin(angle+i*math.pi/3)) for i in range(6)]
135            cover=np.zeros(len(self.coverage.centers),dtype=bool)
136            for p in self.anchors:cover|=self.coverage.mask(p)
137            if not np.all(cover) and not (self.hybrid_cover and covering_radius(self.anchors)[0]<=999.98):continue
138            for merged in ([False,True] if self.merge_trials else [original_merge]):
139                self.merge_stations=merged
140                nodes=FlexibleCoverPlanner.candidates(self)
141                if not nodes:continue
142                order,cost=self.route(nodes)
143                score=cost+30*len(self.unknown())*sum(n[0]=='scan' for n in nodes)
144                if best is None or score<best[0]:best=(score,nodes,order)
145            if self.warm_cover and self.warm_scans:
146                self.anchors=original+[tuple(p) for p in self.warm_scans]
147                self.merge_stations=original_merge
148                nodes=FlexibleCoverPlanner.candidates(self)
149                if nodes:
150                    order,cost=self.route(nodes)
151                    score=cost+30*len(self.unknown())*sum(n[0]=='scan' for n in nodes)
152                    if best is None or score<best[0]:best=(score,nodes,order)
153            finally:self.anchors=original;self.merge_stations=original_merge
154    return best
155
156 def candidates(self):
157    best=self.plan();self.last_plan=best
158    if best is None:return []
159    _,nodes,order=best;first=nodes[order[0]]
160    if self.warm_cover:self.warm_scans=[n[2].copy() for n in nodes if n[0]=='scan']

```

```

161     # A planned scan at the target point must happen before a moving solve.
162     if first[0]=='target':
163         match=[n for n in nodes if n[0]=='scan' and math.dist(n[2],first[2])<1e-5]
164         if match: first=match[0]
165     if self.transit_probes:
166         probe=self.probe_on_leg(first)
167         if probe is not None:
168             self.pending_probe=probe
169         return [('scan',-9999,probe[0])]
170     return [first]
171
172 def at_site_scan(self,point,force=False):
173     if self.initial_design and not self.initial_designed and self.iterations==0 and np.linalg.norm(point)>.1:
174         point=self.design_initial_point();self.initial_designed=True
175         self.robot.record(dict(kind='initial_design',point=point.tolist()))
176     if force and self.pending_probe is not None and math.dist(point,self.pending_probe[0])<1e-6:
177         q,channels=self.pending_probe;self.pending_probe=None
178         self.robot.record(dict(kind='transit_probe',point=q.tolist(),channels=channels))
179         for c in sorted(channels,key=lambda c:(c!=self.robot.channel,c)):self.observe(q,c)
180         return
181     return super().at_site_scan(point,force=force)
182
183 def certificate(self):
184     result=super().certificate()
185     if isinstance(self.coverage,HybridCoverage):
186         result['exact_disk_cover']={str(c):dict(max_nearest_distance_m=self.coverage.exact_status(self.coverage.samples[c])[0],
187         witness=self.coverage.exact_status(self.coverage.samples[c])[1].tolist(),accepted_by_grid=bool(np.all(self.coverage.covered[c]))) for c in
188         ↪ result['absent_channels']}
189     return result
190
191 class TransitSearchPlanner(ServicePlanner):
192     def __init__(self,robot,*,search_margin_s=10.,search_bonus=0.,**kwargs):
193         super().__init__(robot,**kwargs)
194         self.search_margin_s=search_margin_s;self.search_bonus=search_bonus
195
196     def candidates(self):
197         nodes=super().candidates();unknown=self.unknown()
198         if not nodes or not unknown or self.pending_probe is not None: return nodes
199         first=nodes[0];p=np.asarray(self.robot.position);q=first[2]
200         if first[0]=='target':
201             tr=self.tracks[first[1]];safe=tr.safe_clear_point(p)
202             q=safe if safe is not None else tr.next_point(p)
203         if np.linalg.norm(q-p)<350: return nodes
204         previous=self.coverage.covered[unknown].copy();old_position=self.robot.position
205         before=self.last_plan[0];best=None
206         try:
207             for fraction in [.33,.67]:
208                 probe=p+fraction*(q-p);mask=self.coverage.mask(probe)
209                 gain=float(np.mean(mask&~previous[0]))
210                 if gain<.015: continue
211                 self.coverage.covered[unknown]=previous|mask
212                 self.robot.position=tuple(probe)
213                 after=self.plan()
214                 if after is None: continue
215                 cost=np.linalg.norm(probe-p)+30*len(unknown)+after[0]
216                 hidden=max(0,13-len(self.robot.discovered))
217                 bonus=self.search_bonus*hidden*gain/max(.05,float(np.mean(~previous[0])))
218                 saving=(before-cost+bonus)/5
219                 if saving>self.search_margin_s and (best is None or saving>best[0]): best=(saving,probe,gain)
220         finally:
221             self.coverage.covered[unknown]=previous;self.robot.position=old_position
222         if best is not None:
223             self.robot.record(dict(kind='transit_search_plan',point=best[1].tolist(),predicted_saving_s=best[0],new_coverage_fraction=best[2],planning_only=
224             ↪ True))
225             return [('scan',-9998,best[1])]
226     return nodes

```

源文件组 28

q3/v6/strategy.py

```

1  """Frozen optical-action candidate and previous v5 control."""
2  import json
3  from pathlib import Path
4  from service_routing import ServicePlanner
5  from action_value import ActionValuePlanner
6  VERSION='p3-action-value-v6-research'
7  POLICIES=json.loads(Path(__file__).with_name('policies.json').read_text(encoding='utf-8'))
8  def Planner(robot,*,policy='v6'):
9      params=POLICIES[policy].copy();kind=params.pop('_class',None)
10     return (ActionValuePlanner if kind=='action' else ServicePlanner)(robot,**params)

```

源文件组 29

q3/v6/voronoi_coverage.py

```

1  """Exact finite candidates for maximum distance to nearest sensing site in a disk."""
2  import itertools
3  import numpy as np
4  from legacy_strategy import Coverage
5
6
7  def covering_radius(sites,arena_radius=1800.):
8      p=np.unique(np.asarray(sites,float).reshape(-1,2),axis=0)
9      if not len(p):return float('inf'),np.array([arena_radius,0.])
10     candidates=[np.array([arena_radius,0.])]
11     norms=np.linalg.norm(p,axis=1)
12     candidates.extend(-arena_radius*p[norms>1e-10]/norms[norms>1e-10,None])
13     for i,a in enumerate(p):
14         for b in p[i:]:
15             d=b-a;length=np.linalg.norm(d)
16             if length<1e-10:continue
17             normal=d/length;offset=np.dot((a+b)/2,normal)
18             if abs(offset)>arena_radius:continue
19             foot=offset*normal;side=np.sqrt(max(0.,arena_radius**2-offset**2))*np.array([-normal[1],normal[0]])
20             candidates.extend([foot+side,foot-side])
21     triples=np.array(list(itertools.combinations(range(len(p)),3)),dtype=int)
22     if len(triples):
23         a,b,c=p[triples[:,0]],p[triples[:,1]],p[triples[:,2]]
24         u=b-a;v=c-a;det=u[:,0]*v[:,1]-u[:,1]*v[:,0]
25         ok=abs(det)>1e-12*np.maximum(1.,np.linalg.norm(u,axis=1)*np.linalg.norm(v,axis=1))
26         a,u,v,det=a[ok],u[ok],v[ok],det[ok]
27         hu=np.sum(u*u,axis=1)/2;hv=np.sum(v*v,axis=1)/2
28         centers=a+np.column_stack([(hu*v[:,1]-hv*u[:,1])/det,(u[:,0]*hv-v[:,0]*hu)/det])
29         candidates.extend(centers[np.linalg.norm(centers,axis=1)<=arena_radius])
30     x=np.asarray(candidates);values=np.min(np.linalg.norm(x[:,None,:]-p[None,:],axis=2),axis=1)
31     k=int(np.argmax(values));return float(values[k]),x[k].copy()
32
33
34 class HybridCoverage(Coverage):
35     def __init__(self):
36         super().__init__();self.exact_cache={}
37
38     def exact_status(self,sites):
39         key=tuple(sorted(set(tuple(float(x) for x in p) for p in sites)))
40         if key not in self.exact_cache:self.exact_cache[key]=covering_radius(key)
41         return self.exact_cache[key]
42
43     def absent(self,channel):
44         return super().absent(channel) or self.exact_status(self.samples[channel])[0]<=999.98

```

源文件组 30

q4/analysis/redraw_figures.py

```

1  """Compare frozen literature CG, archived M07 and M32; no simulation here."""
2  from pathlib import Path
3  import json, csv, shutil, hashlib, sys
4  import numpy as np
5  from scipy.stats import t
6  import matplotlib
7  matplotlib.use('Agg')
8  import matplotlib.pyplot as plt
9  from matplotlib.patches import Circle
10 HERE=Path(__file__).resolve().parent
11 DATA=HERE/'evidence'; FIG=HERE/'figures'
12 DATA.mkdir(exist_ok=True); FIG.mkdir(parents=True, exist_ok=True)
13 def read(p): return json.loads(p.read_text(encoding='utf-8-sig'))
14 def write(n, o): (DATA/'f'{n}.json').write_text(json.dumps(o, ensure_ascii=False, indent=2), encoding='utf-8')
15 BLUE, TEAL, ORANGE, GRAY, RED = '#245B83', '#26756B', '#B46A24', '#64717C', '#A54246'
16 plt.rcParams.update({'font.family': 'sans-serif', 'font.sans-serif': ['SimSun', 'DejaVu Sans'], 'mathtext.fontset': 'stix',
17 'font.size': 9, 'axes.labelsize': 9, 'axes.titlesize': 9.5, 'xtick.labelsize': 8, 'ytick.labelsize': 8,
18 'legend.fontsize': 8, 'axes.linewidth': .65, 'pdf.fonttype': 42, 'ps.fonttype': 42, 'svg.fonttype': 'none', 'axes.unicode_minus': False})
19 def save(f, n):
20     for ext in ['pdf', 'svg', 'png']: f.savefig(FIG/'f'{n}-{ext}', dpi=400)
21     plt.close(f)
22 def clean(ax): ax.spines[['top', 'right']].set_visible(False)
23 def summary(vals):
24     a = np.array(vals); mean = float(a.mean()); sd = float(a.std(ddof=1)); se = sd/np.sqrt(len(a))
25     return dict(n=len(a), mean=mean, sd=sd, ci95=[float(x) for x in t.interval(.95, len(a)-1, loc=mean, scale=se)],
26               median=float(np.median(a)), p90=float(np.quantile(a, .9)))
27 def independent_difference(a, b):
28     a = np.asarray(a); b = np.asarray(b); u = a.var(ddof=1)/len(a); v = b.var(ddof=1)/len(b)
29     df = (u+v)**2/(u*(len(a)-1)+v*(len(b)-1)); d = a.mean()-b.mean()
30     return dict(mean=float(d), ci95=[float(x) for x in t.interval(.95, df, loc=d, scale=np.sqrt(u+v))])
31
32
33 keys = ['cautious', 'm07', 'm32']; names = ['文献适配 CG', '旧版 M07', '本文 M32']; colors = [ORANGE, GRAY, BLUE]
34 local = read(DATA/'paired_rows.json'); pair = read(DATA/'paired_analysis.json')
35 i = next(i for i, r in enumerate(local['m32']) if r['seed'] == pair['seed'])
36 acts = read(DATA/'paired_actions.json')
37 stats = read(DATA/'official_statistics.json'); raw = read(DATA/'official_rows.json')
38 official = {k: r for r in raw if r['strategy'] == k} for k in keys}
39 fig, axs = plt.subplots(2, 2, figsize=(6.3, 4.65)); fig.subplots_adjust(left=.09, right=.98, bottom=.105, top=.93, hspace=.40, wspace=.28)
40 for ax, k, name, color, letter in zip(axs.ravel()[3:], keys, names, colors, ['a', 'b', 'c']):
41     xy = np.array([[0, 0]] + [a['point'] for a in acts[k]])/1000
42     ax.add_patch(Circle((0, 0), 1.8, fill=False, ls='--', lw=.7, color=GRAY)); ax.plot(*xy.T, color=color, lw=.8)
43     stops = np.array([a['point'] for a in acts[k] if a['kind'] == 'clear' and a['success']])/1000
44     ax.scatter(*stops.T, marker='x', s=15, color=color, zorder=4); ax.scatter([0], [0], s=14, facecolors='white', edgecolors=color, zorder=5)
45     r = local[k][i]; ax.set_title(f'({letter}) {name}: {r["time_s"]:.1f} s', fontsize=9)
46     ax.set(xlim=(-2.1, 2.1), ylim=(-2.1, 2.1), xticks=[-1.8, 0, 1.8], yticks=[-1.8, 0, 1.8], xlabel='$x$ / km', ylabel='$y$ / km'); ax.set_aspect('equal'); clean(ax)
47     ax = axs[1, 1]
48     for k, name, color, ls in zip(keys, names, colors, ['-', '--', '-']):
49         times = [0] + [a['time_s']/60 for a in acts[k] if a['kind'] == 'clear' and a['success']]
50         values = list(range(len(times))); times.append(local[k][i]['time_s']/60); values.append(values[-1])
51         ax.step(times, values, where='post', color=color, ls=ls, label=name, lw=1.3)
52         ax.scatter(times[-1], values[-1], marker='s', s=13, color=color, zorder=4)
53         ax.set(xlabel='累计虚拟时间 / min', ylabel='累计成功清除数', title='(d) 相同场景的清除进度', ylim=(0, 10.6)); ax.legend(loc='upper
54             ↪ left', fontsize=7.5, frameon=False); clean(ax)
55     save(fig, 'fig04-paired-paths')
56 fig, axs = plt.subplots(1, 2, figsize=(6.3, 3.1), gridspec_kw={'width_ratios': [1.05, 1]}); fig.subplots_adjust(left=.11, right=.98, bottom=.23, top=.87, wspace=.35)
57 ax = axs[0]
58 for j, k in enumerate(keys):
59     vals = [r['time_per_actual_target_s'] for r in official[k]]; s = stats[k]; lo, hi = s['ci95']
60     ax.scatter(j + .2*np.sin(np.arange(30)*2.399963), vals, s=12, c=colors[j], alpha=.65)
61     ax.errorbar(j, s['mean'], yerr=[[s['mean']-lo], [hi-s['mean']]], fmt='D', color='black', capsize=3, ms=4, zorder=5)
62     ax.set(xticks=range(3), xticklabels=['CG', 'M07', 'M32'], ylabel='单场平均时间 / (s · 源$^{-1}$)', title='(a) 各 30
63         ↪ 场独立官方演练'); ax.grid(axis='y', alpha=.15); clean(ax)
64     ax.text(.5, -.25, '散点: 单场; 菱形: 均值及 95% t 区间', ha='center', transform=ax.transAxes, fontsize=7.5)
65     ax = axs[1]; left = np.zeros(3)
66     for i, (label, col, hatch) in enumerate(zip(['移动', '测量', '换频', '光学'], [BLUE, TEAL, ORANGE, GRAY], ['/', '/', 'xx', '..')):
67         vals = np.array([stats[k]['costs'][i] for k in
68             ↪ keys]); ax.barh(range(3), vals, left=left, height=.55, label=label, color=col, hatch=hatch, edgecolor='white', lw=.5); left += vals
69     ax.set(yticks=range(3), yticklabels=['CG', 'M07', 'M32'], xlabel='场均分项目时间 / (s · 源$^{-1}$)', title='(b) 时间差来自哪些费用'); ax.invert_yaxis(); clean(ax)
70     ax.legend(loc='upper center', bbox_to_anchor=(.5, -.24), ncol=4, frameon=False, fontsize=7, columnspacing=.5, handlelength=1)
71     save(fig, 'fig03-official-comparison')

```

```

69 """Rebuild Q4 figures from frozen official logs. No simulator is imported or run."""
70 from pathlib import Path
71 import json, hashlib, csv
72 import numpy as np
73 import matplotlib
74 matplotlib.use('Agg')
75 import matplotlib.pyplot as plt
76 from matplotlib.patches import Circle, Polygon as PatchPolygon, Wedge
77 from shapely import wkt
78 from shapely.geometry import box, Point, MultiPoint
79 from shapely.ops import unary_union
80
81 BASE=Path(__file__).resolve().parent
82 OUT=BASE/'figures'
83 DATA=BASE/'evidence'
84 OUT.mkdir(parents=True,exist_ok=True); DATA.mkdir(exist_ok=True)
85 BLUE,TEAL,ORANGE,GRAY,RED='#245B83','#26756B','#B46A24','#64717C','#A54246'
86 plt.rcParams.update({'font.family':'sans-serif','font.sans-serif':['SimSun','DejaVu Sans'],
87 'mathtext.fontset':'stix','font.size':9,'axes.labelsize':9,'axes.titlesize':9.5,
88 'xtick.labelsize':8,'ytick.labelsize':8,'legend.fontsize':8,'axes.linewidth':.65,
89 'pdf.fonttype':42,'ps.fonttype':42,'svg.fonttype':'none','axes.unicode_minus':False,
90 'figure.facecolor':'white','lines.linewidth':1.1})
91 def read(p):return json.loads(p.read_text(encoding='utf-8-sig'))
92 def save(fig,name):
93     for ext in ('pdf','svg','png'):fig.savefig(OUT/f'{name}.{ext}',dpi=400)
94     plt.close(fig)
95 def style(ax,equal=False):
96     ax.spines[['top','right']].set_visible(False)
97     if equal:ax.set_aspect('equal',adjustable='box')
98 def poly(ax,g,**kw):
99     if g.geom_type=='Polygon':ax.add_patch(PatchPolygon(np.asarray(g.exterior.coords,**kw)))
100 elif hasattr(g,'geoms'):
101     for sub in g.geoms:poly(ax,sub,**kw)
102
103
104
105 example=read(DATA/'joint_example.json');events=read(DATA/'joint_event_metrics.json')
106 reductions=[e for e in events if e['new_radius']<e['old_radius']-1e-6]
107 # Figure 1: schematic, no hidden official coordinates.
108 fig,axs=plt.subplots(1,2,figsize=(6.3,2.8));fig.subplots_adjust(left=.06,right=.98,bottom=.12,top=.85,wspace=.32)
109 ax=axs[0];ax.add_patch(Wedge((0,0),1.1,-90,90,facecolor=BLUE,alpha=.12));ax.add_patch(Circle((0,0),1.1,fill=False,color=GRAY,ls='--'))
110 ax.scatter([0],[0],marker='*',s=85,c=BLUE,zorder=3);ax.annotate('$x$',(0,0),xytext=(5,-17),textcoords='offset points')
111 ax.arrow(0,0,6,0,width=.012,head_width=.06,color=BLUE);ax.text(.35,12,'$n$')
112 for q,label,color,marker in [( (.58,46),'收到信号',TEAL,'o'),( (-.65,25),'背向: 无信号',RED,'x'),( (1.25,-.35),'超距: 无信号',ORANGE,'x')]:
113     ax.scatter(*q,color=color,marker=marker,s=32);ax.annotate(label,q,xytext=(0,10),textcoords='offset points',ha='center',fontsize=8)
114 ax.set(xlim=(-1.35,1.65),ylim=(-1.25,1.25),title='(a) 无信号对应两种原因');ax.axis('off');ax.set_aspect('equal')
115 ax=axs[1];ss=np.array([[-.9,-.55],[.9,-.5],[.15,9]]);ax.add_patch(PatchPolygon(ss,facecolor=TEAL,alpha=.1,edgecolor=TEAL))
116 ax.scatter(*ss.T,c=TEAL,marker='s',s=35);ax.scatter(0,0,c=BLUE,marker='*',s=85)
117 for i,s in enumerate(ss):ax.text*(s+np.array([.02,.08])),f'$s_{i+1}$')
118 ax.plot([-1.1,1.1],[0,0],ls='--',c=GRAY);ax.arrow(0,0,.55,width=.01,head_width=.055,color=BLUE)
119 ax.text(.08,.32,'$n$');ax.text(-.13,-.17,'$x$');ax.text(0,-.9,'凸组合的有向投影之和为零',ha='center',fontsize=8)
120 ax.set(xlim=(-1.3,1.3),ylim=(-1.1,1.15),title='(b) 凸包覆盖保证至少一站位于前向');ax.axis('off');ax.set_aspect('equal')
121 save(fig,'fig01-direction-coverage')
122
123 # Figure 3: actual posterior domain, selected without maximising improvement.
124 old=wkt.loads(example['old_region']);new=wkt.loads(example['new_region']);removed=old.difference(new)
125 fig,axs=plt.subplots(1,2,figsize=(6.3,3.3));fig.subplots_adjust(left=.12,right=.98,bottom=.24,top=.86,wspace=.38)
126 ax=axs[0];poly(ax,old,facecolor=BLUE,alpha=.13,edgecolor=BLUE,lw=1,label='原位置域');poly(ax,removed,facecolor=ORANGE,alpha=.5,edgecolor=ORANGE,lw=.6,label='经证书排除');poly(ax,new,facecolor='none',edgecolor=TEAL,lw=1.3,label='保留位置域')
127 x0,y0,x1,y1=old.bounds;pad=max(x1-x0,y1-y0)*.13;ax.set(xlim=(x0-pad,x1+pad),ylim=(y0-pad,y1+pad),xlabel='$x$ / m',ylabel='$y$ / m',title='(a) 实测历史产生的联合裁剪');style(ax,True);ax.ticklabel_format(useOffset=False,style='plain');ax.locator_params(axis='both',nbins=4)
128 fig.legend(*ax.get_legend_handles_labels(),loc='lower center',bbox_to_anchor=(.5,.005),frameon=False,ncol=3,fontsize=8)
129 ax=axs[1];rr=np.array([e['new_radius']/e['old_radius'] for e in events]);rr.sort();ax.plot(np.arange(1,len(rr)+1),rr,'o',ms=2.8,c=BLUE);ax.axhline(1,color=GRAY,ls='--',lw=.8)
130 ax.set(xlabel='裁剪事件 (按半径比排序)',ylabel=r'$\rho_{\rm new}/\rho_{\rm old}$',ylim=(0,1.08),title=f'(b) 全部 {len(events)} 次裁剪的半径变化');ax.text(.08,.18,f'{len(reductions)} 次半径下降\n其余事件仅缩小或改变位置域',transform=ax.transAxes,fontsize=8);style(ax)
131 save(fig,'fig02-joint-domain')
132
133
134 print('All four figures redrawn from evidence only; no API or simulator calls.')
```

源文件组 31

q4/baselines/CG/adaptor_check.py

q4/baselines/M07/adaptor_check.py

```

1  """Offline interface-shape check using only the supplied public Robot.
2
3  Run through the sole shared run_local.py factory option. No physics copy/HTTP.
4  """
5  from strategy import Planner
6
7  class TupleRobot:
8      def __init__(self,robot):self.__robot=robot
9      @property
10     def position(self):return tuple(float(x) for x in self.__robot.position)
11     @property
12     def channel(self):return self.__robot.channel
13     @property
14     def cleared(self):return self.__robot.cleared
15     def record(self,value):self.__robot.record(value)
16     def measure(self,point,channel):
17         z=self.__robot.measure(point,channel)
18         return dict(z,accepted=True,request_id='offline-public-interface-check')
19     def clear(self,point,channel):return self.__robot.clear(point,channel)
20
21 def make(robot,policy='both'):
22     return Planner(TupleRobot(robot),policy=policy)

```

源文件组 32

q4/baselines/CG/base.py

q4/baselines/M07/base.py

q4/M32/base.py

```

1  """Fresh Q4 prototypes. No imports from any previous Q4 strategy.
2
3  Only Q1 geometry is reused. Physics hides truth behind the Robot facade.
4  All strategies share a proved discovery mesh and a finite optical fallback.
5  """
6  from pathlib import Path
7  import sys, math, json, argparse, time, hashlib, platform
8  import numpy as np
9  from shapely.geometry import Point, Polygon
10 from shapely.ops import unary_union
11 from scipy.spatial import Delaunay
12 MODEL = Path(__file__).resolve().parents[1]
13 ROOT = MODEL
14 from bearing_geometry import outer_disk, clip_halfplanes, wedge, enclosing_circle, diameter, unit
15 EPS = 1.0051
16 SPACING = 980.0
17 NAMES = {'staged': 'A 先搜后清', 'interleave': 'B 边搜边清', 'heading': 'C 朝向情景选点', 'optical': 'D 朝向选点加光学捷径', 'ring': 'E 环带围合加朝向选点'}
18
19 def save(path, data):
20     path.parent.mkdir(parents=True, exist_ok=True)
21     tmp = path.with_suffix('.tmp')
22     tmp.write_text(json.dumps(data, ensure_ascii=False, indent=2, allow_nan=False), encoding='utf-8')
23     tmp.replace(path)
24
25 def mesh():
26     mapping = {}
27     faces = []
28     for i in range(-5, 5):
29         for j in range(-5, 5):
30             for ids in [[[i, j], (i + 1, j), (i, j + 1)], [(i + 1, j), (i + 1, j + 1), (i, j + 1)]]:
31                 vs = np.array([(SPACING * (x + 0.5 * y), SPACING * math.sqrt(3) / 2 * y) for (x, y) in ids])
32                 if Polygon(vs).distance(Point(0, 0)) <= 1800.0:

```

```

33         faces.append(ids)
34         for (key, v) in zip(ids, vs):
35             mapping[key] = v
36     keys = sorted(mapping)
37     pts = np.array([mapping[k] for k in keys])
38     idx = {k: i for (i, k) in enumerate(keys)}
39     tri = [[idx[k] for k in face] for face in faces]
40     return (pts, tri)
41 (SITES, FACES) = mesh()
42
43 class Track:
44
45     def __init__(self, ch):
46         self.channel = ch
47         self.region = None
48         self.history = []
49         self.positive = []
50         self.center = None
51         self.radius = np.inf
52         self.poly = None
53
54     def refresh(self):
55         if self.region.is_empty():
56             raise ArithmeticError('empty outer position set')
57         self.poly = np.array(self.region.convex_hull.exterior.coords[:-1])
58         (self.center, self.radius, _) = enclosing_circle(self.poly)
59
60     def update(self, q, z):
61         self.history.append(dict(point=np.asarray(q).tolist(), **z))
62         if z['measure_result'] == 'no_signal':
63             return
64         self.positive.append(self.history[-1])
65         rad = 5.0 if z['measure_result'] == 'near' else 1500.0
66         new = outer_disk(q, rad, 144 if rad > 5 else 32)
67         if z['measure_result'] == 'direction':
68             new = clip_halfplanes(new, *wedge(q, z['svd_deg'], EPS))
69         g = Polygon(new).intersection(Polygon(outer_disk([0, 0], 1800, 144)))
70         self.region = g if self.region is None else self.region.intersection(g)
71         self.refresh()
72
73     def exclude(self, q):
74         self.region = self.region.difference(Point(*q).buffer(19.999, quad_segs=24))
75         self.refresh()
76
77     def samples(track):
78         p = track.poly
79         c = np.array(track.region.representative_point().coords[0])
80         (_, (i, j)) = diameter(p)
81         candidates = [c, *0.9 * ((1 - f) * p[i] + f * p[j]) + 0.1 * c for f in np.linspace(0, 1, 9)]
82         candidates += [0.95 * x + 0.05 * c for x in p]
83         keep = []
84         for x in candidates:
85             if track.region.covers(Point(*x)) and all((np.linalg.norm(x - y) > 0.0001 for y in keep)):
86                 keep.append(x)
87         return np.array(keep[:16])
88
89     def heading_hypotheses(track):
90         g = samples(track)
91         positions = np.repeat(g, 3 * 25, axis=0)
92         radii = np.tile(np.repeat([1000.0, 1250.0, 1500.0], 25), len(g))
93         normals = np.tile(np.vstack([np.zeros(2), np.c_[np.cos(np.arange(24) * np.pi / 12), np.sin(np.arange(24) * np.pi / 12)]]), (len(g) * 3, 1))
94         good = np.ones(len(positions), dtype=bool)
95         for h in track.history:
96             delta = np.asarray(h['point']) - positions
97             d = np.linalg.norm(delta, axis=1)
98             visible = (delta * normals).sum(axis=1) >= -1e-08
99             heard = (d <= radii + 1e-06) & visible
100             if h['measure_result'] == 'no_signal':
101                 good &= ~heard
102             else:
103                 good &= heard

```

```

104         if h['measure_result'] == 'near':
105             good &= d <= 5.001
106         else:
107             good &= d > 4.999
108     return (positions[good], radii[good], normals[good])
109
110 class Planner:
111
112     def __init__(self, robot, variant, sites=None):
113         self.sites = SITES if sites is None else np.asarray(sites, float)
114         self.robot = robot
115         self.variant = variant
116         self.tracks = {}
117         self.negatives = {ch: [] for ch in range(1, 21)}
118         self.visited = set()
119         self.scan_history = []
120         self.hyp_empty = 0
121         self.fallbacks = 0
122         self.lost = 0
123
124     def observe(self, q, ch):
125         z = self.robot.measure(q, ch)
126         if ch not in self.tracks and z['measure_result'] != 'no_signal':
127             t = Track(ch)
128             t.history = list(self.negatives[ch])
129             self.tracks[ch] = t
130         if ch in self.tracks:
131             self.tracks[ch].update(q, z)
132             t = self.tracks[ch]
133             self.robot.record(dict(kind='region', channel=ch, wkt=t.region.wkt, radius_m=t.radius))
134         elif z['measure_result'] == 'no_signal':
135             self.negatives[ch].append(dict(point=np.asarray(q).tolist(), **z))
136         return z
137
138     def try_clear(self, t, q, reason):
139         self.robot.record(dict(kind='clear_reason', channel=t.channel, reason=reason, radius_m=t.radius))
140         ok = self.robot.clear(q, t.channel)
141         if not ok:
142             t.exclude(q)
143         return ok
144
145     def candidates(self, t):
146         p = self.robot.position
147         c = t.center
148         (_, (i, j)) = diameter(t.poly)
149         u = t.poly[j] - t.poly[i]
150         u = u / max(np.linalg.norm(u), 1e-12)
151         v = np.array([-u[1], u[0]])
152         off = max(25.0, min(220.0, t.radius * 0.5))
153         raw = [c + off * v, c - off * v, c]
154         for f in [0.4, 0.7]:
155             mid = p + f * (c - p)
156             raw += [mid, mid + 0.6 * off * v, mid - 0.6 * off * v]
157         raw += [c + 0.25 * off * u, c - 0.25 * off * u]
158         return [q for q in raw if all((math.dist(q, h['point']) > 2 for h in t.history))]
159
160     def select(self, t):
161         candidates = self.candidates(t)
162         if not candidates:
163             return None
164         p = self.robot.position
165         if self.variant in ['staged', 'interleave']:
166             q = min(candidates[:2], key=lambda q: np.linalg.norm(q - p))
167             return q
168         (g, rad, n) = heading_hypotheses(t)
169         if not len(g):
170             self.hyp_empty += 1
171             return min(candidates[:2], key=lambda q: np.linalg.norm(q - p))
172         scores = []
173         for q in candidates:
174             delta = q - g

```

```

175     visible = (np.linalg.norm(delta, axis=1) <= rad) & ((delta * n).sum(axis=1) >= -1e-08)
176     prob = float(visible.mean())
177     vg = g[visible]
178     if len(vg):
179         (ug, counts) = np.unique(np.round(vg, 8), axis=0, return_counts=True)
180         sizes = []
181         for x in ug:
182             d = np.linalg.norm(x - q)
183             if d <= 5:
184                 sizes.append(5.0)
185                 continue
186             angle = math.degrees(math.atan2(x[1] - q[1], x[0] - q[0]))
187             pp = clip_halfplanes(t.poly, *wedge(q, angle, EPS))
188             sizes.append(enclosing_circle(pp)[1] if len(pp) else t.radius)
189         after = float(np.average(sizes, weights=counts))
190     else:
191         after = t.radius
192     cost = np.linalg.norm(q - p) / 5 + 5 + max(0, np.linalg.norm(q - t.center) - 19) / 5
193     cost += prob * (after / 5 + 8 * max(0, math.log(max(after / 19, 1), 2)))
194     cost += (1 - prob) * (35 + min(t.radius, 350) / 2.5)
195     scores.append(float(cost))
196     best = int(np.argmin(scores))
197     self.robot.record(dict(kind='scenario_choice', channel=t.channel, scenarios=len(g), candidates=[dict(point=q.tolist(), cost_s=sc) for (q, sc) in
↪ zip(candidates, scores)], selected=best))
198     return candidates[best]
199
200 def optical_cover(self, t):
201     self.fallbacks += 1
202     step = 25.0
203     (xmin, ymin, xmax, ymax) = t.region.bounds
204     nodes = []
205     for ix in range(math.floor(xmin / step), math.floor(xmax / step) + 1):
206         for iy in range(math.floor(ymin / step), math.floor(ymax / step) + 1):
207             square = Polygon([(ix * step, iy * step), ((ix + 1) * step, iy * step), ((ix + 1) * step, (iy + 1) * step), (ix * step, (iy + 1) * step)])
208             if square.intersects(t.region):
209                 nodes.append(np.array([(ix + 0.5) * step, (iy + 0.5) * step]))
210     while nodes:
211         p = self.robot.position
212         i = min(range(len(nodes)), key=lambda i: np.linalg.norm(nodes[i] - p))
213         q = nodes.pop(i)
214         if not t.region.intersects(Point(*q).buffer(20.01)):
215             continue
216         if self.try_clear(t, q, 'finite optical cover'):
217             return
218     raise ArithmeticError('exhausted continuous optical covering without success')
219
220 def solve(self, ch):
221     t = self.tracks[ch]
222     tries = 0
223     for k in range(10):
224         if t.radius <= 19.5:
225             if not self.try_clear(t, t.center, 'enclosing circle certificate'):
226                 raise ArithmeticError('certified clear failed')
227             return
228         if self.variant == 'optical' and t.radius <= 85 and (tries < 2):
229             tries += 1
230             q = np.array(t.region.representative_point().coords[0])
231             if self.try_clear(t, q, 'optical shortcut'):
232                 return
233             continue
234         q = self.select(t)
235         if q is None:
236             break
237         z = self.observe(q, ch)
238         if z['measure_result'] == 'no_signal':
239             self.lost += 1
240     self.optical_cover(t)
241
242 def run(self):
243     remaining = set(range(len(self.sites)))
244     while remaining:

```

```

245     p = self.robot.position
246     i = min(remaining, key=lambda i: (np.linalg.norm(self.sites[i] - p), i))
247     remaining.remove(i)
248     q = self.sites[i]
249     to_scan = [ch for ch in range(1, 21) if ch not in self.robot.cleared and (ch not in self.tracks or (self.variant == 'staged' and self.tracks[ch].radius >
        ↳ 19.5))]
250     for ch in to_scan:
251         self.observe(q, ch)
252         self.visited.add(i)
253         self.scan_history.append(dict(site=i, point=q.tolist(), channels=to_scan))
254         self.robot.record(dict(kind='scan_site', site=i, channels=to_scan))
255         if self.variant != 'staged':
256             while any((ch not in self.robot.cleared for ch in self.tracks)):
257                 ch = min((ch for ch in self.tracks if ch not in self.robot.cleared), key=lambda ch: np.linalg.norm(self.tracks[ch].center - self.robot.position))
258                 self.solve(ch)
259             if len(self.robot.cleared) == 16:
260                 return 'count upper bound'
261         while any((ch not in self.robot.cleared for ch in self.tracks)):
262             ch = min((ch for ch in self.tracks if ch not in self.robot.cleared), key=lambda ch: np.linalg.norm(self.tracks[ch].center - self.robot.position))
263             self.solve(ch)
264         for ch in range(1, 21):
265             if ch not in self.tracks:
266                 assert len(self.negatives[ch]) == len(self.sites)
267         return 'mesh certificate and all discovered cleared'

```

源文件组 33

q4/baselines/CG/cautious.py

```

1  """Literature-inspired bounded-region adaptation, not a reproduction of the EKF paper.
2
3  Vander Hook, Tokekar & Isler (2014), Algorithm 1 / equation (9).
4  Uniform convex-region moments replace the Gaussian EKF estimate. The same M07
5  search/route/clearance scaffold prevents a missing search guarantee from making
6  the comparator artificially incomplete. No direction/radius posterior or joint
7  negative-observation clipping is used. beta=0.05 is fixed before any evaluation.
8  """
9  import math
10 import numpy as np
11 from scipy.stats import norm
12 from m07 import Planner as Parent
13
14 def moments(poly):
15     """Exact uniform-area first/second moments of a convex polygon by triangles."""
16     poly=np.asarray(poly,float);anchor=poly.mean(axis=0)
17     weight=0.;first=np.zeros(2);second=np.zeros((2,2))
18     for a,b in zip(poly,np.roll(poly,-1,axis=0)):
19         tri=np.array([anchor,a,b]);area=abs(np.linalg.det(np.stack([a-anchor,b-anchor])))/2
20         total=tri.sum(axis=0);first+=area*total/3
21         second+=area*(tri.T@tri+np.outer(total,total))/12;weight+=area
22     if weight<=1e-12:return anchor,np.zeros((2,2))
23     mean=first/weight;cov=second/weight-np.outer(mean,mean)
24     return mean,(cov+cov.T)/2
25
26 class Planner(Parent):
27     def __init__(self,robot,policy='cautious'):
28         if policy!='cautious':raise ValueError(policy)
29         super().__init__(robot,'portfolio')
30         self.beta=.05;self.stop_reason=None
31
32     def select(self,t):
33         center,cov=moments(t.poly);values,vectors=np.linalg.eigh(cov)
34         major=math.sqrt(max(float(values[-1]),0));v=vectors[:,0]
35         sigma_beta=math.pi/(2*norm.ppf(1-self.beta/2))
36         sigma_s=math.radians(1)/math.sqrt(3)
37         radius=major/math.sqrt(sigma_beta**2-sigma_s**2)
38         pair=[center+radius*v,center-radius*v]
39         fresh=lambda q:all(math.dist(q,h['point'])>2 for h in t.history)
40         options=[q for q in pair if fresh(q)]

```

```

41     recovery=False
42     if not options:
43         # A no-signal leaves the bounded region unchanged. Rotate the same
44         # cautious radius only after both preferred locations were measured;
45         # never average repeated same-position measurements as new noise.
46         recovery=True
47         for k in range(1,6):
48             theta=k*math.pi/6;rot=np.array([[math.cos(theta),-math.sin(theta)],[math.sin(theta),math.cos(theta)]])
49             d=rot@v;options=[q for q in [center+radius*d,center-radius*d] if fresh(q)]
50             if options:break
51     if not options:return None
52     q=min(options,key=lambda q:math.dist(q,self.robot.position))
53     self.robot.record(dict(kind='literature_cautious_choice',channel=t.channel,center=center.tolist(),
54         covariance=cov.tolist(),radius=radius,beta=self.beta,angular_recovery=recovery,
55         candidates=[x.tolist() for x in options],selected=q.tolist()))
56     return q
57
58     def run(self):
59         try:self.stop_reason=super().run()
60         finally:self.robot.scanned_sites=len(self.visited)
61         return self.stop_reason
62
63     def certificate(self):
64         return dict(problem=4,reason=self.stop_reason,complete=bool(self.stop_reason),
65             cleared_channels=sorted(self.robot.cleared),observed_but_uncleared=sorted(set(self.tracks)-self.robot.cleared),
66             mesh_points=self.sites.tolist(),visited_sites=sorted(self.visited),
67             negative_counts={str(k):len(v) for k,v in self.negatives.items()},
68             shared_measures=self.shared_measures,optical_fallbacks=self.fallbacks,
69             method='bounded-region adaptation of cautious greedy, fixed beta=.05')

```

源文件组 34

q4/baselines/CG/coverage.py

q4/baselines/M07/coverage.py

q4/M32/coverage.py

```

1  """Continuous position/heading certificate; no simulator or truth access."""
2  import math
3  import numpy as np
4  from scipy.spatial import ConvexHull, QhullError
5  from optimize import RING
6
7  class Coverage:
8      def __init__(self):
9          self.points=np.array(RING['points'],dtype=float)
10         self.corners=np.array([[r['lo'],r['hi']][0],r['lo']][1]],r['hi'],[r['lo']][0],r['hi']][1]]) for r in RING['proof'])
11         self.witness=[list(r['stations']) for r in RING['proof']]
12         self.active=set(range(len(self.points)))
13         self.distances=np.linalg.norm(self.corners[:,None,:]-self.points[None,:,None,:],axis=3).max(axis=2)
14
15     def proposal(self,remove,q=None):
16         """Certify every affected square using any remaining nearby viewpoints."""
17         if q is None:
18             points=self.points;distances=self.distances;active=sorted(self.active-{remove})
19         else:
20             points=np.vstack([self.points,q]);extra=np.linalg.norm(self.corners-q,axis=2).max(axis=1)
21             distances=np.column_stack([self.distances,extra]);active=sorted(self.active-{remove})+[len(self.points)]
22         updates={}
23         for ci,old in enumerate(self.witness):
24             if remove not in old:continue
25             ids=[i for i in active if distances[ci,i]<=999.99]
26             if len(ids)<3:return None
27             try:h=ConvexHull(points[ids])
28             except QhullError:return None
29             if np.max(self.corners[ci]@h.equations[:,2].T+h.equations[:,2])>1e-8:return None
30             updates[ci]=[ids[i] for i in h.vertices]
31         return updates
32

```

```

33     def commit(self,remove,q,updates):
34         self.active.remove(remove)
35         if q is not None:
36             self.active.add(len(self.points));self.points=np.vstack([self.points,q])
37             extra=np.linalg.norm(self.corners-q,axis=2).max(axis=1)
38             self.distances=np.column_stack([self.distances,extra])
39             for ci,ids in updates.items():self.witness[ci]=ids
40
41     def with_points(self,qs):
42         clone=Coverage.__new__(Coverage)
43         clone.corners=self.corners;clone.witness=self.witness.copy()
44         clone.points=np.vstack([self.points,qs])
45         clone.active=self.active;set(range(len(self.points),len(clone.points)))
46         extra=np.linalg.norm(self.corners[:,None,:]-np.asarray(qs)[None,:,None,:],axis=3).max(axis=2)
47         clone.distances=np.column_stack([self.distances,extra])
48         return clone
49
50     def certificate(self):
51         return dict(points=self.points.tolist(),active=sorted(self.active),witness=self.witness)

```

源文件组 35

q4/baselines/CG/m06.py

q4/baselines/M07/m06.py

q4/M32/m06.py

```

1  """M06: certified dynamic discovery coverage and service-cost routing."""
2  import math
3  import numpy as np
4  import base
5  from previous import Planner as Previous
6  from coverage import Coverage
7  from optimize import open_route,length
8
9  class Planner(Previous):
10     def __init__(self,robot,policy='coverage'):
11         if policy not in ['baseline','coverage','service','combined','pool','pool_service','count','count_service','area_service']:raise ValueError(policy)
12         super().__init__(robot,'route_opt')
13         self.new_policy=policy;self.coverage=Coverage() if policy in ['coverage','combined','pool','pool_service'] else None
14         self.service=policy in ['service','combined','pool_service','count_service','area_service'];self.first_choice={};self.service_choices={}
15         self.next_waypoint=None
16         self.replacements=0;self.removed=0
17
18     def select(self,t):
19         if t.channel in self.first_choice:return self.first_choice.pop(t.channel)
20         return super().select(t)
21
22     def try_clear(self,t,q,reason):
23         if self.new_policy=='area_service' and reason=='enclosing circle certificate':
24             # Inner disk of the exact clearance region, with 0.5m safety margin.
25             p=self.robot.position;c=t.center;r=max(0,19.5-t.radius)
26             d=math.dist(p,c);near=c+(p-c)*min(1,r/max(d,1e-12))
27             options=[c,near]+[c+r*np.array([math.cos(a),math.sin(a)]) for a in np.arange(32)*2*math.pi/32]
28             def cost(x):return math.dist(p,x)+(math.dist(x,self.next_waypoint) if self.next_waypoint is not None else 0)
29             q=min(options,key=cost)
30             assert np.linalg.norm(t.poly-q,axis=1).max()<=19.501
31             self.robot.record(dict(kind='service_region_clear',channel=t.channel,center=c.tolist(),radius=t.radius,point=np.asarray(q).tolist(),next=np.asarray(s_j
↵         self.next_waypoint).tolist() if self.next_waypoint is not None else
↵         None))
32         return super().try_clear(t,q,reason)
33
34     def service_options(self,t):
35         if t.radius<=19.5:return [(t.center,5.)]
36         qs=self.candidates(t)
37         if not qs:return [(t.center,5.+t.radius/5)]
38         g,rad,n=base.heading_hypotheses(t)
39         out=[]
40         for q in qs:

```

```

41     if not len(g):after=t.radius;prob=0.
42     else:
43         d=q-g;vis=(np.linalg.norm(d,axis=1)<=rad)&((d*n).sum(axis=1)>=-1e-8)
44         prob=float(vis.mean());vg=g[vis];sizes=[];weights=[]
45         if len(vg):
46             ug,counts=np.unique(np.round(vg,8),axis=0,return_counts=True)
47             for x,count in zip(ug,counts):
48                 if math.dist(x,q)<=5:r=5.
49                 else:
50                     a=math.degrees(math.atan2(x[1]-q[1],x[0]-q[0]))
51                     pp=base.clip_halfplanes(t.poly,*base.wedge(q,a,base.EPS))
52                     r=base.enclosing_circle(pp)[1] if len(pp) else t.radius
53                     sizes.append(r);weights.append(count)
54             after=float(np.average(sizes,weights=weights)) if sizes else t.radius
55             residual=5.+max(0,math.dist(q,t.center)-19)/5
56             residual+=prob*(after/5+8*max(0,math.log(max(after/19,1),2)))
57             residual+=(1-prob)*(35+min(t.radius,350)/2.5)
58             out.append((q,float(residual)))
59     return out
60
61 def joint_order(self,remaining):
62     if not self.service:return super().joint_order(remaining)
63     keys=[('site',i) for i in sorted(remaining)]+[('target',ch) for ch in sorted(self.tracks) if ch not in self.robot.cleared]
64     if not keys:return []
65     exits=[self.robot.position]+[self.sites[k] if kind=='site' else self.tracks[k].center for kind,k in keys]
66     options=[[(self.sites[k],6.*(20-len(self.tracks)))] if kind=='site' else self.service_options(self.tracks[k]) for kind,k in keys]
67     cost=np.zeros((len(exits),len(exits)));choice={}
68     for a,p in enumerate(exits):
69         for b,opts in enumerate(options,1):
70             vals=[math.dist(p,q)/5+r for q,r in opts];j=int(np.argmin(vals));cost[a,b]=vals[j];choice[a,b]=opts[j][0]
71     todo=set(range(1,len(exits)));order=[0]
72     while todo:
73         b=min(todo,key=lambda b:(cost[order[-1],b],b));order.append(b);todo.remove(b)
74     def value(rr):return sum(cost[a,b] for a,b in zip(rr[:-1],rr[1:]))
75     best=value(order)
76     for _ in range(30):
77         winning=None;score=best
78         for i in range(1,len(order)-1):
79             for j in range(i+1,len(order)):
80                 rr=order[:i]+order[i:j+1][::-1]+order[j+1:];v=value(rr)
81                 if v<score-1e-7:score=v;winning=rr
82         if winning is None:break
83         order=winning;best=score
84     first=keys[order[1]-1]
85     if first[0]=='target' and self.tracks[first[1]].radius>19.5:self.first_choice[first[1]]=choice[0,order[1]].copy()
86     self.robot.record(dict(kind='service_route',nodes=[dict(kind=keys[b-1][0],id=keys[b-1][1],entry=choice[a,b].tolist(),exit=np.asarray(exits[b]).tolist()) for
87     ↪ a,b in zip(order[:-1],order[1:]),estimated_s=float(best)))
88     return [keys[b-1] for b in order[1:]]
89
90 def scan(self,q,site=None):
91     channels=[ch for ch in range(1,21) if ch not in self.tracks]
92     channels += [ch for ch,t in sorted(self.tracks.items()) if ch not in self.robot.cleared and self.share_worth(t,q)]
93     for ch in sorted(channels):
94         if ch in self.tracks:self.shared_measures+=1
95         self.observe(q,ch)
96     self.robot.record(dict(kind='scan_site',site=site,point=np.asarray(q).tolist(),channels=sorted(channels)))
97
98 def adapt(self,remaining):
99     if not remaining:return
100     if len(self.tracks)==16 and (self.coverage or self.new_policy in ['count','count_service','area_service']):
101         self.robot.record(dict(kind='skip_search_count_bound',sites=sorted(remaining)))
102         remaining.clear();return
103     if not self.coverage:return
104     if self.new_policy in ['pool','pool_service']:
105         self.pool_adapt(remaining);return
106     # Search all pending stations; certify any replacement before paying for it.
107     p=self.robot.position.copy()
108     # Large savings first, but no unproved station is ever removed.
109     for idx in sorted(remaining,key=lambda i:-math.dist(p,self.sites[i])):
110         updates=self.coverage.proposal(idx)
111         if updates is not None:

```

```

111         self.coverage.commit(idx,None,updates);remaining.remove(idx);self.removed+=1
112         self.robot.record(dict(kind='coverage_remove',site=idx,updated_cells=len(updates)))
113     for idx in sorted(remaining,key=lambda i:math.dist(p,self.sites[i])):
114         if math.dist(p,self.sites[idx])<2:continue
115         updates=self.coverage.proposal(idx,p)
116         if updates is None:continue
117         # Same unknown-channel scan count as a station, now at the current pose.
118         self.scan(p);self.coverage.commit(idx,p,updates);remaining.remove(idx);self.replacements+=1
119         self.robot.record(dict(kind='coverage_replace',site=idx,point=p.tolist(),updated_cells=len(updates)))
120     break
121
122     def pool_adapt(self,remaining):
123         inner=[i for i in remaining if i<8]
124         if not inner:return
125         p=self.robot.position.copy()
126         pending=sorted([t.center for ch,t in self.tracks.items() if ch not in self.robot.cleared],key=lambda q:math.dist(p,q))
127         qs=[]
128         for q in [p]+pending[:2]:
129             if all(math.dist(q,self.coverage.points[i])>2 for i in self.coverage.active) and all(math.dist(q,x)>2 for x in qs):qs.append(q)
130         if not qs:return
131         proposed=self.coverage.with_points(qs);removed=[]
132         for idx in sorted(inner,key=lambda i:-math.dist(p,self.sites[i])):
133             updates=proposed.proposal(idx)
134             if updates is not None:proposed.commit(idx,None,updates);removed.append(idx)
135         if not removed:return
136         newids=list(range(len(self.coverage.points),len(proposed.points)))
137         def route_length(ids,points):
138             pts=np.vstack([points[sorted(ids)],pending]) if pending else points[sorted(ids)]
139             rr=open_route(p,set(range(len(pts))),pts,True)
140             return length(p,rr,pts)
141         old=route_length(remaining,self.sites);new=route_length((remaining-set(removed))|set(newids),proposed.points)
142         extra_scans=(len(newids)-len(removed))*(20-len(self.tracks))
143         saving=(old-new)/5-6*extra_scans
144         self.robot.record(dict(kind='pool_proposal',added=len(newids),removed=removed,estimated_saving_s=float(saving),accepted=saving>20))
145         if saving<=20:return
146         self.coverage=proposed;self.sites=proposed.points.copy();remaining.difference_update(removed);remaining.update(newids)
147         self.removed+=len(removed);self.replacements+=len(newids)
148         for i in newids:
149             if math.dist(p,self.sites[i])<1e-7:self.scan(p,i);remaining.remove(i);self.visited.add(i)
150         self.robot.record(dict(kind='pool_commit',removed=removed,added=[dict(id=i,point=self.sites[i].tolist()) for i in newids]))
151
152     def run(self):
153         if self.new_policy=='baseline':return super().run()
154         remaining=set(range(len(self.sites)))
155         while remaining or any(ch not in self.robot.cleared for ch in self.tracks):
156             self.adapt(remaining)
157             order=self.joint_order(remaining)
158             if not order:break
159             self.next_waypoint=None
160             if len(order)>1:
161                 nk,ni=order[1];self.next_waypoint=self.sites[ni] if nk=='site' else self.tracks[ni].center
162                 kind,k=order[0]
163                 if kind=='target':self.solve(k)
164                 else:self.scan(self.sites[k],k);remaining.remove(k);self.visited.add(k)
165                 if len(self.robot.cleared)==16:break
166         self.robot.record(dict(kind='m06_certificate',policy=self.new_policy,coverage=self.coverage.certificate() if self.coverage else
↪ None,visited=sorted(self.visited),replacements=self.replacements,removed=self.removed))
167         return 'continuous coverage and all discovered cleared'

```

源文件组 36

q4/baselines/CG/m07.py

q4/baselines/M07/m07.py

q4/M32/m07.py

```

1  """M07: action substitution, guaranteed-visible recovery, route portfolios."""
2  import math
3  import numpy as np

```

```

4 from shapely.geometry import Point
5 from m06 import Planner as Parent
6
7 class Planner(Parent):
8     def __init__(self, robot, policy='baseline'):
9         if policy not in ['baseline', 'probe65', 'probe85', 'hull', 'portfolio', 'sticky', 'fusion', 'probe_hull', 'sticky_probe']: raise ValueError(policy)
10        super().__init__(robot, 'count'); self.m07=policy
11        self.threshold=.85 if policy=='probe85' else .65
12        self.probe=policy in ['probe65', 'probe85', 'fusion', 'probe_hull', 'sticky_probe']
13        self.recover=policy in ['hull', 'fusion', 'probe_hull']
14        self.multi=policy in ['portfolio', 'sticky', 'fusion', 'sticky_probe']
15        self.previous_order=[]; self.recovery_count={}; self.recovery_candidates=None
16
17    def candidates(self, t):
18        return self.recovery_candidates if self.recovery_candidates is not None else super().candidates(t)
19
20    def select(self, t):
21        if not self.recover or t.history[-1]['measure_result']!='no_signal' or self.recovery_count.get(t.channel, 0) >= 2: return super().select(t)
22        pts=np.array([h['point'] for h in t.positive])
23        if len(pts)<2: return super().select(t)
24        dm=np.linalg.norm(pts[:, None, :] - pts[None, :, :], axis=2); i, j=np.unravel_index(np.argmax(dm), dm.shape)
25        raw=[pts.mean(axis=0)] + [(1-f)*pts[i] + f*pts[j] for f in [.25, .5, .75]] + [(pts[-1] + pts[-2])/2]
26        qs=[]
27        for q in raw:
28            if all(math.dist(q, h['point']) > 2 for h in t.history) and all(math.dist(q, x) > 1e-6 for x in qs): qs.append(q)
29        if not qs: return super().select(t)
30        self.recovery_candidates=qs
31        try: q=super().select(t)
32        finally: self.recovery_candidates=None
33        self.recovery_count[t.channel]=self.recovery_count.get(t.channel, 0)+1
34        self.robot.record(dict(kind='visible_hull_recovery', channel=t.channel, positive_points=pts.tolist(), point=q.tolist()))
35        return q
36
37    def solve(self, ch):
38        if not self.probe: return super().solve(ch)
39        t=self.tracks[ch]; probed=False
40        for _ in range(10):
41            if t.radius <= 19.5:
42                if not self.try_clear(t, t.center, 'enclosing circle certificate'): raise ArithmeticError('certified clear failed')
43                return
44            q=self.select(t)
45            if q is None: break
46            if not probed:
47                fraction=t.region.intersection(Point(*q).buffer(19.999, quad_segs=24)).area/max(t.region.area, 1e-12)
48                if fraction >= self.threshold:
49                    probed=True
50                    self.robot.record(dict(kind='pre_radio_probe', channel=ch, point=q.tolist(), fraction=float(fraction), region=t.region.wkt, threshold=self.threshold,
51                                          ↪ d))
52                    if self.try_clear(t, q, 'one optical attempt at planned radio viewpoint'): return
53                    # After failed probe, keep the originally selected paid radio viewpoint.
54                    z=self.observe(q, ch)
55                    if z['measure_result']=='no_signal': self.lost+=1
56                    self.optical_cover(t)
57
58    def joint_order(self, remaining):
59        baseline=super().joint_order(remaining)
60        if not self.multi or len(baseline)<2: return baseline
61        keys=[('site', i) for i in sorted(remaining)] + [('target', ch) for ch in sorted(self.tracks) if ch not in self.robot.cleared]
62        pts=np.vstack([self.robot.position] + [self.sites[k] if kind=='site' else self.tracks[k].center for kind, k in keys])
63        dm=np.linalg.norm(pts[:, None, :] - pts[None, :, :], axis=2); index={key: i+1 for i, key in enumerate(keys)}
64        def cost(order): return float(sum(dm[a, b] for a, b in zip(order[:-1], order[1:])))
65        def insert(order):
66            todo=set(range(1, len(pts))) - set(order)
67            while todo:
68                best=None
69                for k in sorted(todo):
70                    for pos in range(1, len(order)+1):
71                        a=order[pos-1]; delta=dm[a, k]
72                        if pos<len(order): b=order[pos]; delta+=dm[k, b]-dm[a, b]
73                        choice=(delta, k, pos)
74                        if best is None or choice<best: best=choice

```

```

74         __,k,pos=best;order.insert(pos,k);todo.remove(k)
75     return order
76 def improve(order):
77     for _ in range(100):
78         best=-1e-7;change=None
79         for i in range(1,len(order)-1):
80             a,b=order[i-1],order[i]
81             for j in range(i+1,len(order)):
82                 c=order[j];delta=dm[a,c]-dm[a,b]
83                 if j+1<len(order):d=order[j+1];delta+=dm[b,d]-dm[c,d]
84                 if delta<best:best=delta;change=(i,j)
85             if change is None:break
86             i,j=change;order[i:j+1]=order[i:j+1][::-1]
87     return order
88 orders=[[0]+[index[k] for k in baseline]]
89 warm=[0]+[index[k] for k in self.previous_order if k in index]
90 orders.append(improve(insert(warm)))
91 far=int(np.argmax(dm[0,1:]))+1;orders.append(improve(insert([0,far])))
92 costs=[cost(o) for o in orders];chosen=int(np.argmin(costs))
93 if self.m07 in ['sticky','sticky_probe'] and costs[1]<=min(costs)+25:chosen=1
94 answer=[keys[i-1] for i in orders[chosen][1:]];self.previous_order=answer.copy()
95 self.robot.record(dict(kind='routing_portfolio',lengths_m=costs,chosen=chosen,order=[dict(kind=k,id=i) for k,i in answer]))
96 return answer

```

源文件组 37

q4/baselines/CG/m13.py

q4/baselines/M07/m13.py

q4/M32/m13.py

```

1  """Change only target representation used in global route ordering."""
2  import numpy as np
3  from m07 import Planner as Parent
4  from posterior import position_cloud,median
5
6  class Planner(Parent):
7      def __init__(self,robot,policy='baseline'):
8          if policy not in ['baseline','centroid','posterior_mean','posterior_median']:raise ValueError(policy)
9          super().__init__(robot,'portfolio');self.m13=policy;self.representatives={}
10
11      def representative(self,t):
12          signature=(len(t.history),t.region.wkb)
13          old=self.representatives.get(t.channel)
14          if old is not None and old[0]==signature:return old[1]
15          if self.m13=='centroid':p=np.array(t.region.centroid.coords[0]);count=0
16          else:
17              pts,w=position_cloud(t);p=w@pts if self.m13=='posterior_mean' else median(pts,w);count=len(pts)
18          self.representatives[t.channel]=(signature,p)
19          self.robot.record(dict(kind='route_representative',channel=t.channel,method=self.m13,mec_center=t.center.tolist(),representative=p.tolist(),mec_radius=
20              ↪ s=t.radius,points=count))
21          return p
22
23      def joint_order(self,remaining):
24          if self.m13=='baseline':return super().joint_order(remaining)
25          saved={ch:t.center.copy() for ch,t in self.tracks.items() if ch not in self.robot.cleared}
26          # MEC centers are restored before observing, selecting or clearing.
27          try:
28              for ch in saved:self.tracks[ch].center=self.representative(self.tracks[ch])
29          finally:
30              for ch,c in saved.items():self.tracks[ch].center=c

```

源文件组 38

q4/baselines/CG/optimize.py

q4/baselines/M07/optimize.py

q4/M32/optimize.py

```

1  """Round 2: route and search/localization integration; frozen M03 base."""
2  from pathlib import Path
3  import json, math, time, argparse, hashlib, sys, platform
4  import numpy as np
5  import base
6  MODEL = Path(__file__).resolve().parents[1]
7  RING = json.loads(Path(__file__).with_name('ring.json').read_text(encoding='utf8'))
8  SITES = np.array(RING['points'])
9  NAMES = {'E0': '原E最近邻', 'R': '开放路线优化', 'J': '顺路定位', 'RJ': '路线与定位融合', 'JD': '只推迟不顺路补测', 'J100': '顺路定位100m', 'J500':
    ↳ '顺路定位500m', 'K': '可见性检查后顺路定位'}
10
11 def length(p, route, sites):
12     return float(np.linalg.norm(np.diff(np.vstack([p, sites[route]]), axis=0), axis=1).sum()) if route else 0.0
13
14 def open_route(p, ids, sites, opt=True):
15     todo = set(ids)
16     r = []
17     q = p
18     while todo:
19         i = min(todo, key=lambda i: (np.linalg.norm(sites[i] - q), i))
20         r.append(i)
21         todo.remove(i)
22         q = sites[i]
23     if not opt:
24         return r
25     best = length(p, r, sites)
26     for repeat in range(30):
27         improved = False
28         for i in range(len(r) - 1):
29             for j in range(i + 1, len(r)):
30                 rr = r[i] + r[i:j + 1][:-1] + r[j + 1:]
31                 value = length(p, rr, sites)
32                 if value < best - 1e-07:
33                     r = rr
34                     best = value
35                     improved = True
36         if not improved:
37             break
38     return r
39
40 class Integrated(base.Planner):
41
42     def __init__(self, robot, variant):
43         super().__init__(robot, 'heading', SITES.copy())
44         self.name = variant
45         self.deferred = 0
46         self.shared_measures = 0
47
48     def route(self, remaining):
49         return open_route(self.robot.position, remaining, self.sites, self.name in ['R', 'RJ'])
50
51     def share_worth(self, t, q):
52         if t.radius <= 19.5:
53             return False
54         if any((math.dist(q, h['point']) <= 2 for h in t.history)):
55             return False
56         d = math.dist(q, t.center)
57         if d > 1200:
58             return False
59         old = np.array(t.positive[-1]['point'])
60         u = old - t.center
61         v = q - t.center
62         angle = math.degrees(math.acos(float(np.clip(u @ v / max(np.linalg.norm(u), 1e-10), -1, 1))))
63         return angle >= 15 or d < 0.7 * np.linalg.norm(u)
64
65     def future_useful(self, t, route):
66         (g, r, n) = base.heading_hypotheses(t)

```

```

67     if not len(g):
68         return False
69     for i in route[:3]:
70         q = self.sites[i]
71         if not self.share_worth(t, q):
72             continue
73         delta = q - g
74         visible = (np.linalg.norm(delta, axis=1) <= r) & ((delta * n).sum(axis=1) >= -1e-08)
75         if float(visible.mean()) >= 0.7:
76             return True
77     return False
78
79 def run(self):
80     remaining = set(range(len(self.sites)))
81     joint = self.name in ['J', 'R', 'J', 'JD', 'J100', 'J500', 'K']
82     detour_limit = {'J100': 100, 'J500': 500}.get(self.name, 250)
83     while remaining:
84         route = self.route(remaining)
85         i = route[0]
86         q = self.sites[i]
87         remaining.remove(i)
88         channels = [ch for ch in range(1, 21) if ch not in self.tracks]
89         if joint and self.name != 'JD':
90             channels += [ch for (ch, t) in sorted(self.tracks.items()) if ch not in self.robot.cleared and self.share_worth(t, q)]
91         channels = sorted(channels)
92         for ch in channels:
93             if ch in self.tracks:
94                 self.shared_measures += 1
95                 self.observe(q, ch)
96             self.visited.add(i)
97             self.scan_history.append(dict(site=i, point=q.tolist(), channels=channels))
98             self.robot.record(dict(kind='scan_site', site=i, channels=channels))
99             while any((ch not in self.robot.cleared for ch in self.tracks)):
100                 p = self.robot.position
101                 pending = [ch for ch in self.tracks if ch not in self.robot.cleared]
102                 next_route = self.route(remaining)
103                 next_q = self.sites[next_route[0]] if next_route else None
104                 costs = {}
105                 for ch in pending:
106                     t = self.tracks[ch]
107                     d = math.dist(p, t.center)
108                     detour = d if next_q is None else d + math.dist(t.center, next_q) - math.dist(p, next_q)
109                     costs[ch] = (detour, d)
110                 ch = min(pending, key=lambda ch: (costs[ch][0], costs[ch][1], ch)) if joint else min(pending, key=lambda ch: (math.dist(p, self.tracks[ch].center),
111                 ↪ ch))
112                 (detour, d) = costs[ch]
113                 if joint and remaining and (len(self.tracks) < 16) and (detour > detour_limit) and (d > 350):
114                     if self.name != 'K' or self.future_useful(self.tracks[ch], next_route):
115                         self.deferred += len(pending)
116                         self.robot.record(dict(kind='defer', channels=pending, next_site=next_route[0], detour_m=detour, distance_m=d))
117                         break
118                     self.robot.record(dict(kind='defer_rejected', channel=ch, reason='no useful visible station in next three'))
119                 self.solve(ch)
120                 if len(self.robot.cleared) == 16:
121                     return 'count upper bound'
122             while any((ch not in self.robot.cleared for ch in self.tracks)):
123                 ch = min((ch for ch in self.tracks if ch not in self.robot.cleared), key=lambda ch: math.dist(self.robot.position, self.tracks[ch].center))
124                 self.solve(ch)
125             for ch in range(1, 21):
126                 if ch not in self.tracks:
127                     assert len(self.negatives[ch]) == len(self.sites)
128     return 'mesh certificate and all discovered cleared'

```

源文件组 39

q4/baselines/CG/posterior.py

q4/baselines/M07/posterior.py

q4/M32/posterior.py

```

1  """Exact heading/radius prior mass at each position, area quadrature of P.
2
3  Used only for route ranking. Uniform area/R/heading and a half-type mixture
4  are modeling assumptions, not official generator facts or hard certificates.
5  """
6  import math
7  import numpy as np
8  from scipy.stats import qmc
9  from shapely.geometry import Point
10 TAU=2*math.pi
11
12 def intersect_arc(intervals,angle):
13     lo=(angle-math.pi/2)%TAU;hi=lo+math.pi
14     pieces=[(lo,min(hi,TAU))]+([(0,hi-TAU)] if hi>TAU else [])
15     return [(max(a,c),min(b,d)) for a,b in intervals for c,d in pieces if min(b,d)>max(a,c)]
16
17 def prior_mass(x,history):
18     positive=[h for h in history if h['measure_result']!='no_signal']
19     negative=[h for h in history if h['measure_result']=='no_signal']
20     angles=[(0,TAU)];lo=1000.
21     for h in positive:
22         d=np.asarray(h['point'])-x;distance=float(np.linalg.norm(d));lo=max(lo,distance)
23         if h['measure_result']=='near' and distance>5.000001:return 0.
24         if h['measure_result']=='direction' and distance<4.999999:return 0.
25         if distance>1e-8:angles=intersect_arc(angles,math.atan2(d[1],d[0]))
26     if lo>=1500:return 0.
27     negatives=[]
28     for h in negative:
29         d=np.asarray(h['point'])-x;distance=float(np.linalg.norm(d))
30         if distance<1e-8:return 0.
31         negatives.append((distance,math.atan2(d[1],d[0])+math.pi))
32     negatives.sort()
33     omni=max(0,min(1500.,negatives[0][0] if negatives else 1500.)-lo)/500.
34     current=lo;mass=0.
35     for radius,back_angle in negatives:
36         stop=min(radius,1500.)
37         if stop>current:mass+=(stop-current)*sum(b-a for a,b in angles)/TAU;current=stop
38         if radius>=1500:break
39         angles=intersect_arc(angles,back_angle)
40         if not angles:break
41     if angles and current<1500:mass+=(1500-current)*sum(b-a for a,b in angles)/TAU
42     return .5*omni+.5*mass/500.
43
44 def position_cloud(track,power=7):
45     poly=track.poly;c=np.asarray(track.region.convex_hull.centroid.coords[0]);a=poly;b=np.roll(poly,-1,axis=0)
46     area=np.abs((a[:,0]-c[0])*(b[:,1]-c[1])-(a[:,1]-c[1])*(b[:,0]-c[0]))/2
47     if area.sum()<1e-10:return np.array([track.center]),np.ones(1)
48     distribution=np.cumsum(area/area.sum());z=qmc.Sobol(3,scramble=False).random_base2(power)
49     idx=np.minimum(np.searchsorted(distribution,z[:,0],side='right'),len(poly)-1);f=np.sqrt(z[:,1])
50     pts=(1-f[:,None])*c+f[:,None]*((1-z[:,2,None])*a[idx+z[:,2,None]*b[idx]])
51     pts=np.array([p for p in pts if np.linalg.norm(p)<=1800+1e-8 and track.region.covers(Point(*p))])
52     if not len(pts):return np.array([track.center]),np.ones(1)
53     weights=np.array([prior_mass(x,track.history) for x in pts]);keep=weights>0
54     if not keep.any():return np.array([track.region.centroid.coords[0]]),np.ones(1)
55     pts=pts[keep];weights=weights[keep];weights/=weights.sum();return pts,weights
56
57 def median(points,weights):
58     x=weights@points
59     for _ in range(100):
60         d=np.maximum(np.linalg.norm(points-x,axis=1),1e-7);w=weights/d;y=(w@points)/w.sum()
61         if np.linalg.norm(y-x)<1e-5:return y
62     x=y
63     return x

```

源文件组 40

q4/baselines/CG/previous.py

q4/baselines/M07/previous.py

q4/M32/previous.py

```

1  """Candidate policy only; all paid physics comes from shared Robot."""
2  import math,json
3  from pathlib import Path
4  import numpy as np
5  from shapely.geometry import Point,Polygon
6  from scipy.spatial import ConvexHull
7  import base
8  from optimize import Integrated,RING
9
10 class Planner(Integrated):
11     def __init__(self,robot,policy='control'):
12         if policy not in ['control','recovery','optcover','mobile','combined','route','route_opt','greedyopt']:raise ValueError(policy)
13         super().__init__(robot,'J500');self.policy=policy;self.relocations=0
14         self.leaves=[]
15         if policy=='mobile':
16             for leaf in RING['proof']:
17                 lo,hi=leaf['lo'],leaf['hi'];c=np.array([lo,hi[0],lo[1]],hi,hi[0],hi[1]))
18                 self.leaves.append((c,leaf['stations']))
19                 self.checked_moves=set()
20     def candidates(self,t):
21         raw=super().candidates(t)
22         if self.policy not in ['recovery','combined'] or t.history[-1]['measure_result']!='no_signal':return raw
23         last=np.array(t.positive[-1]['point']);c=t.center
24         # Recover from a known positive viewpoint; hypotheses still rank these.
25         for f in [.25,.5,.75]:
26             q=last+f*(c-last)
27             if all(math.dist(q,h['point'])>2 for h in t.history):raw.append(q)
28         return raw
29     def optical_cover(self,t):
30         if self.policy in ['greedyopt','route_opt']:return self.greedy_optical(t)
31         if self.policy not in ['optcover','combined']:return super().optical_cover(t)
32         self.fallbacks+=1
33         poly=t.poly;_,(i,j)=base.diameter(poly);u=poly[j]-poly[i];u=u/max(np.linalg.norm(u),1e-10);v=np.array([-u[1],u[0]]);basis=np.array([u,v])
34         rotated=poly@basis.T;lo=rotated.min(axis=0);hi=rotated.max(axis=0);width=hi[1]-lo[1]
35         # Every rectangle has circumradius <=19.5m; intersecting cells retained.
36         if width<=36:
37             height=max(width,1e-6);step=2*math.sqrt(19.5**2-(height/2)**2)
38         else:height=27.5;step=27.5
39         nx=max(1,math.ceil((hi[0]-lo[0])/step));ny=max(1,math.ceil((hi[1]-lo[1])/height));nodes=[]
40         for ix in range(nx):
41             for iy in range(ny):
42                 a=lo+np.array([ix*step,iy*height]);b=np.minimum(hi,a+[step,height]);corners=np.array([a,[b[0],a[1]],b,[a[0],b[1]]])@basis
43                 if Polygon(corners).intersects(t.region):nodes.append(((a+b)/2@basis,corners))
44         self.robot.record(dict(kind='optical_cover_certificate',channel=t.channel,width_m=float(width),cells=[c.tolist() for q,c in nodes],centers=[q.tolist() for
↵ q,c in nodes]))
45         while nodes:
46             k=min(range(len(nodes)),key=lambda k:math.dist(nodes[k][0],self.robot.position));q,c=nodes.pop(k)
47             if not Polygon(c).intersects(t.region):continue
48             if t.radius<=19.5:
49                 if not self.try_clear(t,t.center,'enclosing circle certificate'):raise ArithmeticError('certified clear failed')
50                 return
51             if self.try_clear(t,q,'oriented rectangle optical cover'):return
52             raise ArithmeticError('exhausted certified optical rectangles')
53     def greedy_optical(self,t):
54         self.fallbacks+=1;step=25.;xmin,ymin,xmax,ymax=t.region.bounds;nodes=[]
55         for ix in range(math.floor(xmin/step),math.floor(xmax/step)+1):
56             for iy in range(math.floor(ymin/step),math.floor(ymax/step)+1):
57                 corners=np.array([(ix*step,iy*step),((ix+1)*step,iy*step),((ix+1)*step,(iy+1)*step),[ix*step,(iy+1)*step]])
58                 if Polygon(corners).intersects(t.region):nodes.append((corners.mean(axis=0),corners))
59         self.robot.record(dict(kind='optical_cover_certificate',channel=t.channel,cells=[c.tolist() for q,c in nodes],centers=[q.tolist() for q,c in nodes]))
60         while nodes:
61             if t.radius<=19.5:
62                 if not self.try_clear(t,t.center,'enclosing circle certificate'):raise ArithmeticError('certified clear failed')
63                 return
64             candidates=sorted(range(len(nodes)),key=lambda i:math.dist(nodes[i][0],self.robot.position))[:48]
65             scores=[]
66             for i in candidates:
67                 q,c=nodes[i];area=t.region.intersection(Point(*q).buffer(19.999,quad_segs=16)).area
68                 scores.append(area/(3+math.dist(q,self.robot.position)/5))

```

```

69         k=candidates[int(np.argmax(scores))];q,c=nodes.pop(k)
70         if not Polygon(c).intersects(t.region):continue
71         if self.try_clear(t,q,'optical area per paid second'):return
72         raise ArithmeticError('exhausted optical covering')
73     def route(self,remaining):
74         route=super().route(remaining)
75         if self.policy!='mobile' or not route:return route
76         idx=route[0];q=self.robot.position
77         key=(idx,round(float(q[0]),3),round(float(q[1]),3))
78         if idx==0 or idx>7 or key in self.checked_moves or not 2<math.dist(q,self.sites[idx])<=250:return route
79         self.checked_moves.add(key);new=self.sites.copy();new[idx]=q
80         for corners,ids in self.leaves:
81             if idx not in ids:continue
82             pts=new[ids]
83             if np.linalg.norm(pts[:,None,:]-corners[None,,:],axis=2).max()>999.99:return route
84             try:h=ConvexHull(pts)
85             except Exception:return route
86             if (corners@h.equations[:,2].T+h.equations[:,2]).max()>1e-8:return route
87         self.robot.record(dict(kind='relocate',site=idx,old=self.sites[idx].tolist(),new=q.tolist()))
88         self.sites=new;self.relocations+=1
89         return route
90     def run(self):
91         reason=self.joint_route_run() if self.policy in ['route','route_opt'] else super().run()
92         self.robot.record(dict(kind='final_certificate',reason=reason,sites=self.sites.tolist(),visited=sorted(self.visited),relocations=self.relocations,policy=self.policy,
93         ↪  icy))
94         return reason
95     def joint_order(self,remaining):
96         keys=[('site',i) for i in sorted(remaining)]+[('target',ch) for ch in sorted(self.tracks) if ch not in self.robot.cleared]
97         pts=[self.sites[k] if kind=='site' else self.tracks[k].center for kind,k in keys]
98         if not pts:return []
99         pp=np.vstack([self.robot.position,pts]);dm=np.linalg.norm(pp[:,None,:]-pp[None,,:],axis=2);todo=set(range(1,len(pp)));order=[0]
100         while todo:
101             k=min(todo,key=lambda k:(dm[order[-1],k],k));order.append(k);todo.remove(k)
102         for _ in range(100):
103             change=None;best=-1e-7
104             for i in range(1,len(order)-1):
105                 a,b=order[i-1],order[i]
106                 for j in range(i+1,len(order)):
107                     c=order[j];delta=dm[a,c]-dm[a,b]
108                     if j+1<len(order):
109                         d=order[j+1];delta+=dm[b,d]-dm[c,d]
110                     if delta<best:best=delta;change=(i,j)
111             if change is None:break
112             i,j=change;order[i:j+1]=order[i:j+1][::-1]
113         self.robot.record(dict(kind='joint_route',nodes=[dict(kind=keys[k-1][0],id=keys[k-1][1],point=pp[k].tolist()) for k in
114         ↪  order[1:]],length_m=float(sum(dm[a,b] for a,b in zip(order[:-1],order[1:]))))
115         return [keys[k-1] for k in order[1:]]
116     def joint_route_run(self):
117         remaining=set(range(len(self.sites)))
118         while remaining or any(ch not in self.robot.cleared for ch in self.tracks):
119             order=self.joint_order(remaining);kind,k=order[0]
120             if kind=='target':self.solve(k)
121             else:
122                 q=self.sites[k];remaining.remove(k)
123                 channels=[ch for ch in range(1,21) if ch not in self.tracks]
124                 channels += [ch for ch,t in sorted(self.tracks.items()) if ch not in self.robot.cleared and self.share_worth(t,q)]
125                 for ch in sorted(channels):
126                     if ch in self.tracks:self.shared_measures+=1
127                     self.observe(q,ch)
128                 self.visited.add(k);self.scan_history.append(dict(site=k,point=q.tolist(),channels=sorted(channels)))
129                 self.robot.record(dict(kind='scan_site',site=k,channels=sorted(channels)))
130             if len(self.robot.cleared)==16:return 'count upper bound'
131         for ch in range(1,21):
132             if ch not in self.tracks:assert len(self.negatives[ch])==len(self.sites)
133         return 'mesh certificate and all discovered cleared'

```

源文件组 41

q4/baselines/CG/research_strategy.py

q4/baselines/M07/research_strategy.py

q4/M32/research_strategy.py

```

1  """Ablate posterior-based routing and posterior-centered radio candidates.
2
3  MEC geometry is restored before any paid action or safety certificate.
4  """
5  from m07 import Planner as M07
6  from m13 import Planner as Parent
7
8  class Planner(Parent):
9      def __init__(self, robot, policy='baseline'):
10         if policy not in ['baseline', 'route', 'local', 'both']: raise ValueError(policy)
11         super().__init__(robot, 'posterior_mean'); self.m16=policy
12
13     def joint_order(self, remaining):
14         if self.m16 in ['baseline', 'local']: return M07.joint_order(self, remaining)
15         return super().joint_order(remaining)
16
17     def select(self, t):
18         if self.m16 not in ['local', 'both']: return super().select(t)
19         representative=self.representative(t); original=t.center.copy()
20         try:
21             t.center=representative
22             q=super().select(t)
23             finally: t.center=original
24         if q is not None: self.robot.record(dict(kind='posterior_radio_candidate', channel=t.channel, mec_center=original.tolist(), representative=representative.t
25             ↪ olist(), point=q.tolist()))
26         return q

```

源文件组 42

q4/baselines/CG/run_robot.py

```

1  """ 问题四混合全向/定向源 M16 后验规划策略；尚无官方实测。 """
2  import argparse
3  import datetime as dt
4  import hashlib
5  import json
6  from pathlib import Path
7  import shutil
8  import sys
9  import time
10 import uuid
11
12 ROOT = Path(__file__).resolve().parent
13
14
15 def main():
16     if hasattr(sys.stdout, 'reconfigure'):
17         sys.stdout.reconfigure(encoding='utf-8')
18     parser = argparse.ArgumentParser(description=__doc__)
19     parser.add_argument('--policy', choices=['cautious'], default='cautious', help='默认第四问 M16 后验规划策略')
20     parser.add_argument('--team', help='默认复用旧策略的 config.json 队号')
21     parser.add_argument('--port', type=int, help='默认复用旧配置，缺省端口 2026')
22     parser.add_argument('--config', type=Path, help='指定含 team_no、port 的配置文件')
23     parser.add_argument('--check', action='store_true', help='仅检查依赖和配置，不连接模拟器')
24     args = parser.parse_args()
25     try:
26         import numpy
27         import scipy
28         import shapely
29         from api_client import Robot, BudgetReached
30         from strategy import Planner, VERSION, POLICIES
31     except ImportError as error:
32         print(f'缺少依赖: {error}\n 请安装 requirements.txt 中的依赖后重试。')
33         return 2
34     paths = [args.config] if args.config else [ROOT / 'config.json', ROOT / 'config.json']

```

```

35 config_path = next((p for p in paths if p and p.is_file()), None)
36 config = json.loads(config_path.read_text(encoding='utf-8-sig')) if config_path else {}
37 team, port = args.team or config.get('team_no'), args.port or config.get('port', 2026)
38 if not team:
39     print('找不到队号配置, 请使用 --config 指定旧 config.json, 或使用 --team 参数。')
40     return 2
41 if args.check:
42     print(f'策略 {args.policy}; 第四问覆盖改进版, 仅完成本地验证。')
43     print(f'自检通过: Python {sys.version.split()[0]} / numpy {numpy.__version__} / scipy {scipy.__version__}')
44     print(f'配置可读; 端口 {port}; 未连接模拟器。')
45     return 0
46 print(f'问题 4 研究策略 {VERSION} / {args.policy} | 端口 {port}')
47 print('这是第四问覆盖改进版研究候选, 尚无官方演练成绩。')
48 print('请在模拟器选择 【问题 4 演练测试】, 开始后等待 “接口已就绪/等待机器狗进入”。')
49 print('接口无法查询演练/正式模式, 程序按问题 4 混合全向/定向干扰器规则工作。')
50 input('界面就绪后按 Enter 开始: ')
51 folder = ROOT / 'runs' / (dt.datetime.now().strftime('%Y%m%d-%H%M%S') + '-' + uuid.uuid4().hex[:6])
52 robot = Robot(team, f'http://127.0.0.1:{port}', folder)
53 planner = Planner(robot, policy=args.policy)
54 snapshot = folder / 'source_snapshot'
55 snapshot.mkdir()
56 hashes = {}
57 for p in list(ROOT.glob('*.*py')) + [p for p in ROOT.glob('*.*json') if p.name != 'config.json']:
58     shutil.copy2(p, snapshot / p.name)
59     hashes[p.name] = hashlib.sha256(p.read_bytes()).hexdigest()
60 (folder / 'manifest.json').write_text(json.dumps(dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy],
61     python=sys.version, numpy=numpy.__version__, scipy=scipy.__version__, shapely=shapely.__version__, source_sha256=hashes,
62     config_path=str(config_path), mode='user_selected_P4_practice_not_API_verified'),
63     ensure_ascii=False, indent=2), encoding='utf-8')
64 outcome, error_text, entered = 'completed', None, False
65 robot.started = time.monotonic()
66 try:
67     robot.call('/enter')
68     entered = True
69     planner.run()
70 except BudgetReached as error:
71     outcome, error_text = 'budget_stop', str(error)
72 except (Exception, KeyboardInterrupt) as error:
73     outcome, error_text = 'error', str(error) or '用户中断'
74     # Interruption during HTTP could leave execution state unknown.
75     if isinstance(error, KeyboardInterrupt):
76         robot.transport_uncertain = True
77 finally:
78     if entered and not robot.transport_uncertain:
79         try:
80             robot.call('/exit')
81         except Exception as error:
82             outcome = 'exit_unconfirmed'
83             error_text = (error_text or '') + '\n退出未确认: ' + str(error)
84 count = len(robot.cleared)
85 summary = dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy], outcome=outcome, error=error_text,
86     exit_confirmed=robot.exit_confirmed, transport_uncertain=robot.transport_uncertain,
87     cleared_count=count, cleared_channels=sorted(robot.cleared),
88     observed_but_uncleared=sorted(robot.discovered - robot.cleared),
89     measure_count=robot.measure_count, clear_failures=robot.clear_failures,
90     scanned_sites=robot.scanned_sites, distance_m=robot.distance_m, switches=robot.switches,
91     virtual_time_s=robot.virtual, average_clear_time_s=robot.virtual / count if count else None,
92     local_elapsed_s=time.monotonic() - robot.started,
93     actual_jammer_count=None, completion=planner.certificate(),
94     note='实际目标总数和正式成绩以模拟器为准; 完整性证书以问题 4 半圆定向/全向与 R ≥ 1000 规则为前提。')
95 (folder / 'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
96 robot.log.close()
97 print(f'\n 结束: 清除 {count} 个; 虚拟时间 {robot.virtual:.1f} 秒; 移动 {robot.distance_m:.1f} 米。')
98 print(f'状态 {outcome}; 退出确认 {robot.exit_confirmed}; 结束依据 {planner.stop_reason}')
99 if error_text:
100     print(error_text)
101 print(f'运行记录: {folder}')
102 print('请保存模拟器结束界面、案例编码、实际目标数, 并导出官方 .jlog。')
103 return 0 if outcome == 'completed' and robot.exit_confirmed else 1
104
105

```

```

106 if __name__ == '__main__':
107     sys.exit(main())

```

源文件组 43

q4/baselines/CG/strategy.py

```

1 from cautious import Planner
2 import json
3 from pathlib import Path
4 VERSION='p4-literature-cautious-beta005-v1'
5 POLICIES=json.loads(Path(__file__).with_name('policies.json').read_text(encoding='utf-8'))

```

源文件组 44

q4/baselines/M07/run_robot.py

```

1 """ 问题四混合全向/定向源 M16 后验规划策略；尚无官方实测。 """
2 import argparse
3 import datetime as dt
4 import hashlib
5 import json
6 from pathlib import Path
7 import shutil
8 import sys
9 import time
10 import uuid
11
12 ROOT = Path(__file__).resolve().parent
13
14
15 def main():
16     if hasattr(sys.stdout, 'reconfigure'):
17         sys.stdout.reconfigure(encoding='utf-8')
18     parser = argparse.ArgumentParser(description=__doc__)
19     parser.add_argument('--policy', choices=['baseline'], default='baseline', help='默认第四问 M16 后验规划策略')
20     parser.add_argument('--team', help='默认复用旧策略的 config.json 队号')
21     parser.add_argument('--port', type=int, help='默认复用旧配置，缺省端口 2026')
22     parser.add_argument('--config', type=Path, help='指定含 team_no、port 的配置文件')
23     parser.add_argument('--check', action='store_true', help='仅检查依赖和配置，不连接模拟器')
24     args = parser.parse_args()
25     try:
26         import numpy
27         import scipy
28         import shapely
29         from api_client import Robot, BudgetReached
30         from strategy import Planner, VERSION, POLICIES
31     except ImportError as error:
32         print(f'缺少依赖：{error}\n 请安装 requirements.txt 中的依赖后重试。')
33         return 2
34     paths = [args.config] if args.config else [ROOT / 'config.json', ROOT / 'config.json']
35     config_path = next((p for p in paths if p and p.is_file()), None)
36     config = json.loads(config_path.read_text(encoding='utf-8-sig')) if config_path else {}
37     team, port = args.team or config.get('team_no'), args.port or config.get('port', 2026)
38     if not team:
39         print('找不到队号配置，请使用 --config 指定旧 config.json，或使用 --team 参数。')
40         return 2
41     if args.check:
42         print(f'策略 {args.policy}；第四问覆盖改进版，仅完成本地验证。')
43         print(f'自检通过：Python {sys.version.split()[0]} / numpy {numpy.__version__} / scipy {scipy.__version__}')
44         print(f'配置可读；端口 {port}；未连接模拟器。')
45         return 0
46     print(f'问题 4 研究策略 {VERSION} / {args.policy} | 端口 {port}')
47     print('这是第四问覆盖改进版研究候选，尚无官方演练成绩。')
48     print('请在模拟器选择【问题 4 演练测试】，开始后等待“接口已就绪/等待机器狗进入”。')
49     print('接口无法查询演练/正式模式，程序按问题 4 混合全向/定向干扰器规则工作。')
50     input('界面就绪后按 Enter 开始：')

```

```

51 folder = ROOT / 'runs' / (dt.datetime.now().strftime('%Y%m%d-%H%M%S') + '-' + uuid.uuid4().hex[:6])
52 robot = Robot(team, f'http://127.0.0.1:{port}', folder)
53 planner = Planner(robot, policy=args.policy)
54 snapshot = folder / 'source_snapshot'
55 snapshot.mkdir()
56 hashes = {}
57 for p in list(ROOT.glob('*.*py')) + [p for p in ROOT.glob('*.*json') if p.name != 'config.json']:
58     shutil.copy2(p, snapshot / p.name)
59     hashes[p.name] = hashlib.sha256(p.read_bytes()).hexdigest()
60 (folder / 'manifest.json').write_text(json.dumps(dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy],
61     python=sys.version, numpy=numpy.__version__, scipy=scipy.__version__, shapely=shapely.__version__, source_sha256=hashes,
62     config_path=str(config_path), mode='user_selected_P4_practice_not_API_verified'),
63     ensure_ascii=False, indent=2), encoding='utf-8')
64 outcome, error_text, entered = 'completed', None, False
65 robot.started = time.monotonic()
66 try:
67     robot.call('/enter')
68     entered = True
69     planner.run()
70 except BudgetReached as error:
71     outcome, error_text = 'budget_stop', str(error)
72 except (Exception, KeyboardInterrupt) as error:
73     outcome, error_text = 'error', str(error) or '用户中断'
74     # Interruption during HTTP could leave execution state unknown.
75     if isinstance(error, KeyboardInterrupt):
76         robot.transport_uncertain = True
77 finally:
78     if entered and not robot.transport_uncertain:
79         try:
80             robot.call('/exit')
81         except Exception as error:
82             outcome = 'exit_unconfirmed'
83             error_text = (error_text or '') + '\n退出未确认: ' + str(error)
84     count = len(robot.cleared)
85     summary = dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy], outcome=outcome, error=error_text,
86         exit_confirmed=robot.exit_confirmed, transport_uncertain=robot.transport_uncertain,
87         cleared_count=count, cleared_channels=sorted(robot.cleared),
88         observed_but_uncleared=sorted(robot.discovered - robot.cleared),
89         measure_count=robot.measure_count, clear_failures=robot.clear_failures,
90         scanned_sites=robot.scanned_sites, distance_m=robot.distance_m, switches=robot.switches,
91         virtual_time_s=robot.virtual, average_clear_time_s=robot.virtual / count if count else None,
92         local_elapsed_s=time.monotonic() - robot.started,
93         actual_jammer_count=None, completion=planner.certificate(),
94         note='实际目标总数和正式成绩以模拟器为准; 完整性证书以问题 4 半圆定向/全向与 R>1000 规则为前提。')
95     (folder / 'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
96     robot.log.close()
97     print(f'\n结束: 清除 {count} 个; 虚拟时间 {robot.virtual:.1f} 秒; 移动 {robot.distance_m:.1f} 米。')
98     print(f'状态 {outcome}; 退出确认 {robot.exit_confirmed}; 结束依据 {planner.stop_reason}')
99     if error_text:
100         print(error_text)
101     print(f'运行记录: {folder}')
102     print('请保存模拟器结束界面、案例编码、实际目标数, 并导出官方 .jlog。')
103     return 0 if outcome == 'completed' and robot.exit_confirmed else 1
104
105
106 if __name__ == '__main__':
107     sys.exit(main())

```

源文件组 45

q4/baselines/M07/strategy.py

```

1 """Frozen J500 planner with an official HTTP-compatible lifecycle wrapper."""
2 import json
3 from pathlib import Path
4 from research_strategy import Planner as Integrated
5
6 VERSION = 'p4-m07-portfolio-official-v1'
7 POLICIES = json.loads(Path(__file__).with_name('policies.json').read_text(encoding='utf-8'))

```

```

8
9
10 class Planner(Integrated):
11     def __init__(self, robot, *, policy='baseline'):
12         if policy != 'baseline':
13             raise ValueError(policy)
14         super().__init__(robot, 'baseline')
15         self.stop_reason = None
16
17     def run(self):
18         try:
19             self.stop_reason = super().run()
20         finally:
21             self.robot.scanned_sites = len(self.visited)
22         return self.stop_reason
23
24     def certificate(self):
25         return dict(problem=4, reason=self.stop_reason, complete=bool(self.stop_reason),
26                     cleared_channels=sorted(self.robot.cleared),
27                     observed_but_uncleared=sorted(set(self.tracks) - set(self.robot.cleared)),
28                     mesh_points=self.sites.tolist(), visited_sites=sorted(self.visited),
29                     negative_counts={str(ch):len(points) for ch,points in self.negatives.items()},
30                     deferred=self.deferred, shared_measures=self.shared_measures,
31                     optical_fallbacks=self.fallbacks, known_channel_radio_losses=self.lost,
32                     assumptions='M16: fixed 22-site continuous coverage, posterior planning only, original MEC and finite optical clearance')

```

源文件组 46

q4/M32/joint_planner.py

```

1  """M16 with certified projection of joint radius/heading constraints."""
2  from research_strategy import Planner as M16
3  from joint_region import tighten
4
5  class Planner(M16):
6      def __init__(self, robot, policy='joint'):
7          if policy not in ['baseline', 'joint']: raise ValueError(policy)
8          super().__init__(robot, 'both'); self.joint_policy = policy
9
10     def observe(self, q, ch):
11         z = super().observe(q, ch)
12         if self.joint_policy == 'baseline' or ch not in self.tracks: return z
13         t = self.tracks[ch]
14         if not any(h['measure_result'] == 'no_signal' for h in t.history): return z
15         old = t.region; old_radius = t.radius
16         new, proof = tighten(old, t.history)
17         if not proof['rejected']: return z
18         t.region = new; t.refresh()
19         self.robot.record(dict(kind='joint_region_certificate', channel=ch, history=t.history.copy(), old_region=old.wkt, new_region=new.wkt, old_radius=old_r,
20                                ↪ radius=new_radius=t.radius, proof=proof))
21         self.robot.record(dict(kind='region', channel=ch, wkt=t.region.wkt, radius_m=t.radius))
22     return z

```

源文件组 47

q4/M32/joint_region.py

```

1  """Conservative cell rejection for joint position/radius/heading feasibility.
2  Closed relaxed arcs retain zero-measure and boundary possibilities.
3  """
4  import math
5  import numpy as np
6  from shapely.geometry import box
7  from shapely.affinity import affine_transform
8  from shapely.ops import unary_union
9  TAU = 2 * math.pi
10

```

```

11 def arc_intersection(arcs,angle,width):
12     if width>=math.pi:return arcs
13     lo=(angle-width)%TAU;hi=lo+2*width
14     pieces=[[lo,min(hi,TAU)]]
15     if hi>=TAU:pieces.append((0,hi-TAU))
16     return [(max(a,c),min(b,d)) for a,b in arcs for c,d in pieces if max(a,c)<=min(b,d)]
17
18 def reject(center,rho,history):
19     positive=[h for h in history if h['measure_result']!='no_signal']
20     negative=[h for h in history if h['measure_result']=='no_signal']
21     pd=[(np.asarray(h['point'])-center) for h in positive]
22     lower=max([1000.]+[float(np.linalg.norm(d))-rho for d in pd])
23     if lower>1500.+1e-5:return dict(reason='radius impossible',lower_radius=lower)
24     forced=[j for j,h in enumerate(negative) if np.linalg.norm(np.asarray(h['point'])-center)+rho<lower-1e-5]
25     if not forced:return None # Omnidirectional compatibility not ruled out.
26     arcs=[[0,TAU]]
27     for d in pd+[center-np.asarray(negative[j]['point']) for j in forced]:
28         distance=float(np.linalg.norm(d))
29         if distance<=rho+1e-8:continue
30         width=math.pi/2+math.asin(min(1.,rho/distance))+1e-9
31         arcs=arc_intersection(arcs,math.atan2(d[1],d[0]),width)
32         if not arcs:return dict(reason='joint heading impossible',lower_radius=lower,forced_negative_indices=forced)
33     return None
34
35 def tighten(region,history,resolution=10.,max_boxes=4096):
36     if not any(h['measure_result']!='no_signal' for h in history):return region,dict(boxes=0,rejected=[])
37     rectangle=np.array(region.minimum_rotated_rectangle.exterior.coords[-1]);u=rectangle[1]-rectangle[0];u/=max(np.linalg.norm(u),1e-12);v=np.array([-u[1],_
↪ u[0]])
38     transformed=affine_transform(region,[u[0],u[1],v[0],v[1],0,0]);queue=[transformed.boundaries];removed=[];certificates=[];count=0
39     while queue and count<max_boxes:
40         bounds=queue.pop();g=transformed.intersection(box(*bounds))
41         if g.is_empty or g.area<=1e-12:continue
42         a,b,c,d=g.boundaries;center=(np.array([(a+c)/2,(b+d)/2])@np.array([u,v]));rho=math.hypot(c-a,d-b)/2+2e-6;count+=1
43         proof=reject(center,rho,history)
44         if proof is not None:
45             # The padded rectangle is INSIDE the certified disk. Its overlap
46             # avoids coincident polygon boundary fragments from separate clips.
47             world=affine_transform(box(a-1e-6,b-1e-6,c+1e-6,d+1e-6),[u[0],v[0],u[1],v[1],0,0]);removed.append(world)
48             certificates.append(dict(center=center.tolist(),radius=rho,wkt=world.wkt,**proof));continue
49         if max(c-a,d-b)<=resolution:continue
50         if c-a>=d-b:
51             middle=(a+c)/2;queue.extend([(a,b,middle,d),(middle,b,c,d)])
52         else:
53             middle=(b+d)/2;queue.extend([(a,b,c,middle),(a,middle,c,d)])
54     result=region.difference(unary_union(removed)) if removed else region
55     if result.is_empty:raise ArithmeticError('joint feasible region unexpectedly empty')
56     if not result.is_valid:raise ArithmeticError('joint feasible region invalid topology')
57     return result,dict(boxes=count,budget_remaining_boxes=len(queue),rejected=certificates)

```

源文件组 48

q4/M32/run_robot.py

```

1  """ 问题四 M32 joint 正式测试入口；冻结策略已完成 30 场官方演练。 """
2  import argparse
3  import datetime as dt
4  import hashlib
5  import json
6  from pathlib import Path
7  import shutil
8  import sys
9  import time
10 import uuid
11
12 ROOT = Path(__file__).resolve().parent
13
14
15 def main():
16     if hasattr(sys.stdout, 'reconfigure'):

```

```

17     sys.stdout.reconfigure(encoding='utf-8')
18     parser = argparse.ArgumentParser(description=__doc__)
19     parser.add_argument('--policy', choices=['joint'], default='joint', help='第四问 M32 joint 联合可行域策略')
20     parser.add_argument('--team', help='默认复用旧策略的 config.json 队号')
21     parser.add_argument('--port', type=int, help='默认复用旧配置, 缺省端口 2026')
22     parser.add_argument('--config', type=Path, help='指定含 team_no、port 的配置文件')
23     parser.add_argument('--check', action='store_true', help='仅检查依赖和配置, 不连接模拟器')
24     args = parser.parse_args()
25     try:
26         import numpy
27         import scipy
28         import shapely
29         from api_client import Robot, BudgetReached
30         from strategy import Planner, VERSION, POLICIES
31     except ImportError as error:
32         print(f'缺少依赖: {error}\n 请安装 requirements.txt 中的依赖后重试。')
33         return 2
34     paths = [args.config] if args.config else [ROOT / 'config.json', ROOT.parent / 'simple-strategy' / 'config.json']
35     config_path = next((p for p in paths if p and p.is_file()), None)
36     config = json.loads(config_path.read_text(encoding='utf-8-sig')) if config_path else {}
37     team, port = args.team or config.get('team_no'), args.port or config.get('port', 2026)
38     if not team:
39         print('找不到队号配置, 请使用 --config 指定旧 config.json, 或使用 --team 参数。')
40         return 2
41     if args.check:
42         print(f'策略 {args.policy}; 第四问 M32 joint; 已完成 30 场官方演练。')
43         print(f'自检通过: Python {sys.version.split()[0]} / numpy {numpy.__version__} / scipy {scipy.__version__}')
44         print(f'配置可读; 端口 {port}; 未连接模拟器。')
45         return 0
46     print(f'问题 4 研究策略 {VERSION} / {args.policy} | 端口 {port}')
47     print('这是已完成 30 场官方演练的冻结 M32 joint 策略。')
48     print('请在模拟器选择 【问题 4 正式测试】, 开始后等待至少 5 秒, 确认 “等待机器狗进入”。')
49     print('接口无法查询演练/正式模式, 程序按问题 4 混合全向/定向干扰器规则工作。')
50     input('界面就绪后按 Enter 开始: ')
51     folder = ROOT / 'runs' / (dt.datetime.now().strftime('%Y%m%d-%H%M%S') + '-' + uuid.uuid4().hex[6])
52     robot = Robot(team, f'http://127.0.0.1:{port}', folder)
53     planner = Planner(robot, policy=args.policy)
54     snapshot = folder / 'source_snapshot'
55     snapshot.mkdir()
56     hashes = {}
57     for p in list(ROOT.glob('*.py')) + [p for p in ROOT.glob('*.json') if p.name != 'config.json']:
58         shutil.copy2(p, snapshot / p.name)
59         hashes[p.name] = hashlib.sha256(p.read_bytes()).hexdigest()
60     (folder / 'manifest.json').write_text(json.dumps(dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy],
61     python=sys.version, numpy=numpy.__version__, scipy=scipy.__version__, shapely=shapely.__version__, source_sha256=hashes,
62     config_path=str(config_path), mode='user_selected_P4_formal_not_API_verified'),
63     ensure_ascii=False, indent=2), encoding='utf-8')
64     outcome, error_text, entered = 'completed', None, False
65     robot.started = time.monotonic()
66     try:
67         robot.call('/enter')
68         entered = True
69         planner.run()
70     except BudgetReached as error:
71         outcome, error_text = 'budget_stop', str(error)
72     except (Exception, KeyboardInterrupt) as error:
73         outcome, error_text = 'error', str(error) or '用户中断'
74         # Interruption during HTTP could leave execution state unknown.
75         if isinstance(error, KeyboardInterrupt):
76             robot.transport_uncertain = True
77     finally:
78         if entered and not robot.transport_uncertain:
79             try:
80                 robot.call('/exit')
81             except Exception as error:
82                 outcome = 'exit_unconfirmed'
83                 error_text = (error_text or '') + '\n 退出未确认: ' + str(error)
84     count = len(robot.cleared)
85     summary = dict(strategy=VERSION, policy=args.policy, policy_parameters=POLICIES[args.policy], outcome=outcome, error=error_text,
86     exit_confirmed=robot.exit_confirmed, transport_uncertain=robot.transport_uncertain,
87     cleared_count=count, cleared_channels=sorted(robot.cleared),

```

```

88         observed_but_uncleared=sorted(robot.discovered - robot.cleared),
89         measure_count=robot.measure_count, clear_failures=robot.clear_failures,
90         scanned_sites=robot.scanned_sites, distance_m=robot.distance_m, switches=robot.switches,
91         virtual_time_s=robot.virtual, average_clear_time_s=robot.virtual / count if count else None,
92         local_elapsed_s=time.monotonic() - robot.started,
93         actual_jammer_count=None, completion=planner.certificate(),
94         note='实际目标总数和正式成绩以模拟器为准；完整性证书以问题 4 半圆定向/全向与  $R \geq 1000$  规则为前提。')
95     (folder / 'summary.json').write_text(json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
96     robot.log.close()
97     print(f'\n 结束：清除 {count} 个；虚拟时间 {robot.virtual:.1f} 秒；移动 {robot.distance_m:.1f} 米。')
98     print(f'状态 {outcome}；退出确认 {robot.exit_confirmed}；结束依据 {planner.stop_reason}')
99     if error_text:
100         print(error_text)
101     print(f'运行记录：{folder}')
102     print('请保存模拟器结束界面、案例编码、实际目标数，并导出官方 .jlog。')
103     return 0 if outcome == 'completed' and robot.exit_confirmed else 1
104
105
106 if __name__ == '__main__':
107     sys.exit(main())

```

源文件组 49

q4/M32/strategy.py

```

1  """Frozen J500 planner with an official HTTP-compatible lifecycle wrapper."""
2  import json
3  from pathlib import Path
4  from joint_planner import Planner as Integrated
5
6  VERSION = 'p4-m32-frozen-official-20260912'
7  POLICIES = json.loads(Path(__file__).with_name('policies.json').read_text(encoding='utf-8'))
8
9
10 class Planner(Integrated):
11     def __init__(self, robot, *, policy='joint'):
12         if policy != 'joint':
13             raise ValueError(policy)
14         super().__init__(robot, 'joint')
15         self.stop_reason = None
16
17     def run(self):
18         try:
19             self.stop_reason = super().run()
20         finally:
21             self.robot.scanned_sites = len(self.visited)
22         return self.stop_reason
23
24     def certificate(self):
25         return dict(problem=4, reason=self.stop_reason, complete=bool(self.stop_reason),
26                    cleared_channels=sorted(self.robot.cleared),
27                    observed_but_uncleared=sorted(set(self.tracks) - set(self.robot.cleared)),
28                    mesh_points=self.sites.tolist(), visited_sites=sorted(self.visited),
29                    negative_counts={str(ch):len(points) for ch,points in self.negatives.items()},
30                    deferred=self.deferred, shared_measures=self.shared_measures,
31                    optical_fallbacks=self.fallbacks, known_channel_radio_losses=self.lost,
32                    assumptions='m32/joint: frozen research policy; original certified coverage and finite optical clearance')

```

源文件组 50

verify_materials.py

```

1  """Read-only support-file verification. Never connects to the simulator."""
2  from pathlib import Path
3  import ast
4  import hashlib
5  import json

```

```

6
7 ROOT=Path(__file__).resolve().parent
8 def read(p):return json.loads(p.read_text(encoding='utf-8-sig'))
9 def digest(p):return hashlib.sha256(p.read_bytes()).hexdigest()
10
11 def main():
12     sums=ROOT/'SHA256SUMS.json'
13     if sums.exists():
14         for name,wanted in read(sums)['files'].items():
15             assert digest(ROOT/name)==wanted, f'Hash mismatch: {name}'
16     for p in ROOT.rglob('*.*py'):
17         if '__pycache__' not in p.parts:ast.parse(p.read_text(encoding='utf-8-sig'),filename=str(p.relative_to(ROOT)))
18     rows=read(ROOT/'formal-results.json')
19     assert len(rows)==6
20     assert len(list(ROOT.glob('q*/formal/logs/*.jlog')))==6
21     for r in rows:
22         base=ROOT/f"q{r['problem']}/formal"
23         assert digest(base/r['official_log'])==r['official_log_sha256']
24         actions=[json.loads(x) for x in (base/r['actions']).read_text(encoding='utf-8').splitlines()]
25         requests={e['payload']['request_id']:e for e in actions if e['kind']=='request'}
26         accepted=[e for e in actions if e['kind']=='response' and e.get('response',{}).get('accepted')]
27         enters=[e['response'] for e in accepted if requests[e['request_id']]['path']=='/enter']
28         exits=[e['response'] for e in accepted if requests[e['request_id']]['path']=='/exit']
29         assert len(enters)==len(exits)==1
30         cleared={requests[e['request_id']]['payload']['channel'] for e in accepted
31                  if requests[e['request_id']]['path']=='/clear' and e['response'].get('clear_result')=='success'}
32         assert len(cleared)==r['cleared_count']
33         assert abs(exits[0]['virtual_time_s']-r['virtual_time_s'])<1e-6
34         assert abs((exits[0]['real_timestamp_ms']-enters[0]['real_timestamp_ms'])/1000-r['program_runtime_s'])<1e-9
35         assert abs(r['virtual_time_s']/r['cleared_count']-r['average_clear_time_s'])<1e-9
36         assert all(e['payload'].get('robot_id')=='ANONYMIZED' for e in requests.values())
37         assert all(e['request_id'] in requests for e in actions if e['kind']=='response')
38     a=read(ROOT/'q3/analysis/data/analysis.json')
39     assert a['official']['literature']['runs']==30
40     assert a['official']['v4']['runs']==a['official']['v6']['runs']==50
41     assert a['local']['paired_successes']==30
42     b=read(ROOT/'q4/analysis/evidence/official_rows.json')
43     assert len(b)==90
44     for name in ('cautious','m07','m32'):assert sum(x['strategy']==name for x in b)==30
45     for p in ROOT.rglob('config.json'):raise AssertionError(f'Private config must not be published: {p.name}')
46     print('PASS: file hashes, complete Python syntax, 6 original official logs, official timestamps/time ratios, anonymous request-response links, and practice
47           ↪ sample counts.')
48 if __name__=='__main__':main()

```